LinkStudio

A modular physical-layer communication simulator

Theory and User Manual



Wireless · Wireline · Optical · Coding · Synchronization · MIMO

Engine v0.4 · October 24, 2026

Contents

About this manual vi

I Using and extending LinkStudio 1

1 Overview and quick start 2
1.1 What LinkStudio is . . . 2
1.2 Installation . . . 3
1.3 Three ways to run a simulation . . . 3
1.4 Repository map . . . 4

2 The simulation engine 5
2.1 Signals . . . 5
2.2 Blocks . . . 5
2.3 The registry . . . 7
2.4 Graphs . . . 7
2.5 Subsystems . . . 9
2.6 Simulations and stop criteria . . . 9
2.7 Sweeps . . . 9
2.8 Metadata propagation . . . 10
2.9 Scenarios . . . 11

3 Graphical editor, API and command line 12
3.1 The graphical editor . . . 12
3.2 The HTTP and WebSocket API . . . 13
3.3 Command line . . . 14

4 Developer guide: writing blocks 15
4.1 A complete example: free-space path loss . . . 15
4.2 Conventions . . . 16
4.3 Testing a block against theory . . . 16
4.4 Adding a scenario . . . 17

II Theory and implementation 18

5 Baseband signal model, noise and Monte-Carlo estimation 19
5.1 The complex envelope . . . 19
5.2 Additive white Gaussian noise . . . 19
5.3 Monte-Carlo estimation of error rates . . . 21
5.4 Closed-form references . . . 22

6 Digital modulation and demodulation 23
6.1 Constellations . . . 23
6.2 Optimal detection . . . 24
6.3 Error probability over AWGN . . . 25

7 Pulse shaping and matched filtering **26**
7.1 Linear modulation . . . 26
7.2 The Nyquist criterion . . . 26
7.3 The matched filter . . . 26
7.4 Discrete-time implementation . . . 27
7.5 Eye diagrams . . . 28

8 Radio front-end impairments **30**
8.1 Carrier frequency and phase offsets . . . 30
8.2 Oscillator phase noise . . . 30
8.3 IQ imbalance . . . 31
8.4 Power amplifier nonlinearity . . . 31
8.5 Analog-to-digital conversion . . . 32
8.6 Timing offset . . . 33

9 Fading channels and equalization **34**
9.1 The multipath channel . . . 34
9.2 Rayleigh and Rician fading . . . 34
9.3 Tapped delay lines and 3GPP profiles . . . 36
9.4 Error probability with fading . . . 36
9.5 Equalization . . . 36

10 Synchronization **39**
10.1 The preamble . . . 39
10.2 Symbol timing recovery . . . 39
10.3 Frame synchronization and coarse CFO . . . 42
10.4 Carrier phase tracking . . . 42

11 OFDM **44**
11.1 Cyclic prefix and circular convolution . . . 44
11.2 Subcarrier layout . . . 45
11.3 Channel estimation and equalization . . . 45
11.4 Blind timing and frequency synchronization . . . 46
11.5 Peak-to-average power ratio . . . 47

12 Multiple antennas **49**
12.1 The flat-fading MIMO channel . . . 49
12.2 Detection for spatial multiplexing . . . 49
12.3 Diversity . . . 50

13 Channel coding **52**
13.1 Cyclic redundancy check . . . 52
13.2 LFSR sequences and scrambling . . . 52
13.3 Interleaving . . . 53
13.4 Block codes: repetition and Hamming . . . 53
13.5 Convolutional codes and the Viterbi algorithm . . . 53
13.6 LDPC codes . . . 55
13.7 Polar codes . . . 57
13.8 Comparison . . . 58

14 Wireline communications **60**
14.1 Line codes . . . 60
14.2 The cable as a transmission line . . . 60
14.3 Crosstalk . . . 62
14.4 Full duplex: hybrid echo and echo cancellation . . . 62
14.5 SerDes equalization . . . 63
14.6 Discrete multitone (DSL) . . . 64

15 Optical communications **66**
15.1 Transmitters . . . 66
15.2 Propagation in single-mode fiber . . . 68
15.3 Optical amplification and OSNR . . . 71
15.4 Wavelength-division multiplexing . . . 73
15.5 Free-space optics . . . 73
15.6 Direct detection . . . 74
15.7 Coherent detection . . . 75

16 Validation **79**
16.1 Running the checks . . . 79
16.2 Physics checks . . . 79
16.3 Known limitations . . . 80

III Reference **82**

A Block reference **83**
A.1 SOURCE . . . 83
A.2 MODULATION . . . 83
A.3 WAVEFORM . . . 83
A.4 CHANNEL . . . 84
A.5 SYNC . . . 85
A.6 RECEIVER . . . 85
A.7 MIMO . . . 86
A.8 CODING . . . 86
A.9 WIRELINE . . . 87
A.10 OPTICAL . . . 88
A.11 METRICS . . . 90

B Metadata keys **91**

C Scenario library **93**

Bibliography **95**

About this manual

LinkStudio is a block-based simulator of the physical layer (PHY) of communication systems. A simulation is a directed graph of blocks: sources, modulators, pulse shapers, channels, synchronizers, equalizers, decoders and meters. The same engine covers radio links, copper lines and optical fiber. It is driven from Python, from YAML scenario files, from the command line or from a browser-based editor.

This manual has two goals and is organized accordingly.

Part I – Using and extending LinkStudio. How the engine works (signals, blocks, graphs, Monte-Carlo runs, sweeps, scenarios) and how to drive it from the GUI, the CLI and the Python API. It ends with a developer guide for writing new blocks.

Part II – Theory and implementation. One chapter per subsystem. Every chapter first develops the theory with the notation used in the code, then shows *where* and *how* it is implemented, and finally validates the implementation against a closed-form result. Part II can be read as a compact textbook on digital transmission whose equations can all be checked in the code.

How to read the chapters of Part II

Each theory chapter follows the same pattern:

1. **Model.** The physical or mathematical model, derived with the assumptions stated explicitly.
2. **Implementation.** A blue *In the code* box names the blocks, files and metadata involved. Short excerpts show the lines that realize the key equations, for example the split-step solver of the nonlinear Schrödinger equation (Listing 15.5). Excerpts are extracted automatically from the source, so their file names and line numbers match the code base this manual was built from.
3. **Validation.** A figure comparing the simulator with theory. All measured curves in this manual (markers in the plots) are produced by running the real blocks with `manual/tools/generate.py`; solid lines are closed-form results.

Green *Key idea* boxes summarize a result; orange *Watch out* boxes point at common mistakes or at modelling limitations.

Typographic conventions

`awgn`	a block type (as written in scenarios and in the GUI palette)
`snr_db`	a block parameter
`noise_var`	a metadata key travelling with a signal
`linkstudio/phy/optical.py`	a file, relative to `backend/`
SAMPLES	a signal kind (Section 2.1)

Notation

$x(t)$, $x[n]$	complex envelope (continuous, sampled)	f_s	sample rate
T, $R_s = 1/T$	symbol period, symbol rate	N_{sps}	samples per symbol
E_s, E_b	energy per symbol, per information bit	N_0	one-sided noise PSD
M, $k = \log_2 M$	constellation size, bits per symbol	R	code rate
$\mathcal{CN}(\mu, \sigma^2)$	circular complex Gaussian	$\mathrm{Q}(x)$	Gaussian tail function
$(\cdot)^*$, $(\cdot)^{\mathsf{T}}$, $(\cdot)^{\mathsf{H}}$	conjugate, transpose, Hermitian	$\star$	convolution
$\mathcal{F}\{\cdot\}$	Fourier transform	$\mathbb{E}[\cdot]$	expectation

Fourier transforms follow the NumPy sign convention used everywhere in the code:

$$X(\omega) = \int x(t)\, e^{-j\omega t}\, dt, \qquad x(t) = \frac{1}{2\pi} \int X(\omega)\, e^{j\omega t}\, d\omega. \tag{1}$$

This convention fixes the signs of the dispersion operator and of the nonlinear phase in the fiber model (Section 15.2). The discrete transforms are `np.fft.fft` and `np.fft.ifft`; they are unitary (`norm="ortho"`) in OFDM and DMT.

Rebuilding this manual

```
python manual/tools/generate.py      # run the simulator, extract excerpts and tables
cd manual && latexmk -pdf linkstudio-manual.tex
```

Re-running the generator after a code change updates every figure, excerpt and reference table, so the manual stays consistent with the code.

Part I

Using and extending LinkStudio

Chapter 1

Overview and quick start

1.1 What LinkStudio is

LinkStudio simulates the physical layer of a communication link at the level of *complex-envelope samples*. A link is described as a graph of blocks. Every block is a small, self-contained model (a QAM mapper, an 80-km fiber span, a Viterbi decoder, a BER meter) with typed inputs and outputs and validated parameters. The engine executes the graph on *batches* of independent frames and accumulates metrics until a stop criterion is met.

Figure 1.1 shows the layers of the code base. They depend on each other only downwards:

- the **core** knows nothing about communications;
- the **PHY library** knows nothing about the user interface;
- the **API** turns graphs and results into JSON for the browser front end.

The same scenario therefore runs identically from a Python script, the CLI or the GUI.

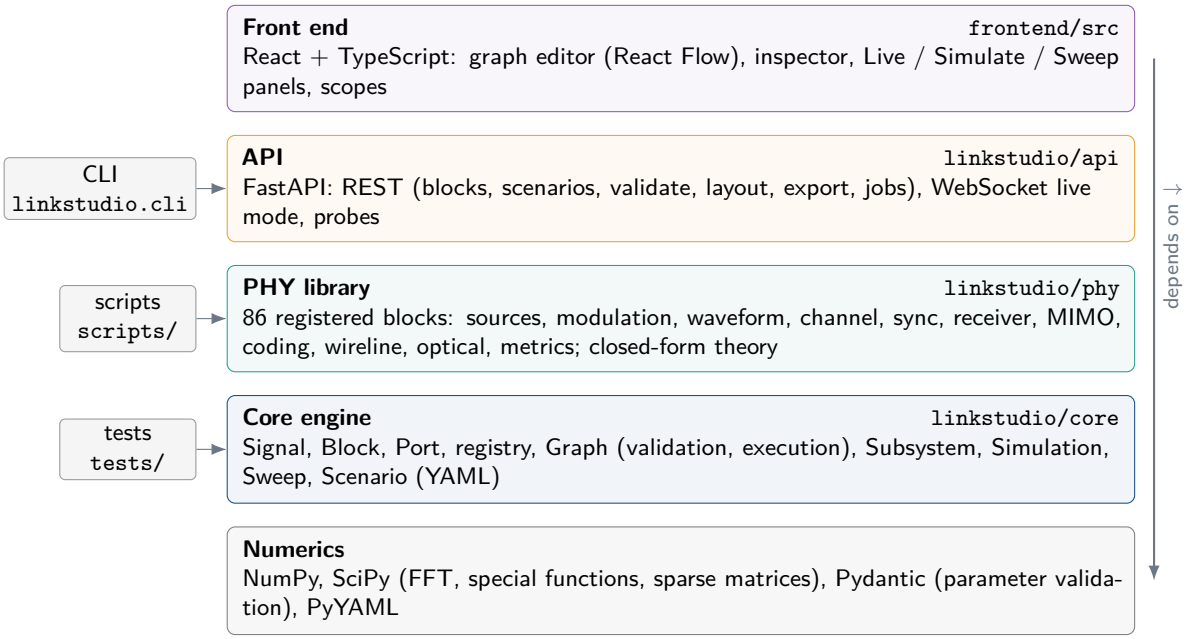


Figure 1.1: Layers of LinkStudio. Paths are relative to the repository; Python packages live in `backend/`.

1.1.1 Capabilities at a glance

Domain	Models
Modulation	PSK, square QAM, PAM, custom constellations with Gray labels; hard, exact and max-log soft demapping
Waveforms	RRC/RC/Gaussian/rectangular pulses, matched filter, rate changes, OFDM with pilots and CP, DMT with water-filling
Channels	AWGN (SNR, E_s/N_0, E_b/N_0), Rayleigh/Rician flat fading, Jakes Doppler, 3GPP EPA/EVA/ETU, MIMO with antenna correlation
RF front end	carrier offset, phase noise, IQ imbalance, PA models (Rapp, Saleh, clipping), quantizer, timing offset
Synchronization	Schmidl–Cox frame sync, Gardner timing loop with Oerder–Meyr start, decision-directed PLL, CP-based OFDM sync
Receivers	one-tap, ZF/MMSE FIR, LMS/NLMS, FFE+DFE, CTLE, echo canceller, ZF/MMSE/ML MIMO detection, Alamouti
Coding	CRC, scrambler, interleavers, repetition, Hamming, convolutional + Viterbi, puncturing, QC-LDPC (BP), polar (SC)
Wireline	NRZ, Manchester, AMI, MLT-3, 4B/5B, 64B/66B, skin-effect cables, ANSI NEXT/FEXT, impulse noise, jitter, ADSL/VDSL DMT
Optical	laser/EML with chirp and RIN, IQ Mach–Zehnder, NLSE/Manakov fiber with PMD, EDFA with ASE, WDM, PIN/APD, coherent receiver and DSP, FSO turbulence
Metrics	BER with confidence interval, SER, FER/BLER, EVM/MER, power/PAPR, OSNR, iteration counts

1.2 Installation

LinkStudio needs Python 3.11 or newer. The front end is already built in `frontend/dist`; Node.js is only needed to modify it.

```
cd backend
pip install -e ".[ui,viz,dev]"    # engine + web server + plotting + tests
python -m pytest                   # the test-suite (about 170 tests)
python scripts/main.py --quick     # runs every demo and compares with theory
```

1.3 Three ways to run a simulation

1. The graphical editor. `python -m linkstudio serve` opens `http://127.0.0.1:8000`. Pick a scenario from the browser (Chapter 3), press *Start live* and move the `snr_db` slider of the `awgn` block: constellation, spectrum and BER update while the simulation runs.

2. A YAML scenario from the command line. Scenario files describe the graph and how to run it (Section 2.9):

```
python -m linkstudio run scenarios/tutorials/01_bpsk_awgn.yaml
python -m linkstudio blocks              # list all block types
python -m linkstudio describe fiber      # parameters and ports of one block
```

3. The Python API. The snippet below builds the graph of Figure 1.2 and measures the BER of QPSK at $E_b/N_0 = 6\,\mathrm{dB}$. The result is about 2.4×10^{-3}, the value of $\mathrm{Q}(\sqrt{2E_b/N_0})$.

```
import linkstudio as cs
```

```
g = cs.Graph("qpsk")
g.create("src", "random_bits", n_bits=2000)
g.create("map", "mapper", scheme="psk", order=4)
g.create("awgn", "awgn", snr_db=6.0)                    # Eb/N0 by default
g.create("demap", "demapper", method="hard")
g.create("ber", "ber_meter")
g.chain("src", "map", "awgn", "demap", "ber.rx")      # default ports
g.connect("src", "ber.tx")                              # explicit edge

result = cs.Simulation(g, batch_size=64, seed=1).run(
    stop=cs.StopCriteria.errors("ber.bit_errors", 200))
print(result.summary())
```

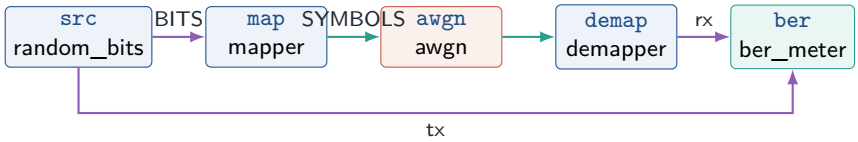


Figure 1.2: The graph built by the quick-start snippet. Edge colours follow the signal kind.

1.4 Repository map

`backend/linkstudio/core/`	engine: `signal.py`, `block.py`, `registry.py`, `graph.py`, `composite.py`, `simulation.py`, `sweep.py`, `scenario.py`
`backend/linkstudio/phy/`	block library; `dsp/` holds shared numerical helpers (filters, constellations, LFSR, analysis), `theory.py` closed-form references
`backend/linkstudio/api/`	web server: `app.py`, `editor.py` (GUI ↔ graph), `jobs.py`, `probes.py`
`backend/scenarios/`	ready-made scenarios grouped by category (Chapter C)
`backend/scripts/`	`main.py` and one demo per subsystem, each checking theory
`backend/tests/`	pytest suite (engine, DSP, physics, API, every scenario)
`frontend/`	React application
`docs/`	short Markdown notes (architecture, engine, GUI)
`manual/`	this manual and its generator

Chapter 2

The simulation engine

The engine (`linkstudio/core`) is about one thousand lines of Python and contains no communication theory. It provides five abstractions:

- *signals* that flow along edges;
- *blocks* that transform them;
- *graphs* that connect blocks;
- *simulations* that run graphs batch after batch;
- *sweeps* that repeat simulations over a parameter grid.

This chapter describes each of them and the rules they enforce.

2.1 Signals

A `Signal` (`core/signal.py`) is an immutable record with four fields:

`data` a NumPy array. Axis 0 is always the *batch* (independent frames simulated in parallel) and the last axis is time, bit index or subcarrier index. Multi-dimensional kinds add axes in between, as in Figure 2.1.

`kind` the semantic type of the data, a flag of the `Kind` enumeration (Table 2.1).

`fs` the sample or symbol rate in Hz, when it is meaningful.

`meta` a dictionary of metadata that travels downstream with the data (Section 2.8).

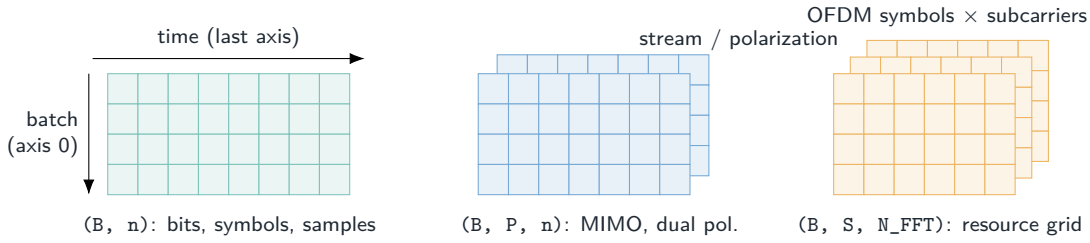


Figure 2.1: Array conventions. Every block may assume that axis 0 is the batch and the last axis is time (or frequency for grids).

Signals are never modified in place. A block derives a new signal with `x.evolve(new_data, key=value)`. This keeps the input's kind, rate and metadata and merges the new keys, so information added upstream is preserved unless a block deliberately overrides it.

2.2 Blocks

A block (`core/block.py`) declares three things:

- a `Params` model, a Pydantic class whose fields carry types, defaults, bounds and descriptions;
- named, typed `inputs` and `outputs` (`Port` objects);

Table 2.1: Signal kinds. Kinds are bit flags, so a port can accept several of them (`Kind.BITS | Kind.LLR`). The colours are those used by the GUI.

Kind		dtype	Meaning
BITS		`uint8`	hard bits in $\{0, 1\}$
LLR		`float64`	log-likelihood ratios, $L = \ln P(b{=}0)/P(b{=}1)$ (positive means 0)
SYMBOLS		`complex128`	one sample per symbol (constellation domain)
SAMPLES		`complex128/float64`	oversampled waveform; optical fields in $\sqrt{\mathrm{W}}$
GRID		`complex128`	OFDM resource grid (frequency domain)
CSI		`complex128`	channel state information; the form is given by `csi_type`
FLAGS		`bool`	one flag per frame (e.g. CRC check)

- a `process(inputs, ctx)` method mapping one batch of input signals to output signals.

Measurement blocks also override `metrics()`.

Ports carry the accepted (or produced) kinds, a `required` flag for inputs and, for outputs, an optional `same_as` reference: `Port.like("in")` produces the same kind as input `in`. The GUI uses this information to colour handles and to refuse invalid connections, and the graph validator uses it to propagate kinds (Section 2.4.1).

2.2.1 Life cycle

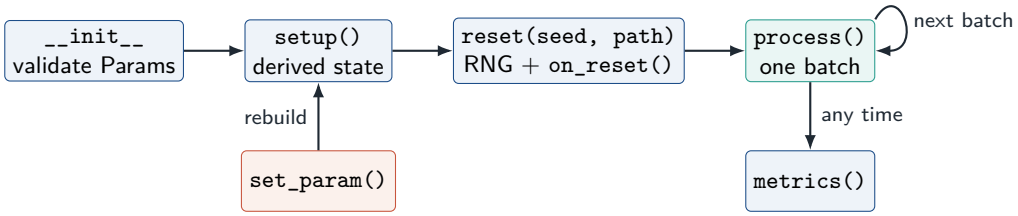


Figure 2.2: Block life cycle. `setup()` rebuilds derived state (filter taps, code matrices, trellises) after construction and after every parameter change. `on_reset()` clears accumulators before a run.

2.2.2 Reproducible randomness

Every block owns a private NumPy generator. On reset it is re-seeded from the master seed of the run *and* the block's hierarchical path:

Code 2.1: Seeding the random generator of a block. `linkstudio/core/block.py`, lines 102–110

```
102 def reset(self, seed: int, path: str) -> None:
103     """Re-seed the block random generator and clear runtime state.
104
105     The generator is derived from the master ``seed`` and the block ``path``
106     (its hierarchical name), so every block gets an independent but
107     reproducible random stream.
108     """
109     self.rng = np.random.default_rng([seed, zlib.crc32(path.encode())])
110     self.on_reset()
```

Because the CRC-32 of the path enters the seed sequence, two awgn blocks named differently draw independent noise. The same graph with the same seed produces bit-identical results, whatever the execution order of unrelated blocks and whether the run starts from the GUI, the CLI or a script.

2.3 The registry

`@register_block("type_name", category=...)` adds a class to a global registry (`core/registry.py`). Registered blocks can be created by name (`create_block`, `Graph.create`) and are described by `block_catalogue()`. The catalogue contains the documentation, the ports and the JSON Schema of the parameters. The GUI builds its palette and its parameter forms from this catalogue, and Chapter A of this manual is generated from it.

2.4 Graphs

A `Graph` holds named blocks and edges. Edges are stored as a map destination input $\mapsto$ source output, so every input has at most one driver, while an output may fan out to any number of inputs. Ports are addressed as `"block.port"`; omitting the port selects the block's first declared port, which makes `g.chain("src", "map", "awgn")` a compact way to wire a pipeline. Parameters are addressed as `"block.param"`, and `"/"` descends into subsystems (`"rx/demap.method"`).

2.4.1 Validation

`Graph.validate()` enforces three rules and returns the resolved kind of every output:

1. every *required* input is driven, either by an edge or, inside a subsystem, by an external input;
2. the graph is acyclic;
3. along every edge the produced kind intersects the accepted kinds.

Kinds are resolved in topological order. An output declared with `same_as` takes the kind actually delivered to the referenced input, intersected with the kinds that input accepts. An `interleaver` fed with LLRs therefore produces `LLR`, and the validator can reject an LLR output connected to a block that only accepts symbols, before anything is executed.

The topological order is computed with Kahn's algorithm:

Code 2.2: Topological sort of the block graph. `linkstudio/core/graph.py`, lines 193–215

```
193 def _toposort(self) -> list[str]:
194     succ: dict[str, set[str]] = {n: set() for n in self.blocks}
195     for dst, src in self.edges.items():
196         if dst.block == src.block:
197             raise GraphError(f"Block '{dst.block}' is connected to itself.")
198         succ[src.block].add(dst.block)
199     indeg = {n: 0 for n in self.blocks}
200     for targets in succ.values():
201         for t in targets:
202             indeg[t] += 1
203     ready = [n for n in self.blocks if indeg[n] == 0]
204     order: list[str] = []
205     while ready:
206         n = ready.pop(0)
207         order.append(n)
208         for t in sorted(succ[n], key=list(self.blocks).index):
```

```
            indeg[t] -= 1
            if indeg[t] == 0:
                ready.append(t)
    if len(order) != len(self.blocks):
        cyclic = [n for n in self.blocks if n not in order]
        raise GraphError(f"The graph contains a cycle involving: {cyclic}")
    return order
```

Ties are broken by insertion order, so the execution order is deterministic. Feedback loops (PLLs, adaptive filters, decision feedback) live *inside* blocks, where they can run at the sample rate. The graph itself carries one batch at a time from sources to sinks.

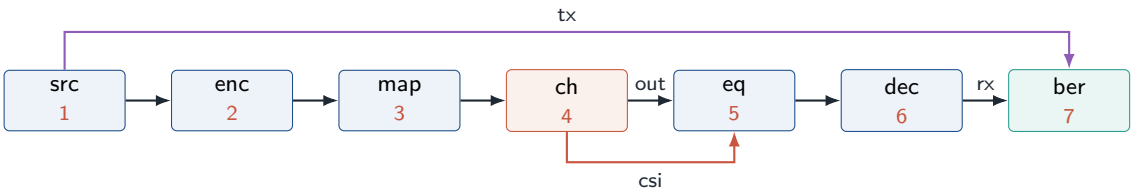


Figure 2.3: A graph with fan-out (`src.out` drives `enc.in` and `ber.tx`) and a side channel (`ch.csi`). Red numbers are the execution order.

2.4.2 Execution

`Graph.execute(ctx)` walks the blocks in order, gathers their inputs from the signals already produced, checks the delivered kinds at run time and calls `process`:

Code 2.3: The core of the executor. `linkstudio/core/graph.py`, lines 240–265

```
for name in self.order():
    block = self.blocks[name]
    inputs: dict[str, Signal] = {}
    for pname, port in block.input_ports.items():
        ep = Endpoint(name, pname)
        src = self.edges.get(ep)
        if src is not None:
            sig = produced[src]
        elif ep in external:
            sig = external[ep]
        else:
            continue
        if sig.kind not in port.kinds:
            raise BlockError(f"Input '{ep}' accepts {port.kinds.label()} but received {sig.kind.label()}.")
        inputs[pname] = sig
    try:
        outputs = block.process(inputs, ctx)
    except (BlockError, GraphError):
        raise
    except Exception as exc:  # add context, keep the original traceback
        raise BlockError(f"Block '{name}' ({block.type_name}) failed: {exc}") from exc
    for pname in block.output_ports:
        if pname not in outputs:
            raise BlockError(f"Block '{name}' did not produce output '{pname}'.")
        produced[Endpoint(name, pname)] = outputs[pname]
        values[f"{name}.{pname}"] = outputs[pname]
```

All produced signals are returned keyed by `"block.port"`; internal signals of subsystems appear as `"sub/block.port"`. These keys are what the GUI probes and what `Simulation.`

`run(capture=...)` returns. Exceptions raised inside a block are re-raised as `BlockError` with the block name, so the user sees which block failed and why.

2.5 Subsystems

A `Subsystem` (`core/composite.py`) wraps a whole graph as a single block. Its ports map to chosen inner ports, and its output kinds are those resolved by validating the inner graph. A transmitter or a receiver can thus be built once and reused in many scenarios; the same mechanism will let a future network layer treat a whole PHY link as one node. Parameters and metrics of inner blocks are reached with the `"/"` separator: `rx/dec.max_iter`, `rx/dec/avg_iterations`.

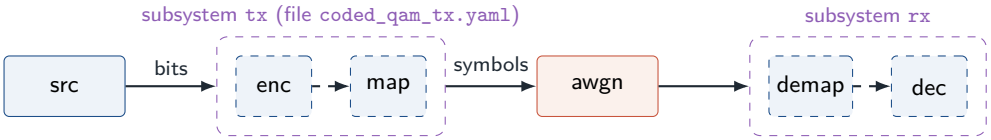


Figure 2.4: Subsystems nest graphs; from outside they behave as ordinary blocks.

2.6 Simulations and stop criteria

A `Simulation` (`core/simulation.py`) runs a graph batch after batch. One batch contains `batch_size` independent frames, which NumPy processes as a single vectorized operation, so a larger batch amortizes the Python overhead. Meter blocks accumulate counts across batches. The run stops at the first satisfied condition of a `StopCriteria`:

Code 2.4: Stop criteria are checked after every batch. `linkstudio/core/simulation.py`, lines 35–47

```
35 def check(self, n_batches: int, elapsed_s: float, metrics: dict[str, float]) -> str | None:
36     if n_batches < self.min_batches:
37         return None
38     if self.metric is not None and self.threshold is not None:
39         if self.metric not in metrics:
40             raise GraphError(f"Stop metric '{self.metric}' not found. Available: {sorted(
       metrics)}")
41         if metrics[self.metric] >= self.threshold:
42             return f"{self.metric} >= {self.threshold:g}"
43     if n_batches >= self.max_batches:
44         return "max_batches"
45     if self.max_time_s is not None and elapsed_s >= self.max_time_s:
46         return "max_time"
47     return None
```

The classic rule for error-rate estimation is `StopCriteria.errors("ber.bit_errors", 100)`: stop once 100 errors have been counted. Section 5.3 shows why this gives a relative accuracy of about $\pm 20\,\%$ independent of the BER itself. `run` also accepts an `on_batch` callback, which the web server uses to report progress and to cancel jobs.

2.7 Sweeps

A `Sweep` (`core/sweep.py`) runs a fresh simulation at every point of a parameter grid. In `grid` mode the points are the Cartesian product of the value lists; in `zip` mode the lists are paired element-wise. It returns a table with one record per point, which can be filtered, printed or written to CSV.

Code 2.5: Each sweep point sets the parameters and runs a new simulation. `linkstudio/core/sweep.py`, lines 116–122

```
116 for i, point in enumerate(points):
117     if should_stop is not None and should_stop():
118         break
119     for key, value in point.items():
120         self.graph.set_param(key, value)
121     sim = Simulation(self.graph, self.batch_size, self.seed if self.same_seed else self.
        seed + i)
122     res = sim.run(n_batches=self.n_batches, stop=self.stop, on_batch=keep_going)
```

Common random numbers By default every point reuses the same seed (`same_seed=True`). Two points then see the same bits and the same noise realizations, scaled differently. If X and Y are the estimates at two neighbouring points,

$$\operatorname{Var}(X - Y) = \operatorname{Var} X + \operatorname{Var} Y - 2\operatorname{Cov}(X, Y),$$

and the positive covariance created by shared randomness makes the *differences* between points much less noisy. The curves look smooth and comparisons between configurations are fair. Use different seeds only when the points must be statistically independent.

2.8 Metadata propagation

Physical quantities that a downstream block needs but cannot infer from the samples travel in `Signal.meta`. Examples:

- the constellation used by the mapper;
- the oversampling factor;
- the nominal signal power;
- the noise variance after equalization;
- the accumulated chromatic dispersion of a fiber link.

Each block copies the metadata of its input and adds or overrides what it changes. Table 2.2 follows the most important keys through a pulse-shaped link. The mechanism is what lets the `awgn` block set E_b/N_0 correctly whatever comes before it, and lets a soft demapper compute exact LLRs without being told the SNR. Chapter B lists all keys.

Table 2.2: Metadata along a pulse-shaped QPSK link with N_{sps} samples per symbol and $E_s/N_0 = \gamma$. The waveform power is per sample; symbols have unit energy.

after block	`power`	`sps`	`noise_var`	also sets
`mapper`	1	1	–	`constellation`, `bits_per_symbol`
`pulse_shaper`	$1/N_{\text{sps}}$	N_{sps}	–	`pulse_taps`, `delay`
`awgn` (`esn0`)	$1/N_{\text{sps}}$	N_{sps}	$N_0 = 1/\gamma$	
`matched_filter`	1	1	$1/\gamma$	`n_symbols`
`demapper`	uses `constellation` and `noise_var` for exact LLRs			

Watch out Metadata describes the *nominal* signal. A block that changes the signal in a way it cannot describe (a CTLE, an unknown channel) sets the affected keys to `None`,

> and downstream blocks fall back to measuring. When you write a block, propagate or invalidate every key your processing changes.

2.9 Scenarios

A scenario (`core/scenario.py`) is a graph plus its run configuration, stored as YAML:

```yaml
name: QPSK over AWGN
category: Tutorials            # used by the GUI scenario browser
description: ...
seed: 1
batch_size: 64
blocks:
  src:   {type: random_bits, params: {n_bits: 2000}}
  map:   {type: mapper, params: {scheme: psk, order: 4}}
  awgn:  {type: awgn, params: {snr_db: 6}}
  demap: {type: demapper, params: {method: hard}}
  ber:   {type: ber_meter}
  rx:    {type: subsystem, file: subsystems/coded_qam_rx.yaml}   # optional
chain: [src, map, awgn, demap, ber.rx]      # default-port connections
edges:
  - src.out -> ber.tx                        # explicit connections
simulation: {stop: {metric: ber.bit_errors, threshold: 200, max_batches: 500}}
sweep: {params: {awgn.snr_db: [0, 2, 4, 6, 8]}}   # optional: run a sweep instead
probes: [{key: awgn.out, view: constellation}]   # GUI scopes
layout: {src: [0, 0], map: [230, 0]}             # GUI positions
```

Files saved from the GUI contain only non-default parameters, so they stay short and readable and remain usable from the CLI. Chapter C lists the scenarios shipped with LinkStudio.

Chapter 3

Graphical editor, API and command line

3.1 The graphical editor

`python -m linkstudio serve` starts the web server and opens the editor in the browser. Figure 3.1 shows its layout.

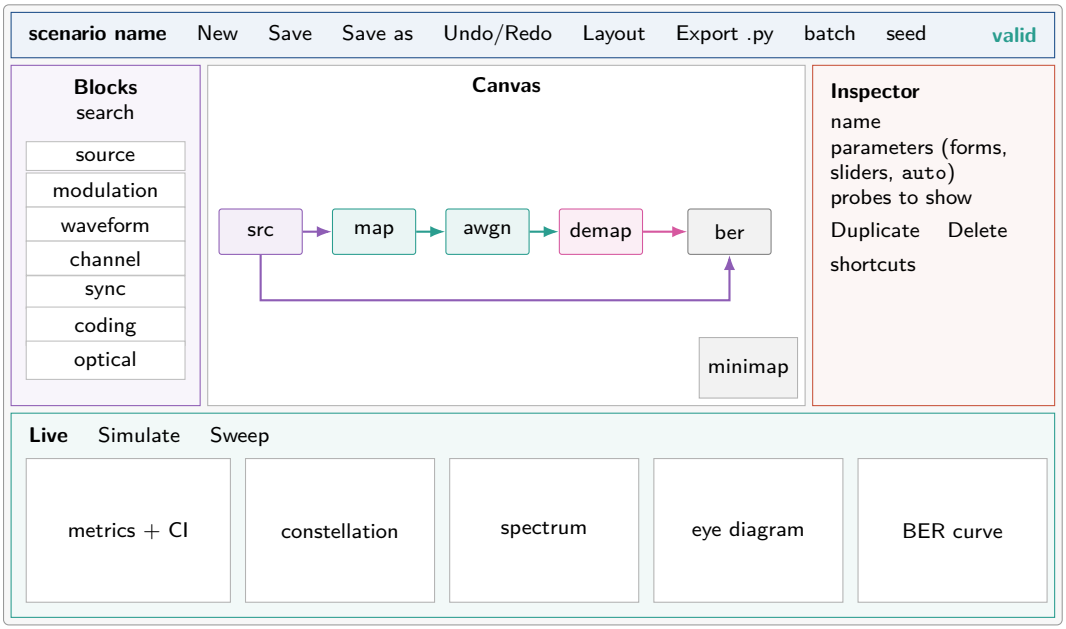


Figure 3.1: Layout of the editor: palette, canvas, inspector and the run dock with its three modes.

Editing.

- **Adding blocks:** drag a block from the palette or double-click it.
- **Connecting:** drag from an output handle (right side) to an input handle (left side). Handle colours are the signal kinds of Table 2.1; incompatible connections are refused, and dashed handles are optional inputs.
- **Inspector:** parameter forms are generated from each block's schema. Bounded and dB-valued parameters get sliders, and `auto` leaves a value to upstream metadata.
- **Shortcuts:** Ctrl+Z/Y (undo/redo), Ctrl+C/V/D (copy, paste, duplicate, with internal edges), Ctrl+L (automatic layout), Ctrl+S (save), Ctrl+O (scenario browser).

The run buttons are disabled while the graph is invalid, and the validator's message is shown next to them.

Modes.

Live runs the chain continuously at 1–30 batches per second and streams every probe. Parameter changes are applied between batches and the meters restart, so the effect of a slider is visible immediately. Structural edits restart the session.

Simulate runs a Monte-Carlo job in the background, either for a fixed number of batches or until a metric reaches a threshold. It shows the final metrics (BER with its 95 % confidence interval) and the probes of the last batch.

Sweep runs one or two nested parameter sweeps and plots any metric, with an optional theoretical curve. Curves can be kept and overlaid, and the results exported as CSV or PNG.

Probes and views. Ticking an output in the inspector adds a scope for it. The available views depend on the signal kind.

Kind	Views
SYMBOLS	constellation, time, spectrum
SAMPLES	spectrum (Welch), constellation, eye diagram, time
GRID	constellation, resource-grid magnitude
CSI	impulse / frequency response
LLR	histogram, time
BITS	time

3.2 The HTTP and WebSocket API

The editor talks to the FastAPI application in `linkstudio/api/app.py`. Everything it does can also be scripted.

Method	Path	Purpose
GET	`/api/blocks`	block catalogue: documentation, ports, parameter JSON Schema
GET	`/api/scenarios`	scenario list with name, category, description, block count
GET/PUT	`/api/scenarios/{path}`	load / save a scenario in editor format
POST	`/api/validate`	validate an editor graph; returns errors and port kinds
POST	`/api/layout`	automatic layout of the blocks
POST	`/api/export/python`	standalone Python script equivalent to the graph
POST	`/api/jobs`	start a simulate or sweep job
GET	`/api/jobs/{id}`	job status, progress, partial and final results
POST	`/api/jobs/{id}/cancel`	cancel a job
WS	`/api/live`	interactive live mode

Jobs run in a thread pool (`api/jobs.py`). Their results contain JSON-safe metrics (NaN becomes `null`), the probe data of the last batch and, for sweeps of a single parameter whose value matches a known modulation, theoretical BER curves. A failed job reports the message `failed` with the details in a separate `detail` field.

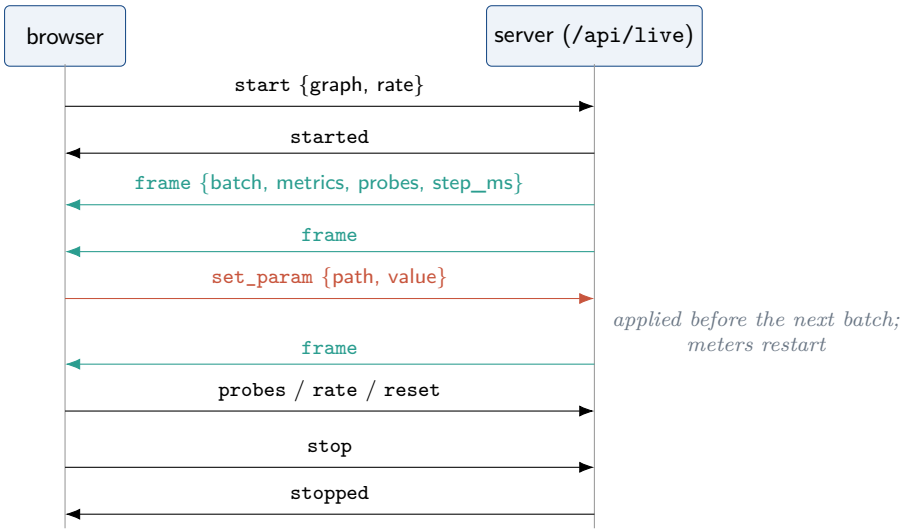


Figure 3.2: Message flow of the live mode. The server steps the simulation in a worker thread and paces frames to the requested rate.

3.3 Command line

`python -m linkstudio blocks [--category C]`	list block types
`python -m linkstudio describe TYPE`	ports, parameters and documentation of a block
`python -m linkstudio run FILE.yaml [--csv OUT]`	run a scenario (simulation or sweep)
`python -m linkstudio serve [--host H] [--port P] [--no-browser]`	start the editor
`python scripts/main.py [--quick]`	run all demos with theory checks

The demo scripts in `scripts/` (one per subsystem) are short, readable examples of the Python API. Each prints simulated values next to theory and returns a pass/fail status.

Chapter 4

Developer guide: writing blocks

Adding a model to LinkStudio means writing one block class. The engine, the CLI, the scenario loader and the GUI pick it up automatically once it is registered. This chapter builds a complete example and lists the conventions a block must respect to behave like the built-in ones.

4.1 A complete example: free-space path loss

Suppose we want a block that attenuates a radio signal by the free-space loss between two isotropic antennas at distance d and carrier frequency f_c (Friis):

$$L_{\mathrm{fs}} = \left(\frac{4\pi d f_c}{c}\right)^2, \qquad L_{\mathrm{fs}}\big|_{\mathrm{dB}} = 20\log_{10} d + 20\log_{10} f_c - 147.55. \tag{4.1}$$

The amplitude is scaled by $L_{\mathrm{fs}}^{-1/2}$. The nominal power metadata and any noise already present must be scaled by L_{fs}^{-1}, so that a downstream `awgn` block in `nominal` mode still computes the SNR from the right reference.

Code 4.1: A new block, e.g. in `linkstudio/phy/channel/pathloss.py`.

```python
import numpy as np
from pydantic import Field

from linkstudio.core import Block, BlockParams, Kind, Port, RunContext, Signal,
    register_block

C_LIGHT = 299_792_458.0


@register_block("free_space_loss", category="channel")
class FreeSpaceLoss(Block):
    """Friis free-space path loss between isotropic antennas."""

    class Params(BlockParams):                          # validated, serializable, shown in
    the GUI
        distance_m: float = Field(100.0, gt=0, description="Link distance.")
        carrier_hz: float = Field(2.4e9, gt=0, description="Carrier frequency.")

    inputs = {"in": Port(Kind.SYMBOLS | Kind.SAMPLES)}
    outputs = {"out": Port.like("in")}                  # same kind as the input

    def setup(self) -> None:                            # derived state, rebuilt on parameter
    change
        p = self.params
        self.loss = (4 * np.pi * p.distance_m * p.carrier_hz / C_LIGHT) ** 2

    def on_reset(self) -> None:                         # runtime state, cleared before a run
        self._frames = 0

    def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
        x = inputs["in"]
        meta = {k: x.meta[k] / self.loss for k in ("power", "noise_var") if x.meta.get(k)
    is not None}
        self._frames += x.batch_size
        return {"out": x.evolve(x.data / np.sqrt(self.loss), **meta)}
```

```
32
33      def metrics(self) -> dict[str, float]:
34          return {"loss_db": 10 * np.log10(self.loss), "frames": self._frames}
```

To register it, import the module in the package `__init__.py` (`linkstudio/phy/channel/__init__.py`). From then on:

- `g.create("pl", "free_space_loss", distance_m=500)` works from Python;
- `{type: free_space_loss, params: {distance_m: 500}}` works in YAML;
- the block appears in the GUI palette under *channel*, with a form for its two parameters;
- `pl.distance_m` can be swept;
- `pl.loss_db` is reported among the metrics.

4.2 Conventions

C1. Shapes. Accept any leading batch shape. Operate on the last axis and use `...` indexing, so the block works for `(B, n)` as well as `(B, P, n)` arrays.

C2. Vectorize over the batch. Loops over samples are acceptable for adaptive algorithms, but each iteration should process all frames at once (see the LMS loop in Listing 9.4).

C3. Kinds. Declare the narrowest accepted kinds; use `Port.like` when the output kind follows the input.

C4. Metadata. Propagate with `evolve`. Update the keys your processing changes (`power`, `noise_var`, `sps`, `delay`, ...) and set them to `None` when you cannot describe the result.

C5. Randomness. Draw only from `self.rng`; never use NumPy's global generator. This keeps runs reproducible (Section 2.2).

C6. Real signals stay real. Wireline and IM/DD chains are real-valued; do not return complex arrays for real inputs.

C7. Errors. Raise `BlockError` with a message telling the user how to fix the problem (*"Timing recovery needs an oversampled input (sps >= 2)"*).

C8. Metrics accumulate. Sum counts in `process`, compute ratios in `metrics`, clear them in `on_reset`.

4.3 Testing a block against theory

The test-suite follows one rule: a physical model is validated against a closed-form result, not against itself. `tests/helpers.py` provides `uncoded_link` and `ber_at`. A typical test reads:

```
def test_awgn_ber_matches_theory():
    g = uncoded_link("qam", 16, snr_db=10.0, n_bits=4 * 1200)
    assert np.isclose(ber_at(g, errors=300), theory.ber_qam(16, 10.0), rtol=0.2)
```

With 300 errors the relative standard deviation of the estimate is about $1/\sqrt{300} \approx 6\,\%$, so a $20\,\%$ tolerance makes false alarms negligible but still detects a 1 dB error. For our example the natural check is the metric itself: at 100 m and 2.4 GHz, $L_{\mathrm{fs}} = 80.05$ dB. Chapter 16 lists the physics checks of the whole library.

4.4 Adding a scenario

Save a graph from the GUI, or write a YAML file under `backend/scenarios/<category>/` with `name`, `category` and `description`. The test `tests/test_scenarios.py` loads and runs every scenario file, so a broken scenario is caught automatically.

Part II

Theory and implementation

Chapter 5

Baseband signal model, noise and Monte-Carlo estimation

Every chapter of Part II relies on three conventions:

- a signal is represented by its sampled complex envelope;
- noise is circular complex Gaussian with a variance tied to N_0;
- performance is estimated by counting errors.

This chapter makes the three precise.

5.1 The complex envelope

A real band-pass signal centred on the carrier f_c can always be written as

$$s(t) = \Re\left\{ x(t)\, e^{j2\pi f_c t} \right\} = x_I(t)\cos 2\pi f_c t - x_Q(t)\sin 2\pi f_c t, \qquad x(t) = x_I(t) + j\, x_Q(t), \tag{5.1}$$

where $x(t)$ is the *complex envelope*. If s occupies $|f \pm f_c| < B/2$, then x is a low-pass signal of bandwidth $B/2$. It can be sampled at $f_s \geq B$, whatever the carrier frequency. Linear time-invariant band-pass systems act on x through their low-pass equivalent impulse response. A carrier frequency error Δf becomes a rotation $x(t)e^{j2\pi\Delta f t}$, and an oscillator phase error $\phi(t)$ becomes $x(t)e^{j\phi(t)}$.

> **In the code** All SYMBOLS and SAMPLES signals are sampled complex envelopes $x[n] = x(n/f_s)$ with `Signal.fs`$= f_s$. Two exceptions are worth noting:
>
> - wireline and IM/DD electrical signals are *real* baseband signals ($x_Q \equiv 0$), and the blocks keep their arrays real;
> - optical fields are envelopes around the optical carrier, scaled in $\sqrt{\mathrm{W}}$ (Chapter 15).
>
> Powers are *envelope* powers, $P = \mathbb{E}|x[n]|^2$. The factor $\frac{1}{2}$ relating envelope and band-pass powers cancels in every SNR, so it is never applied.

5.2 Additive white Gaussian noise

5.2.1 From N_0 to a sample variance

Thermal noise at the receiver input is modelled as white with two-sided band-pass power spectral density $N_0/2$. Its complex envelope $w(t)$ is circular complex Gaussian with two-sided PSD N_0 (in envelope units). Observed through any filter of noise bandwidth f_s and sampled at f_s, it gives independent samples

$$w[n] \sim \mathcal{CN}(0, \sigma^2), \qquad \sigma^2 = N_0 f_s, \qquad \mathbb{E}\big[(\Re w)^2\big] = \mathbb{E}\big[(\Im w)^2\big] = \sigma^2/2. \tag{5.2}$$

The code works in units where the sample period is the time unit, so N_0 and σ^2 coincide numerically. The metadata key `noise_var` is σ^2, the noise variance per complex sample (or per symbol after matched filtering).

5.2.2 SNR definitions

Let P be the signal power per sample and N_{sps} the number of samples per symbol. The energy per symbol is $E_s = P\,N_{\text{sps}}T_s$ and the noise PSD is $N_0 = \sigma^2 T_s$, so

$$\frac{E_s}{N_0} = \frac{P\,N_{\text{sps}}}{\sigma^2}. \tag{5.3}$$

A symbol carries $k = \log_2 M$ coded bits. A code of rate R and N_s spatially multiplexed streams turn these into kRN_s information bits per channel use, so

$$\frac{E_b}{N_0} = \frac{E_s}{kRN_sN_0} \quad\Longrightarrow\quad \sigma^2 = \frac{P\,N_{\text{sps}}}{(E_b/N_0)\,kRN_s}. \tag{5.4}$$

The `awgn` block implements the three definitions of Table 5.1. N_{sps}, k, R and N_s are read from the metadata set by the pulse shaper (or OFDM modulator), the mapper, the encoders and the stream demultiplexer. Equation (5.4) is therefore correct for any chain, which is why the coded and MIMO curves of later chapters can be compared with uncoded theory on the same E_b/N_0 axis.

Table 5.1: SNR modes of the `awgn` block.

`mode`	meaning of `snr_db`	noise variance σ^2
`snr`	signal-to-noise ratio per sample	P/SNR
`esn0`	E_s/N_0	$PN_{\text{sps}}/(E_s/N_0)$
`ebn0`	E_b/N_0 (default)	$PN_{\text{sps}}/\big((E_b/N_0)\,kRN_s\big)$

The reference power P is, in order of priority:

1. the parameter `signal_power`;
2. the nominal power `power` carried by the metadata (`power_reference=nominal`, the textbook definition of *average* SNR);
3. the power measured over the batch.

The nominal reference matters for fading: the average SNR is defined by the transmitted power, not by the instantaneous power after a deep fade.

Code 5.1: Noise variance from the SNR definition (`awgn.py`). `linkstudio/phy/channel/awgn.py`, lines 45–64

```
45 def noise_variance(self, x: Signal) -> float:
46     p = self.params
47     if p.signal_power is not None:
48         power = p.signal_power
49     elif p.power_reference == "nominal" and x.meta.get("power") is not None:
50         power = float(x.meta["power"])
51     else:
52         power = float(np.mean(np.abs(x.data) ** 2))
53     snr = 10 ** (p.snr_db / 10)
54     if p.mode == "snr":
55         return power / snr
56     sps = x.meta.get("sps", 1)
57     if p.mode == "esn0":
58         return power * sps / snr
59     k = p.bits_per_symbol or x.meta.get("bits_per_symbol")
60     if k is None:
61         raise BlockError("Eb/N0 mode needs bits_per_symbol (mapper upstream or parameter).")
       )
```

```
    rate = p.code_rate or x.meta.get("code_rate", 1.0)
    # Spatial multiplexing carries several symbols per channel use.
    return power * sps / (snr * k * rate * x.meta.get("spatial_rate", 1))
```

Code 5.2: Adding the noise. Real inputs get $\sigma^2/2$ per real dimension. linkstudio/phy/channel/awgn.py, lines 66–74

```
def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
    x = inputs["in"]
    n0 = self.noise_variance(x)
    if np.iscomplexobj(x.data):
        noise = self.rng.standard_normal((*x.data.shape, 2)).view(np.complex128)[..., 0]
    else:  # real baseband (wireline, IM/DD): N0/2 per real dimension
        noise = self.rng.standard_normal(x.data.shape)
    y = x.data + np.sqrt(n0 / 2) * noise
    return {"out": x.evolve(y, noise_var=x.meta.get("noise_var", 0.0) + n0)}
```

Remark. For a real signal only the in-phase noise component matters, with variance $\sigma^2/2 = N_0/2$. A real antipodal (NRZ, 2-PAM) signal then has the same error probability as BPSK, as it should. This is tested in `tests/test_wireline_optical.py`.

5.3 Monte-Carlo estimation of error rates

5.3.1 The estimator and its accuracy

Let N bits be observed and K of them be in error. If errors are independent with probability p, then $K \sim \mathrm{Binomial}(N, p)$ and the natural estimator $\hat{p} = K/N$ is unbiased with

$$\operatorname{Var}\hat{p} = \frac{p(1-p)}{N}, \qquad \frac{\sqrt{\operatorname{Var}\hat{p}}}{p} = \sqrt{\frac{1-p}{Np}} \approx \frac{1}{\sqrt{\mathbb{E}K}}. \tag{5.5}$$

The relative accuracy depends only on the number of *errors*, not on the number of bits. Stopping a simulation after a fixed number of errors (Section 2.6) therefore gives the same relative accuracy at every SNR: 100 errors give about $\pm 10\,\%$ standard deviation, and $\pm 20\,\%$ at 95 % confidence.

5.3.2 The Wilson score interval

The normal approximation $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/N}$ fails when K is small: with $K = 0$ it gives a zero-width interval. The Wilson interval [18] instead inverts the test $|\hat{p} - p| \le z\sqrt{p(1-p)/N}$ for the unknown p. Squaring gives the quadratic $(1 + z^2/N)p^2 - (2\hat{p} + z^2/N)p + \hat{p}^2 \le 0$, whose roots are

$$p_{\pm} = \frac{\hat{p} + \dfrac{z^2}{2N} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{N} + \dfrac{z^2}{4N^2}}}{1 + \dfrac{z^2}{N}}, \qquad z = 1.96 \text{ for } 95\,\%. \tag{5.6}$$

The interval is never empty, stays inside $[0, 1]$ and is accurate even for a handful of errors. `ber_meter` reports it as `ber_ci_low` and `ber_ci_high`, and the GUI shows it next to the BER.

Code 5.3: The Wilson interval in `metrics.py`. linkstudio/phy/metrics.py, lines 13–21

```
def wilson_interval(errors: int, trials: int, z: float = 1.96) -> tuple[float, float]:
    """95 % Wilson score confidence interval of an error probability."""
    if trials == 0:
        return float("nan"), float("nan")
    p = errors / trials
    denom = 1 + z * z / trials
```

```
    centre = (p + z * z / (2 * trials)) / denom
    half = z * np.sqrt(p * (1 - p) / trials + z * z / (4 * trials * trials)) / denom
    return float(max(centre - half, 0.0)), float(min(centre + half, 1.0))
```

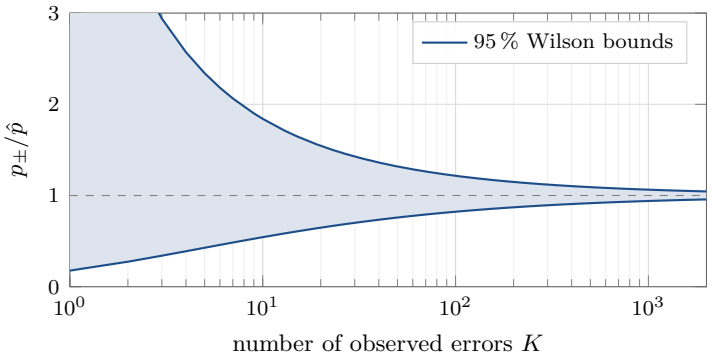


Figure 5.1: Width of the 95 % confidence interval relative to the estimate, as a function of the number of observed errors (computed with `wilson_interval` at $p = 10^{-4}$). Ten errors leave a factor of about two of uncertainty; one hundred errors bring it to ±20 %.

Correlated errors The binomial model assumes independent errors. After a decoder, or in block fading, errors come in bursts within a frame. The interval is then optimistic, because the effective number of independent observations is closer to the number of erroneous *frames*. For coded systems, judge the accuracy by `frame_errors` as well.

5.4 Closed-form references

`linkstudio/phy/theory.py` collects the closed-form results used in tests, demos, the GUI and this manual. All functions accept arrays and SNRs in dB.

`ber_bpsk`, `ber_pam`, `ber_qam`	exact Gray-coded BER over AWGN (Section 6.3)
`ser_psk`, `ber_psk`	Craig's integral for M-PSK
`ber_bpsk_rayleigh`, `ber_bpsk_mrc`, `ber_rayleigh`	fading and diversity (Section 9.4)
`ber_hamming_hard`	post-decoding BER of Hamming codes
`ook_q_factor`, `q_to_ber`, `osnr_to_esn0_db`	optical receivers (Chapter 15)
`gaussian_pulse_broadening`	dispersion of a Gaussian pulse
`shannon_capacity`, `ebn0_min_db`	information-theoretic limits

Chapter 6

Digital modulation and demodulation

6.1 Constellations

A memoryless modulator maps each group of k bits $\boldsymbol{b} = (b_0, \dots, b_{k-1})$ to one of $M = 2^k$ complex points, $s = \mathcal{S}(\boldsymbol{b})$. The label $\ell = \sum_i b_i 2^{k-1-i}$ is read MSB first and indexes the table of points. LinkStudio builds four families, all normalized to unit average energy $\mathbb{E}|s|^2 = 1$:

$$\text{M-PAM:}\quad s_\ell = \frac{2i - (M-1)}{\sqrt{(M^2-1)/3}}, \quad \ell = g(i), \tag{6.1}$$

$$\text{M-QAM:}\quad s_\ell = \frac{a_{\ell_I} + j\, a_{\ell_Q}}{\sqrt{2(M-1)/3}}, \quad \ell = (\ell_I, \ell_Q),\ a.\ \text{Gray } \sqrt{M}\text{-PAM levels}, \tag{6.2}$$

$$\text{M-PSK:}\quad s_\ell = e^{\,j(2\pi i/M + \phi_0)}, \quad \ell = g(i), \quad \phi_0 = \pi/4 \text{ for QPSK, else } 0, \tag{6.3}$$

where $g(i) = i \oplus \lfloor i/2 \rfloor$ is the binary-reflected Gray code. Adjacent points therefore differ in exactly one bit. The denominators are the average energies of the unnormalized PAM and QAM grids, $\frac{1}{M}\sum_i (2i - M + 1)^2 = (M^2-1)/3$ per dimension. Custom constellations are normalized the same way.

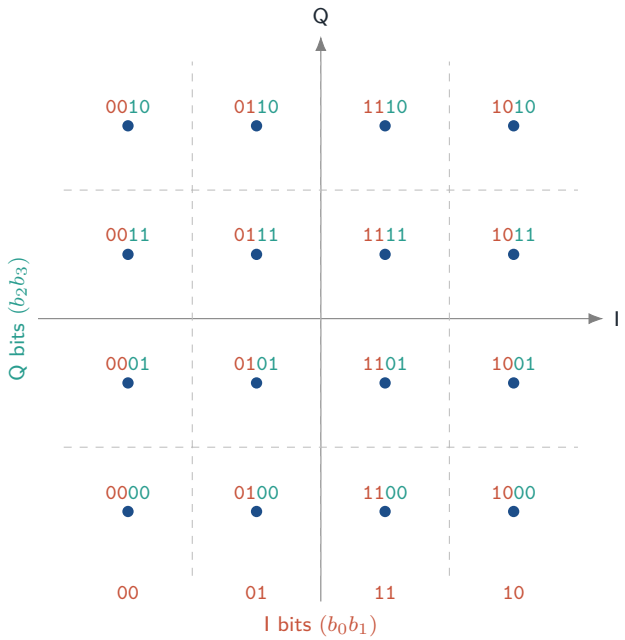


Figure 6.1: 16-QAM with the labels produced by `qam(16)`. The first half of the label selects the in-phase level and the second half the quadrature level, each with a Gray 4-PAM labelling. Dashed lines are the decision boundaries.

Code 6.1: Square QAM as the product of two Gray PAMs (`dsp/constellations.py`).
linkstudio/phy/dsp/constellations.py, lines 118–125

```
118 side = int(round(np.sqrt(order)))
119 if side * side != order or side & (side - 1):
120     raise ValueError(f"Square QAM needs M = 4^n, got {order}.")
```

```
half = int(np.log2(side))
levels = _pam_levels(side)
labels = np.arange(order)
points = levels[labels >> half] + 1j * levels[labels & (side - 1)]
return Constellation(points, name=f"{order}-QAM")
```

In the code `mapper` (`phy/modulation.py`) builds the constellation in `setup()` and maps frames of nk bits to n symbols. It sets `constellation`, `bits_per_symbol`, `power`$= 1$, `sps`$= 1$ and the symbol rate `symbol_rate` as `fs`. `demapper` takes the constellation from the metadata unless one is given explicitly.

6.2 Optimal detection

For the observation $y = s + w$ with $w \sim \mathcal{CN}(0, \sigma^2)$ and equiprobable symbols, the maximum-likelihood decision is the nearest point:

$$\hat{s} = \arg\max_{s \in \mathcal{S}} p(y \mid s) = \arg\max_{s} \frac{1}{\pi\sigma^2} e^{-|y-s|^2/\sigma^2} = \arg\min_{s \in \mathcal{S}} |y - s|^2. \tag{6.4}$$

`Constellation.nearest` evaluates all M distances in chunks of 2^{16} symbols, which bounds memory. `method=hard` returns the label bits of the nearest point.

6.2.1 Bit log-likelihood ratios

Soft-input decoders need, for every bit, the a-posteriori log-likelihood ratio. With $\mathcal{S}_i^{(b)}$ the subset of points whose i-th label bit equals b, and equiprobable bits,

$$L(b_i) = \ln \frac{P(b_i = 0 \mid y)}{P(b_i = 1 \mid y)} = \ln \sum_{s \in \mathcal{S}_i^{(0)}} e^{-|y-s|^2/\sigma^2} - \ln \sum_{s \in \mathcal{S}_i^{(1)}} e^{-|y-s|^2/\sigma^2}. \tag{6.5}$$

Keeping only the dominant term of each sum gives the max-log approximation

$$L(b_i) \approx \frac{1}{\sigma^2}\Big(\min_{s \in \mathcal{S}_i^{(1)}} |y - s|^2 - \min_{s \in \mathcal{S}_i^{(0)}} |y - s|^2 \Big), \tag{6.6}$$

which is exact for BPSK and loses only a few tenths of a dB with good codes. A positive LLR means bit 0 throughout LinkStudio. The variance σ^2 may differ from symbol to symbol, for example after zero-forcing equalization of a fading channel; `noise_var` may therefore be an array broadcastable to the symbols.

Code 6.2: Exact (log-sum-exp) and max-log LLRs. `linkstudio/phy/dsp/constellations.py`, lines 72–86

```
k = self.bits_per_symbol
flat = y.reshape(-1)
nv = np.broadcast_to(np.asarray(noise_var, dtype=np.float64), y.shape).reshape(-1)
llr = np.empty((flat.size, k))
zero = self.bit_labels == 0  # (M, k)
for s in range(0, flat.size, _CHUNK):
    metric = -np.abs(flat[s:s + _CHUNK, None] - self.points[None, :]) ** 2
    metric /= np.maximum(nv[s:s + _CHUNK, None], 1e-300)
    for i in range(k):
        m0, m1 = metric[:, zero[:, i]], metric[:, ~zero[:, i]]
        if method == "exact":
            llr[s:s + _CHUNK, i] = logsumexp(m0, axis=1) - logsumexp(m1, axis=1)
        else:
            llr[s:s + _CHUNK, i] = m0.max(axis=1) - m1.max(axis=1)
return llr.reshape(*y.shape[:-1], -1)
```

Key idea The LLR scale matters. Equalizers update `noise_var` with their noise enhancement ($\sigma^2/|h|^2$ for a one-tap ZF equalizer, $\sigma^2\|\boldsymbol{w}\|^2$ for an FIR equalizer, the post-detection variance for MIMO). The decoder downstream then receives LLRs with the correct reliability without any manual tuning.

6.3 Error probability over AWGN

With $\mathrm{Q}(x) = \frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$ the Gaussian tail, the classic results implemented in `theory.py` are:

- **BPSK and Gray QPSK**: $P_b = \mathrm{Q}\left(\sqrt{2E_b/N_0}\right)$. QPSK is two orthogonal BPSK channels sharing the symbol energy.
- **Gray M-PAM**, exact (Cho and Yoon [4]): with $L = M$, $m = \log_2 L$ and $\gamma_b = E_b/N_0$,

$$P_b = \frac{1}{m}\sum_{q=1}^{m}\frac{1}{L}\sum_{i=0}^{(1-2^{-q})L-1}(-1)^{\lfloor i2^{q-1}/L\rfloor}\left(2^{q-1} - \left\lfloor\frac{i2^{q-1}}{L} + \frac{1}{2}\right\rfloor\right)\operatorname{erfc}\left((2i+1)\sqrt{\frac{3m\gamma_b}{L^2-1}}\right). \quad (6.7)$$

- **Square Gray M-QAM**: the in-phase and quadrature components are independent $\sqrt{M}$-PAM signals carrying half the bits each, so $P_b^{\mathrm{QAM}}(M) = P_b^{\mathrm{PAM}}(\sqrt{M})$ at the same E_b/N_0.
- **M-PSK**: the symbol error rate is Craig's integral [5]

$$P_s = \frac{1}{\pi}\int_0^{(M-1)\pi/M}\exp\left(-\frac{E_s}{N_0}\frac{\sin^2(\pi/M)}{\sin^2\theta}\right)d\theta, \qquad P_b \approx P_s/k, \quad (6.8)$$

  where the Gray approximation $P_b \approx P_s/k$ is tight at moderate SNR.

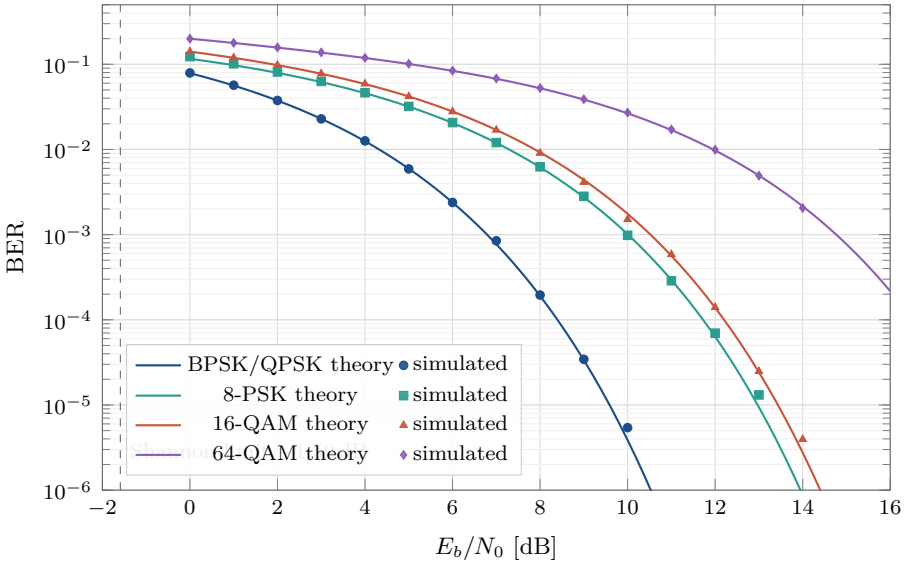


Figure 6.2: Bit error rate over AWGN: simulated (markers, chain `random_bits`→`mapper`→`awgn`→`demapper`→`ber_meter`, up to 200 errors per point) and exact theory (lines). Points with fewer than 10 errors are omitted.

The ultimate limit. For a spectral efficiency η bit/s/Hz, the AWGN capacity $C = \log_2(1 + \mathrm{SNR})$ with $\mathrm{SNR} = \eta E_b/N_0$ requires $E_b/N_0 \geq (2^\eta - 1)/\eta$, which tends to $\ln 2 = -1.59\,\mathrm{dB}$ as $\eta \to 0$ (`ebn0_min_db`). The gap between this limit and the uncoded curves of Figure 6.2 is what channel coding recovers (Chapter 13).

Chapter 7

Pulse shaping and matched filtering

7.1 Linear modulation

A linearly modulated waveform is a train of pulses weighted by the symbols,

$$x(t) = \sum_{m} a_m \, g(t - mT). \tag{7.1}$$

For i.i.d. zero-mean symbols with $\mathbb{E}|a_m|^2 = E_a$, the power spectral density is

$$S_x(f) = \frac{E_a}{T} \, |G(f)|^2, \tag{7.2}$$

so the spectrum is shaped entirely by the pulse. The receiver filters with $h(t)$ and samples the output at $t = mT + \tau_0$. With $p = g \star h$ the end-to-end pulse,

$$y(mT + \tau_0) = a_m \, p(\tau_0) + \underbrace{\sum_{l \neq m} a_l \, p((m-l)T + \tau_0)}_{\text{intersymbol interference}} + (w \star h)(mT + \tau_0). \tag{7.3}$$

7.2 The Nyquist criterion

The ISI vanishes for every symbol sequence if and only if $p(nT) = \delta[n]$ (taking $\tau_0 = 0$). By the Poisson summation formula this is equivalent to the frequency-domain condition

$$\sum_{k=-\infty}^{\infty} P\!\left(f - \frac{k}{T}\right) = T \qquad \forall f. \tag{7.4}$$

The narrowest solution is the ideal low-pass of bandwidth $1/(2T)$, the sinc pulse, which is not realizable and decays only as $1/t$. The *raised-cosine* (RC) family trades bandwidth for decay. Its spectrum has a cosine roll-off of relative width β:

$$P_{\mathrm{RC}}(f) = \begin{cases} T, & |f| \le \frac{1-\beta}{2T}, \\ \frac{T}{2}\left[1 + \cos\left(\frac{\pi T}{\beta}\left(|f| - \frac{1-\beta}{2T}\right)\right)\right], & \frac{1-\beta}{2T} < |f| \le \frac{1+\beta}{2T}, \\ 0, & \text{otherwise}, \end{cases} \tag{7.5}$$

$$p_{\mathrm{RC}}(t) = \operatorname{sinc}\frac{t}{T} \, \frac{\cos(\pi\beta t/T)}{1 - (2\beta t/T)^2}. \tag{7.6}$$

The occupied bandwidth is $(1+\beta)R_s$ and the pulse decays as $1/t^3$.

7.3 The matched filter

Among all receive filters, the one maximizing the SNR at the sampling instant follows from the Cauchy–Schwarz inequality. For white noise of PSD N_0,

$$\mathrm{SNR} = \frac{\left|\int G(f)H(f)\,df\right|^2}{N_0 \int |H(f)|^2 df} \le \frac{\int |G(f)|^2 df}{N_0} = \frac{E_g}{N_0}, \tag{7.7}$$

with equality for $H(f) = G^*(f)$, that is $h(t) = g^*(-t)$. To have both a matched filter and zero ISI, the RC response is split equally between transmitter and receiver. Each side uses the *root-raised-cosine* (RRC) pulse, $|G_{\mathrm{RRC}}(f)|^2 = P_{\mathrm{RC}}(f)$, whose impulse response is

$$g_{\mathrm{RRC}}(t) = \frac{\sin\left(\pi \frac{t}{T}(1-\beta)\right) + 4\beta \frac{t}{T} \cos\left(\pi \frac{t}{T}(1+\beta)\right)}{\pi \frac{t}{T}\left(1 - (4\beta \frac{t}{T})^2\right)}, \tag{7.8}$$

with the finite limits $1-\beta+4\beta/\pi$ at $t = 0$ and $\frac{\beta}{\sqrt{2}}\left[(1+\frac{2}{\pi})\sin\frac{\pi}{4\beta} + (1-\frac{2}{\pi})\cos\frac{\pi}{4\beta}\right]$ at $t = \pm T/(4\beta)$. These special points are handled explicitly in the code:

Code 7.1: RRC taps with their singular points (`dsp/filters.py`). linkstudio/phy/dsp/filters.py, lines 19–36

```
def rrc_taps(sps: int, span: int, rolloff: float) -> np.ndarray:
    """Root-raised-cosine pulse, unit energy."""
    t = _time_axis(sps, span)
    b = rolloff
    if b == 0:
        return _unit_energy(np.sinc(t))
    h = np.empty_like(t)
    eps = 1e-9
    at_zero = np.abs(t) < eps
    at_sing = np.abs(np.abs(4 * b * t) - 1) < eps
    rest = ~(at_zero | at_sing)
    tt = t[rest]
    h[rest] = (np.sin(np.pi * tt * (1 - b)) + 4 * b * tt * np.cos(np.pi * tt * (1 + b))) /
    (
        np.pi * tt * (1 - (4 * b * tt) ** 2))
    h[at_zero] = 1 - b + 4 * b / np.pi
    h[at_sing] = b / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * b))
                                   + (1 - 2 / np.pi) * np.cos(np.pi / (4 * b)))
    return _unit_energy(h)
```

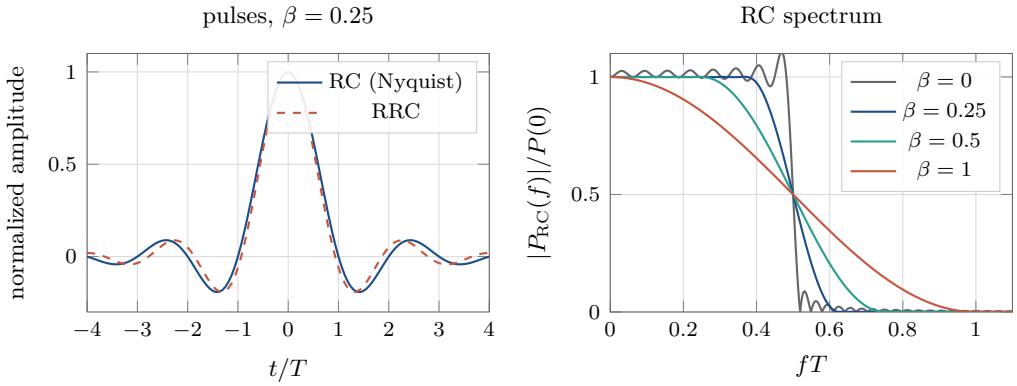


Figure 7.1: Left: the RC pulse crosses zero at every non-zero multiple of T, while the RRC pulse alone does not. Right: spectra of the truncated RC taps produced by `rc_taps` (span 32 symbols). All curves pass through 0.5 at $fT = 0.5$, as required by Equation (7.4).

7.4 Discrete-time implementation

In the code `pulse_shaper` upsamples by N_{sps} (zero stuffing) and convolves with unit-energy taps spanning `span` symbols. It records the taps in `pulse_taps`, the group delay $(L-1)/2$ in `delay`, and the power per sample $1/N_{\mathrm{sps}}$ (unit symbol energy spread over

N_{sps} samples).
`matched_filter` convolves with $g^*[-n]$ (or another pulse), samples at the known delay and rescales.

Code 7.2: The pulse shaper and its metadata. `linkstudio/phy/waveform/pulse_shaping.py`, lines 38–45

```
38 def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
39     x = inputs["in"]
40     sps = self.params.sps
41     y = fir_filter(upsample(x.data, sps), self.taps)
42     return {"out": Signal(y, Kind.SAMPLES, fs=x.fs * sps if x.fs else None, meta={
43         **x.meta, "sps": sps, "pulse_taps": self.taps, "n_symbols": x.length,
44         "power": (x.meta.get("power") or 1.0) / sps,  # unit-energy pulse spreads Es over
       sps samples
45         "delay": x.meta.get("delay", 0.0) + (len(self.taps) - 1) / 2})}
```

With unit-energy taps the peak of the end-to-end pulse is $p[0] = \sum_n |g[n]|^2 = 1$, and the noise gain of the matched filter is $\sum_n |h[n]|^2 = 1$. The block nevertheless normalizes by the actual peak and scales `noise_var` by $\sum |h|^2 / |p[0]|^2$, so that mismatched receive filters (an RC receive filter, a rectangular integrate-and-dump) also yield symbols at nominal amplitude with the correct noise variance:

Code 7.3: Normalization of the matched filter output. `linkstudio/phy/waveform/pulse_shaping.py`, lines 88–98

```
88 y = fir_filter(x.data, rx_taps)
89 n_sym = x.meta.get("n_symbols", x.length // sps)
90 start = int(round(x.meta.get("delay", 0.0) + (len(rx_taps) - 1) / 2))
91
92 noise_gain = float(np.sum(np.abs(rx_taps) ** 2))
93 if p.normalize and tx_taps is not None:
94     overall = np.convolve(tx_taps, rx_taps)
95     peak = overall[int(round((len(tx_taps) - 1) / 2 + (len(rx_taps) - 1) / 2))]
96     y = y / peak
97     noise_gain /= abs(peak) ** 2
98 noise = {"noise_var": x.meta["noise_var"] * noise_gain} if x.meta.get("noise_var") is not
       None else {}
```

Following Equation (5.3), the AWGN block sets $\sigma^2 = P N_{\text{sps}} / (E_s/N_0) = 1/(E_s/N_0)$ on the waveform. After the matched filter the symbols have unit energy and noise variance σ^2, so the symbol SNR equals E_s/N_0, the matched-filter bound. This is verified in `tests/test_physics.py::test_pulse_shaped_link_matches_theory`: the BER of an RRC-shaped link equals the symbol-rate theory.

Truncation Taps are truncated to `span` symbols. Short spans raise the spectral side lobes and leave a small residual ISI that grows as $\beta \to 0$. For dense WDM or very small roll-offs, use spans of 32 symbols or more (the coherent optical scenarios use 32).

7.5 Eye diagrams

Overlaying traces of two symbol periods shows the ISI at a glance (Figure 7.2). After the RRC transmit filter alone the eye is partially closed, because an RRC pulse is not a Nyquist pulse. After the matched filter all traces pass through ± 1 at the sampling instant.

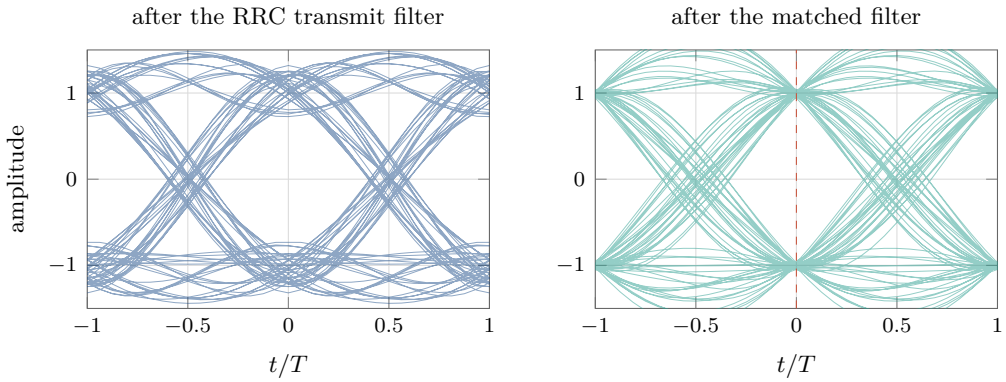


Figure 7.2: Eye diagrams of binary PAM with RRC pulses ($\beta = 0.35$, 16 samples/symbol, 30 dB SNR), cut from the outputs of `pulse_shaper` and `matched_filter` (`decimate=false`). The dashed line is the sampling instant used by the matched filter.

Chapter 8

Radio front-end impairments

Real transmitters and receivers distort the signal before any propagation effect. This chapter models the impairments implemented in `phy/channel/impairments.py`. Figure 8.1 shows their signatures on a 16-QAM constellation.

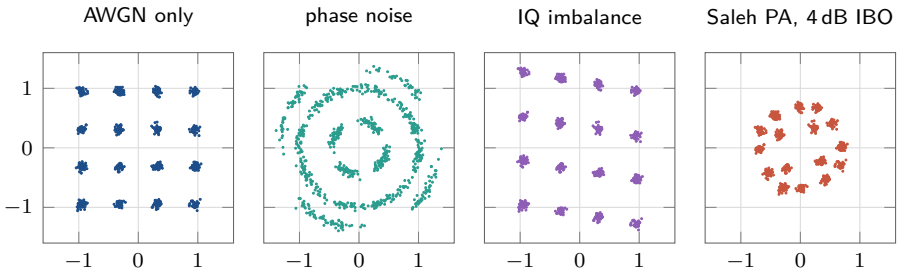


Figure 8.1: 16-QAM at $E_s/N_0 = 26\,\mathrm{dB}$ through `phase_noise` (150 Hz linewidth at 1 MBd), `iq_imbalance` (1.5 dB, 8°) and `power_amplifier` (Saleh). Phase noise smears points along circles; IQ imbalance turns the square grid into a parallelogram; the PA compresses and rotates the outer points.

8.1 Carrier frequency and phase offsets

A local oscillator at $f_c + \Delta f$ with phase ϕ_0 turns the received envelope into

$$y[n] = x[n]\, e^{\,j(2\pi \Delta f\, n/f_s + \phi_0)}. \tag{8.1}$$

`carrier_offset` applies this rotation (`cfo_hz`, `phase_deg`, or a uniformly random phase per frame). Synchronization (Chapter 10) estimates and removes it.

8.2 Oscillator phase noise

A free-running oscillator accumulates phase errors. The standard model, and the one used for lasers in Chapter 15, is a Wiener process: the instantaneous frequency deviation is white, so the phase performs a random walk. In discrete time,

$$\phi[n] = \phi[n-1] + \varepsilon[n], \qquad \varepsilon[n] \sim \mathcal{N}\!\left(0,\ \frac{2\pi\Delta\nu}{f_s}\right). \tag{8.2}$$

Proposition 8.1. The field $e^{j\phi(t)}$ of a Wiener phase with increment variance $2\pi\Delta\nu|\tau|$ has a Lorentzian spectrum of full width at half maximum $\Delta\nu$.

Proof. $\phi(t+\tau) - \phi(t) \sim \mathcal{N}(0, 2\pi\Delta\nu|\tau|)$, and $\mathbb{E}e^{jX} = e^{-\operatorname{Var} X/2}$ for a Gaussian X, so the autocorrelation is $R(\tau) = e^{-\pi\Delta\nu|\tau|}$. Its Fourier transform is

$$S(f) = \frac{\Delta\nu/(2\pi)}{f^2 + (\Delta\nu/2)^2},$$

which falls to half its peak at $f = \pm\Delta\nu/2$. □

Code 8.1: Wiener phase noise (`phase_noise`). `linkstudio/phy/channel/impairments.py`, lines 74–79

```
74 def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
75     x = inputs["in"]
76     fs = _require_fs(x, "Phase noise")
77     var = 2 * np.pi * self.params.linewidth_hz / fs
78     phi = np.cumsum(self.rng.standard_normal(x.data.shape) * np.sqrt(var), axis=-1)
79     return {"out": x.evolve(x.data * np.exp(1j * phi))}
```

The phase variance grows linearly with time. What matters for a receiver is therefore the product $\Delta\nu\, T$ of linewidth and symbol duration: carrier recovery must track the phase faster than it wanders.

8.3 IQ imbalance

In a quadrature down-converter the Q branch may have a gain $g \neq 1$ and a phase error φ relative to the I branch. Writing the two branch outputs and recombining them gives the widely-linear model

$$y = K_1 x + K_2 x^{*} + d, \qquad K_1 = \frac{1 + g\, e^{-j\varphi}}{2}, \qquad K_2 = \frac{1 - g\, e^{j\varphi}}{2}, \tag{8.3}$$

where d is a DC offset. The conjugate term creates a mirror image of the signal. Its strength is measured by the image rejection ratio $\mathrm{IRR} = |K_1|^2/|K_2|^2$; for $g = 1.5\,\mathrm{dB}$ and $\varphi = 8^\circ$ it is about 19 dB. On a constellation it shears the square grid (Figure 8.1).

Code 8.2: IQ imbalance and DC offset (`iq_imbalance`). `linkstudio/phy/channel/impairments.py`, lines 117–124

```
117 def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
118     x = inputs["in"]
119     p = self.params
120     g = 10 ** (p.amplitude_db / 20)
121     phi = np.deg2rad(p.phase_deg)
122     k1 = (1 + g * np.exp(-1j * phi)) / 2
123     k2 = (1 - g * np.exp(1j * phi)) / 2
124     return {"out": x.evolve(k1 * x.data + k2 * np.conj(x.data) + complex(p.dc_i, p.dc_q))}
```

8.4 Power amplifier nonlinearity

A memoryless band-pass nonlinearity acts on the envelope $x = re^{j\theta}$ as

$$y = A(r)\, e^{\,j(\theta + \Phi(r))}, \tag{8.4}$$

where $A(r)$ is the AM/AM curve and $\Phi(r)$ the AM/PM curve. The input is first scaled so that its mean power lies `input_backoff_db` below the saturation level $A_{\mathrm{sat}} = 1$ (input back-off, IBO). Three models are implemented:

$$\text{Rapp (solid state):} \quad A(r) = \frac{r}{\left(1 + r^{2p}\right)^{1/(2p)}}, \qquad \Phi = 0, \tag{8.5}$$

$$\text{Saleh (TWTA):} \quad A(r) = \frac{r}{1 + \beta_a r^2}, \qquad \Phi(r) = \frac{\alpha_\phi r^2}{1 + \beta_\phi r^2}, \tag{8.6}$$

$$\text{ideal clipper:} \quad A(r) = \min(r, 1), \qquad \Phi = 0. \tag{8.7}$$

The Saleh AM/AM curve is divided by α_a so that all models have unit small-signal gain; the default constants are those of Saleh's original TWT fit [6]. The smoothness p of the Rapp model interpolates between a soft compression ($p = 1$) and the ideal clipper ($p \to \infty$).

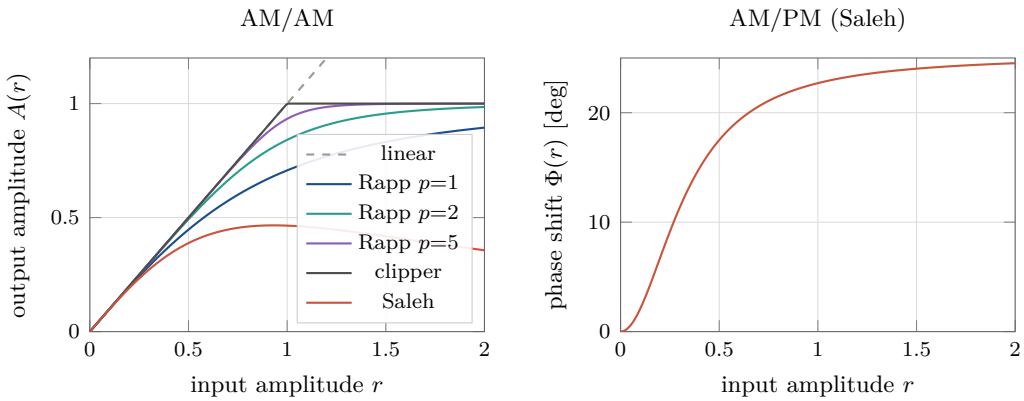


Figure 8.2: PA characteristics measured by driving the `power_amplifier` block with an amplitude ramp (back-off chosen so that the block's input scaling is unity).

Code 8.3: Back-off scaling and the AM/AM, AM/PM models. `linkstudio/phy/channel/impairments.py`, lines 150–163

```
p_in = float(np.mean(np.abs(x.data) ** 2))
scale = np.sqrt(10 ** (-p.input_backoff_db / 10) / max(p_in, 1e-300))
u = x.data * scale
r, theta = np.abs(u), np.angle(u)
if p.model == "rapp":
    a = r / (1 + r ** (2 * p.smoothness)) ** (1 / (2 * p.smoothness))
    y = a * np.exp(1j * theta)
elif p.model == "saleh":
    # Divided by alpha_a so that the small-signal gain is 1, like the other models.
    a = r / (1 + p.saleh_beta_a * r ** 2)
    dphi = p.saleh_alpha_p * r ** 2 / (1 + p.saleh_beta_p * r ** 2)
    y = a * np.exp(1j * (theta + dphi))
else:
    y = np.minimum(r, 1.0) * np.exp(1j * theta)
```

> **Watch out** The nonlinearity generates spectral regrowth (out-of-band emission) and an in-band error that is *not* Gaussian. For OFDM, whose envelope is nearly Gaussian (Section 11.5), an IBO of several dB is needed. Compare the `power_meter` PAPR with the back-off.

8.5 Analog-to-digital conversion

`quantizer` models a uniform mid-rise ADC with b bits on each of I and Q. The full scale A is either given or set to `loading_factor`$\times$ the RMS value σ_x of each component. The step is $\Delta = 2A/2^b$. For a signal that does not clip, the quantization error is approximately uniform with variance $\Delta^2/12$, giving

$$\mathrm{SQNR} = \frac{\sigma_x^2}{\Delta^2/12} = \frac{3 \cdot 2^{2b}}{(A/\sigma_x)^2} \;\Rightarrow\; \mathrm{SQNR}_{\mathrm{dB}} \approx 6.02\,b + 4.77 - 20\log_{10}\frac{A}{\sigma_x}. \tag{8.8}$$

With the default loading factor of 4 (clipping probability about 6×10^{-5} for a Gaussian component), an 8-bit ADC gives about 41 dB.

Code 8.4: Uniform mid-rise quantization with clipping. `linkstudio/phy/channel/impairments.py`, lines 191–198

```
levels = 1 << p.bits
```

```
step = 2 * fs / levels

def q(v: np.ndarray) -> np.ndarray:
    idx = np.clip(np.floor(v / step), -levels // 2, levels // 2 - 1)
    return (idx + 0.5) * step

return {"out": x.evolve(q(x.data.real) + 1j * q(x.data.imag))}
```

8.6 Timing offset

`timing_offset` delays the signal by $d = \lfloor d \rfloor + \mu$ samples. The integer part is a shift. The fractional part uses a Blackman-windowed sinc interpolator of 33 taps, $h[n] = \operatorname{sinc}(n - 16 - \mu)\, w[n] / \sum_n \operatorname{sinc}(n - 16 - \mu)\, w[n]$, which is accurate as long as the signal is oversampled. The frame length is preserved, so a delay pushes the tail out of the frame; the `preamble` block appends guard symbols for this reason.

Chapter 9

Fading channels and equalization

9.1 The multipath channel

In a radio channel the transmitted signal reaches the receiver along many paths with different delays τ_l, complex gains and Doppler shifts. The low-pass equivalent channel is a linear time-varying filter,

$$y(t) = \sum_l a_l(t)\, x(t-\tau_l) + w(t), \qquad h(\tau;t) = \sum_l a_l(t)\,\delta(\tau-\tau_l). \tag{9.1}$$

Two scales characterize it:

- the *delay spread* σ_τ, whose inverse is the coherence bandwidth $B_c \approx 1/(5\sigma_\tau)$;
- the maximum Doppler shift $f_D = v f_c/c$, whose inverse sets the coherence time $T_c \approx 0.423/f_D$.

A signal of bandwidth $B \ll B_c$ sees *flat* fading (a single complex gain). If $B \gtrsim B_c$ the fading is frequency selective and causes ISI.

9.2 Rayleigh and Rician fading

When a path is the superposition of many unresolved scatterers with random phases, the central limit theorem makes its gain circular complex Gaussian, $h \sim \mathcal{CN}(0,1)$. The envelope $|h|$ is then Rayleigh distributed and the power $|h|^2$ exponential. A line-of-sight component of relative power K (the Rician factor) adds a deterministic term:

$$h = \sqrt{\frac{K}{K+1}}\, e^{j\theta_0} + \sqrt{\frac{1}{K+1}}\, h_{\mathrm{Ray}}, \qquad \mathbb{E}|h|^2 = 1. \tag{9.2}$$

9.2.1 Time correlation: the Clarke–Jakes model

For isotropic scattering in the horizontal plane, a path arriving at angle α is shifted by $f_D\cos\alpha$. Averaging over a uniform α gives [19] the autocorrelation and the U-shaped Doppler spectrum

$$R_h(\tau) = \mathbb{E}[h(t+\tau)h^*(t)] = J_0(2\pi f_D\tau), \qquad S_h(f) = \frac{1}{\pi f_D\sqrt{1-(f/f_D)^2}},\ |f|<f_D. \tag{9.3}$$

LinkStudio generates it with a random-phase sum of sinusoids:

$$h[n] = \frac{1}{\sqrt{N}}\sum_{i=1}^{N} e^{\,j(2\pi f_D\cos\alpha_i\, n/f_s+\phi_i)}, \qquad \alpha_i,\phi_i \sim \mathcal{U}[0,2\pi). \tag{9.4}$$

Each term has autocorrelation $\mathbb{E}_\alpha e^{j2\pi f_D\cos\alpha\,\tau} = J_0(2\pi f_D\tau)$, so the sum reproduces Equation (9.3) exactly in expectation. Its first-order statistics approach Gaussian as N grows (16 sinusoids by default).

Code 9.1: Sum-of-sinusoids Rayleigh process (`channel/fading.py`).
`linkstudio/phy/channel/fading.py`, lines 28–40

```
28 def sum_of_sinusoids(rng: np.random.Generator, batch: int, n: int, doppler_norm: float,
```

```
                      n_sinusoids: int = 16) -> np.ndarray:
    """Time-varying Rayleigh process with Jakes (Clarke) Doppler spectrum.

    Random-phase sum-of-sinusoids model; ``doppler_norm`` is f_D / f_s.
    """
    t = np.arange(n)
    h = np.zeros((batch, n), dtype=np.complex128)
    alpha = rng.uniform(0, 2 * np.pi, (batch, n_sinusoids))
    phi = rng.uniform(0, 2 * np.pi, (batch, n_sinusoids))
    for i in range(n_sinusoids):
        h += np.exp(1j * (2 * np.pi * doppler_norm * np.cos(alpha[:, i, None]) * t + phi[:,
     i, None]))
    return h / np.sqrt(n_sinusoids)
```

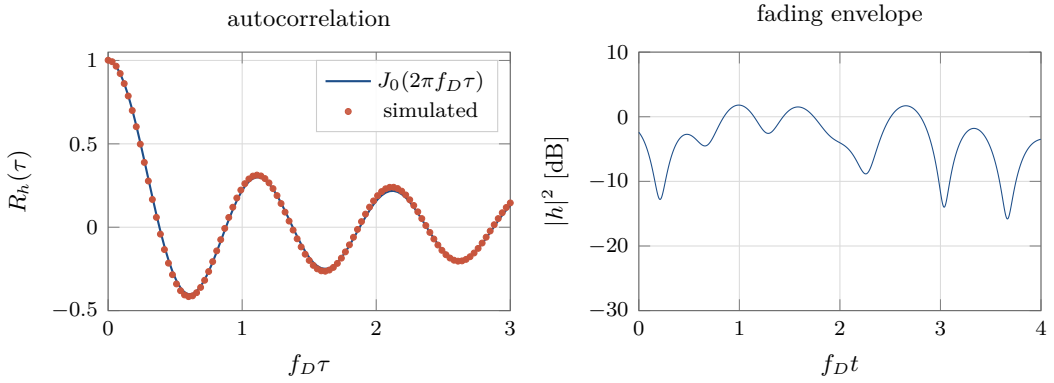


Figure 9.1: Left: autocorrelation of `sum_of_sinusoids` (400 realizations, $f_D/f_s = 0.01$) against Equation (9.3). Right: a realization; fades deeper than 20 dB occur about once per wavelength travelled.

In the code `flat_fading` multiplies the input by h and publishes the true gains on its `csi` output (`csi_type="gain"`). Three time-variation modes are available:

- `block`: one coefficient per frame (quasi-static);
- `fast`: independent per sample (ideal interleaving);
- `jakes`: correlated with Doppler `doppler_hz`; needs `fs`.

Code 9.2: Fading generation and the Rician line-of-sight term. `linkstudio/phy/channel/fading.py`, lines 69–80

```
if p.variation == "block":
    h = complex_normal(self.rng, (B, 1))
elif p.variation == "fast":
    h = complex_normal(self.rng, (B, n))
else:
    if not x.fs:
        raise BlockError("Jakes fading needs the sample rate (fs) of the signal.")
    h = sum_of_sinusoids(self.rng, B, n, p.doppler_hz / x.fs, p.n_sinusoids)
if p.model == "rician":
    k = 10 ** (p.k_factor_db / 10)
    los = np.exp(1j * self.rng.uniform(0, 2 * np.pi, (B, 1)))
    h = np.sqrt(k / (k + 1)) * los + np.sqrt(1 / (k + 1)) * h
```

9.3 Tapped delay lines and 3GPP profiles

`multipath` implements the discrete model $y[n] = \sum_l h_l[n]\, x[n - d_l]$ with integer sample delays $d_l = \mathrm{round}(\tau_l f_s)$. Taps falling on the same sample have their powers added, and the profile is normalized to unit total power. Taps are Rayleigh (or static with `fading=false`), constant per frame or each with an independent Jakes process. The 3GPP extended models of TS 36.104 [20] are built in:

Model	delays [ns]	relative powers [dB]
EPA	0, 30, 70, 90, 110, 190, 410	0, −1, −2, −3, −8, −17.2, −20.8
EVA	0, 30, 150, 310, 370, 710, 1090, 1730, 2510	0, −1.5, −1.4, −3.6, −0.6, −9.1, −7, −12, −16.9
ETU	0, 50, 120, 200, 230, 500, 1600, 2300, 5000	−1, −1, −1, 0, 0, 0, −3, −5, −7

Their RMS delay spreads are 43, 357 and 991 ns. The channel impulse response is published on `csi`: `csi_type="cir"` with shape (B, L) for block fading, or `"cir_tv"` with shape (B, L, n) for time-varying taps.

9.4 Error probability with fading

With coherent detection and perfect channel knowledge, the instantaneous SNR of BPSK in flat fading is $\gamma = |h|^2 E_b/N_0$, exponentially distributed with mean $\bar\gamma$. Averaging the AWGN error probability over it gives

$$\bar P_b = \int_0^\infty \mathrm{Q}\left(\sqrt{2\gamma}\right) \frac{e^{-\gamma/\bar\gamma}}{\bar\gamma}\, d\gamma = \frac{1}{2}\left(1 - \sqrt{\frac{\bar\gamma}{1+\bar\gamma}}\right) \xrightarrow{\bar\gamma\to\infty} \frac{1}{4\bar\gamma}. \tag{9.5}$$

The error rate now falls only as $1/\mathrm{SNR}$ instead of exponentially: 10 dB more SNR buys one decade of BER. Diversity, the subject of Section 12.3, restores a steeper slope. `ber_rayleigh` performs the same average numerically for any AWGN curve. Figure 12.2 compares Equation (9.5) with the chain `flat_fading`→`awgn`→`one_tap_equalizer`.

9.5 Equalization

9.5.1 One-tap equalization

For flat fading, $y = hs + w$, and dividing by the known gain gives $\hat s = s + w/h$. The noise variance becomes $\sigma^2/|h|^2$ symbol by symbol, and `one_tap_equalizer` writes exactly this array to `noise_var`, so the soft demapper weights faded symbols correctly. The `phase` variant only derotates, which is sufficient for PSK.

9.5.2 Linear FIR equalizers designed from the channel

For a frequency-selective channel with impulse response $\boldsymbol{h} = (h_0, \dots, h_{L-1})$, a linear equalizer $\boldsymbol{w}$ of N taps produces the combined response $\mathbf{H}\boldsymbol{w}$, where $\mathbf{H}$ is the $(N + L - 1) \times N$ convolution matrix of $\boldsymbol{h}$. Zero forcing asks the combined response to approximate a unit impulse $\boldsymbol{e}_d$ at decision delay d. The MMSE design adds the noise power σ^2 as a regularizer:

$$\boldsymbol{w}_d = \arg\min_{\boldsymbol{w}} \; \|\mathbf{H}\boldsymbol{w} - \boldsymbol{e}_d\|^2 + \lambda\|\boldsymbol{w}\|^2 = (\mathbf{H}^{\mathsf H}\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^{\mathsf H}\boldsymbol{e}_d, \qquad \lambda = \begin{cases} \sigma^2 & \text{MMSE} \\ \approx 0 & \text{ZF.} \end{cases} \tag{9.6}$$

All delays are solved at once, since the columns of $(\mathbf{H}^{\mathsf H}\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^{\mathsf H}$ are the solutions for every d. The delay with the largest useful gain $(\mathbf{H}\boldsymbol{w}_d)_d$ is selected. The output is then scaled by $1/(\mathbf{H}\boldsymbol{w}_d)_d$ to remove the MMSE bias, and the noise variance becomes $\sigma^2\|\boldsymbol{w}\|^2$.

Code 9.3: Least-squares design of ZF/MMSE equalizers with delay selection (`linear_equalizer`).
linkstudio/phy/receiver.py, lines 87–108

```
H = np.zeros((B, Lc, n), dtype=np.complex128)
for j in range(n):
    H[:, j:j + L, j] = h
nv = x.meta.get("noise_var", 0.0)
lam = np.mean(nv) if self.params.method == "mmse" else 1e-9
A = np.conj(np.swapaxes(H, 1, 2)) @ H + lam * np.eye(n)
W = np.linalg.solve(A, np.conj(np.swapaxes(H, 1, 2)))  # (B, n, Lc): column d = taps for
    delay d
if self.params.delay is not None:
    if self.params.delay >= Lc:
        raise BlockError(f"Delay must be < n_taps + L - 1 = {Lc}.")
    d = np.full(B, self.params.delay)
else:
    gain = np.real(np.einsum("bij,bji->bi", H, W))  # (H W)[d, d]
    d = np.argmax(gain, axis=1)
rows = np.arange(B)
w = W[rows, :, d]  # (B, n)
bias = np.einsum("bij,bj->bi", H, w)[rows, d]
w = w / bias[:, None]
full = np.stack([np.convolve(x.data[b], w[b]) for b in range(B)])
idx = d[:, None] + np.arange(x.length)
y = np.take_along_axis(np.pad(full, ((0, 0), (0, Lc))), idx, axis=1)
new_nv = np.asarray(nv) * np.sum(np.abs(w) ** 2, axis=1)[:, None]
```

9.5.3 Adaptive equalization: LMS

When the channel is unknown, the equalizer $z[n] = \boldsymbol{w}^{\mathsf{H}}\boldsymbol{u}[n]$, with $\boldsymbol{u}[n] = (y[n], y[n-1], \dots)$, can minimize $\mathbb{E}|d[n] - z[n]|^2$ by stochastic gradient descent. The instantaneous gradient with respect to $\boldsymbol{w}^*$ is $-\boldsymbol{u}\,e^*$ with $e = d - z$, giving the LMS update and its normalized version:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \mu\,\boldsymbol{u}[n]\,e^*[n], \qquad \mu_{\text{NLMS}} = \frac{\tilde{\mu}}{\|\boldsymbol{u}[n]\|^2 + \epsilon}, \quad 0 < \tilde{\mu} < 2. \tag{9.7}$$

The reference $d[n]$ is the known training sequence for the first `n_train` symbols and the nearest constellation point afterwards (decision-directed mode). The residual MSE measured after training is written to `noise_var`.

Code 9.4: The (N)LMS loop runs over time but processes all frames of the batch at once.
linkstudio/phy/receiver.py, lines 147–166

```
for t in range(N + D):
    u = ypad[:, t:t + n][:, ::-1]  # [y[t], y[t-1], ...]
    z = np.sum(np.conj(w) * u, axis=1)
    k = t - D
    if k < 0:
        continue
    out[:, k] = z
    if k < p.n_train:
        target = ref.data[:, k]
    elif p.decision_directed:
        target = const.points[const.nearest(z)]
    else:
        continue
    e = target - z
    if k >= p.n_train:
        err_power += float(np.sum(np.abs(e) ** 2))
        err_count += B
    mu = p.step_size / (1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=True)) if p.
    normalized \
```

```
        else p.step_size
    w += mu * u * np.conj(e)[:, None]
```

ZF versus MMSE Zero forcing inverts spectral nulls and amplifies the noise there. MMSE balances ISI against noise and tends to ZF at high SNR. The decision-feedback equalizer of Section 14.5 removes post-cursor ISI without noise enhancement, at the cost of error propagation.

Chapter 10

Synchronization

A receiver does not know when symbols start, where the frame begins, or the exact carrier frequency and phase. Figure 10.1 shows the synchronized single-carrier receiver of `phy/sync.py`, which recovers these quantities in three stages.

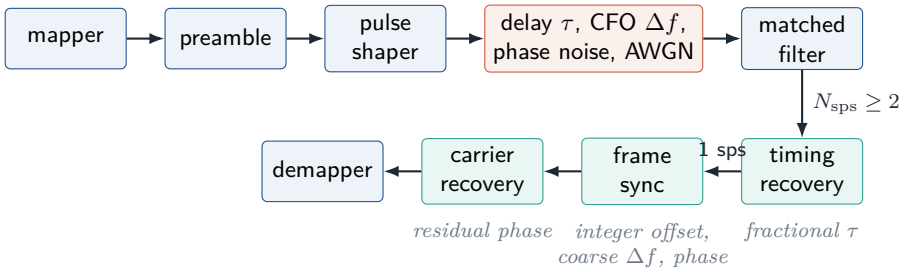


Figure 10.1: The synchronized receiver of the *Synchronized QPSK receiver* scenario.

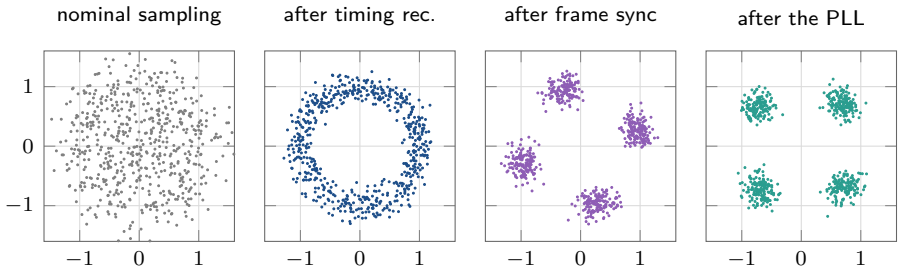


Figure 10.2: QPSK through a delay of 9.37 samples, a 3 kHz CFO at 1 MBd, 10 Hz phase noise and 12 dB E_b/N_0. Panels: sampled at the nominal instants (ISI and rotation), after Gardner timing recovery (rotating ring), after preamble-based CFO and phase correction (residual spread), after the decision-directed PLL.

10.1 The preamble

`preamble` prepends L known QPSK symbols made of two identical halves of length $h = L/2$, the Schmidl–Cox structure [9], and appends `guard` zero symbols. The repetition makes the timing metric insensitive to the CFO and yields a frequency estimate (Section 10.3). The guard gives the receiver slack for unknown delays.

10.2 Symbol timing recovery

After the matched filter the waveform is sampled at N_{sps} samples per symbol, but the optimum instants $mT + \tau$ fall between samples. Timing recovery has three parts:

- an *interpolator* that evaluates the waveform at arbitrary instants;
- a *timing error detector* (TED);
- a *loop filter* that turns detector outputs into corrections.

10.2.1 Interpolation

A cubic Lagrange (Farrow) interpolator evaluates $x(i+\mu)$ for $0 \le \mu < 1$ from four neighbours:

$$x(i+\mu) = c_0 + c_1\mu + c_2\mu^2 + c_3\mu^3, \quad \begin{aligned} c_0 &= x_0, & c_1 &= -\tfrac{1}{3}x_{-1} - \tfrac{1}{2}x_0 + x_1 - \tfrac{1}{6}x_2, \\ c_2 &= \tfrac{1}{2}x_{-1} - x_0 + \tfrac{1}{2}x_1, & c_3 &= -\tfrac{1}{6}x_{-1} + \tfrac{1}{2}x_0 - \tfrac{1}{2}x_1 + \tfrac{1}{6}x_2. \end{aligned} \tag{10.1}$$

Code 10.1: Farrow-structure cubic interpolation (`cubic_interpolate`). linkstudio/phy/sync.py, lines 36–44

```
n = x.shape[-1]
i = np.floor(t).astype(np.int64)
mu = t - i
rows = np.arange(x.shape[0])
xm1, x0, x1, x2 = (x[rows, np.clip(i + d, 0, n - 1)] for d in (-1, 0, 1, 2))
c1 = -xm1 / 3 - x0 / 2 + x1 - x2 / 6
c2 = xm1 / 2 - x0 + x1 / 2
c3 = -xm1 / 6 + x0 / 2 - x1 / 2 + x2 / 6
return ((c3 * mu + c2) * mu + c1) * mu + x0
```

10.2.2 Feedforward start: the Oerder–Meyr estimator

A linearly modulated signal is cyclostationary. The mean of $|x(t)|^2$ is periodic in T, and its first Fourier coefficient has a phase proportional to the timing offset. Over K symbols,

$$\hat{\varepsilon} = -\frac{1}{2\pi}\arg \sum_{k=0}^{KN_{\mathrm{sps}}-1} |x[k]|^2 e^{-j2\pi k/N_{\mathrm{sps}}} \qquad \text{(in symbols, modulo 1)}, \tag{10.2}$$

which needs $N_{\mathrm{sps}} \ge 3$ [8]. The block uses it to initialize the loop, which removes the long acquisition (and the risk of hang-up) of a loop started from an arbitrary phase.

Code 10.2: Oerder–Meyr initial estimate. linkstudio/phy/sync.py, lines 117–123

```
if p.initial_estimate and sps >= 3:
    # Oerder & Meyr: phase of the symbol-rate spectral line of |x|^2.
    seg = slice(int(start), int(start) + p.estimate_symbols * sps)
    k = np.arange(seg.start, min(seg.stop, L))
    line = np.sum(np.abs(data[:, k]) ** 2 * np.exp(-2j * np.pi * k / sps), axis=1)
    eps = -np.angle(line) / (2 * np.pi)  # peak position modulo one symbol
    tau = np.mod(eps - start / sps + 0.5, 1.0) - 0.5
```

10.2.3 The Gardner detector

Gardner's TED [7] uses two samples per symbol, the current strobe and the midpoint between strobes:

$$e[m] = \Re\Big\{ x^*\big((m-\tfrac{1}{2})T + \hat{\tau}\big)\,\big[x(mT+\hat{\tau}) - x((m-1)T+\hat{\tau})\big]\Big\}. \tag{10.3}$$

It is independent of the carrier phase and works before carrier recovery. Its mean response to a timing error, the *S-curve*, is zero at the correct instant and has slope K_d there (Figure 10.3). With the convention of the code, $e > 0$ means the strobes are late.

10.2.4 The second-order loop

A proportional-plus-integral loop filter makes the loop second order. It tracks a constant offset with zero error and a frequency (clock drift) with zero steady-state error. The gains follow from

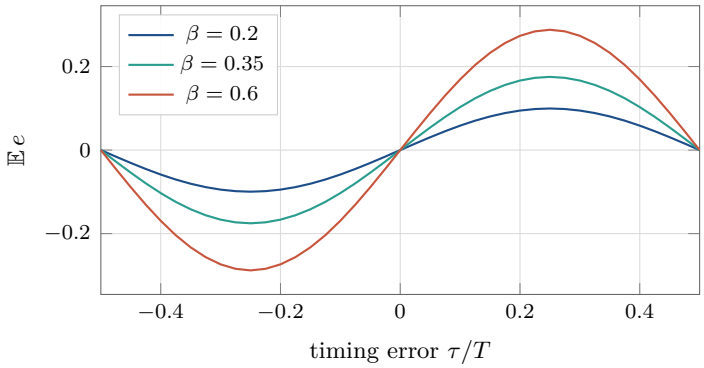


Figure 10.3: Measured S-curves of the Gardner detector for QPSK with RRC pulses (8 samples per symbol, matched-filtered output of the real blocks, interpolated with the same cubic interpolator). The slope at the origin grows with the roll-off; for $\beta = 0.35$ it is close to 1, the default `detector_gain`.

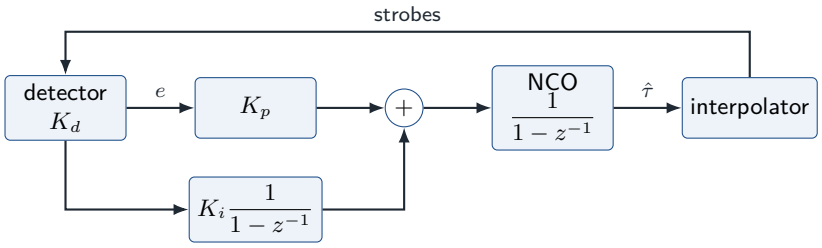


Figure 10.4: Proportional-plus-integral loop used for timing (and, with a phase detector, for the PLL).

the normalized noise bandwidth $B_n T$ and the damping ζ [3]:

$$\theta = \frac{B_n T}{\zeta + \frac{1}{4\zeta}}, \qquad K_p = \frac{4\zeta\theta}{(1 + 2\zeta\theta + \theta^2)K_d}, \qquad K_i = \frac{4\theta^2}{(1 + 2\zeta\theta + \theta^2)K_d}. \tag{10.4}$$

Code 10.3: Second-order loop gains. `linkstudio/phy/sync.py`, lines 24–31

```
24 def loop_gains(bandwidth: float, damping: float, detector_gain: float = 1.0) -> tuple[float,
       float]:
25     """Proportional and integral gains of a second-order loop.
26
27     ``bandwidth`` is the noise bandwidth normalized to the update rate.
28     """
29     theta = bandwidth / (damping + 1 / (4 * damping))
30     denom = 1 + 2 * damping * theta + theta ** 2
31     return 4 * damping * theta / denom / detector_gain, 4 * theta ** 2 / denom /
       detector_gain
```

Code 10.4: The timing loop: interpolate, detect, filter, advance. `linkstudio/phy/sync.py`, lines 128–136

```
128 for m in range(1, n_out):
129     t = np.clip(start + (m + tau) * sps, 1, L - 3)
130     y = cubic_interpolate(data, t)
131     mid = cubic_interpolate(data, np.clip(t - sps / 2, 1, L - 3))
132     err = np.real(np.conj(mid) * (y - prev))  # > 0 when sampling late
133     integ += ki * err
134     tau -= kp * err + integ
135     out[:, m] = y
136     prev = y
```

A small B_nT averages noise better but tracks more slowly; 0.01 (the default) is a common compromise.

10.3 Frame synchronization and coarse CFO

At symbol rate, with a residual CFO of ϵ cycles per symbol, the two halves of the preamble differ only by the phase $2\pi\epsilon h$. The Schmidl–Cox metric is evaluated at every candidate start d:

$$P(d) = \sum_{k=0}^{h-1} r^*[d+k]\, r[d+k+h], \qquad M(d) = \frac{|P(d)|^2}{\left(\sum_k |r[d+k+h]|^2\right)^2}, \qquad \hat{\epsilon} = \frac{\arg P(\hat{d})}{2\pi h}. \tag{10.5}$$

The estimate is unambiguous for $|\epsilon| < 1/(2h)$. Because $M(d)$ has a plateau, its maximum is used only for the CFO estimate. After derotation, the frame start is located by cross-correlation with the known preamble, $\hat{d} = \arg\max_d \left|\sum_k r[d+k]\, p^*[k]\right|$. The CFO is then refined at the exact position, and the common phase is taken from the correlation peak. Finally the payload symbols are extracted.

Code 10.5: Schmidl–Cox CFO, correlation timing and refinement (`frame_sync`).
linkstudio/phy/sync.py, lines 191–212

```
if self.params.correct_cfo:
    cand = np.arange(D + 1)[:, None] + np.arange(h)  # (D+1, h)
    P = np.sum(np.conj(data[:, cand]) * data[:, cand + h], axis=-1)  # (B, D+1)
    R = np.sum(np.abs(data[:, cand + h]) ** 2, axis=-1)
    d0 = np.argmax(np.abs(P) ** 2 / np.maximum(R, 1e-12) ** 2, axis=1)
    eps = np.angle(P[b_idx, d0]) / (2 * np.pi * h)  # cycles per symbol
    data = data * np.exp(-2j * np.pi * eps[:, None] * n_idx)

win = np.arange(D + 1)[:, None] + np.arange(L)  # (D+1, L)
corr = np.sum(data[:, win] * np.conj(pre), axis=-1)  # (B, D+1)
d = np.argmax(np.abs(corr), axis=1)

if self.params.correct_cfo:  # refine with the exact preamble position
    first = d[:, None] + np.arange(h)
    fine = np.angle(np.sum(np.conj(data[rows, first]) * data[rows, first + h], axis=-1)) /
    (2 * np.pi * h)
    data = data * np.exp(-2j * np.pi * fine[:, None] * n_idx)
    eps = eps + fine
pwin = d[:, None] + np.arange(L)
phase = np.angle(np.sum(data[rows, pwin] * np.conj(pre), axis=-1))

idx = d[:, None] + L + np.arange(n_pay)
out = data[rows, idx] * np.exp(-1j * phase)[:, None]
```

10.4 Carrier phase tracking

The residual phase (CFO residue and phase noise) is tracked by a decision-directed PLL. For $y = s\, e^{j\Delta\phi} + w$ and a correct decision $\hat{s}$,

$$e = \frac{\Im\{y\, \hat{s}^*\}}{|\hat{s}|^2} = \sin\Delta\phi + \text{noise} \approx \Delta\phi, \tag{10.6}$$

so the detector gain is 1. The same second-order loop as above updates the phase estimate every symbol. Like every decision-directed scheme it has an M-fold phase ambiguity, which the preamble phase estimate resolves beforehand.

Code 10.6: Decision-directed PLL (`carrier_recovery`). `linkstudio/phy/sync.py`, lines 246–252

```
246 for n in range(N):
247     y = x.data[:, n] * np.exp(-1j * phi)
248     d = const.points[const.nearest(y)]
249     err = np.imag(y * np.conj(d)) / np.maximum(np.abs(d) ** 2, 1e-12)
250     integ += ki * err
251     phi += kp * err + integ
252     out[:, n] = y
```

Watch out The PLL bandwidth must exceed the phase-noise linewidth relative to the symbol rate. It must also be small enough that decision errors at low SNR do not make it slip cycles. A cycle slip rotates all subsequent decisions by $2\pi/M$ and appears as a burst of errors until the next preamble. Differential encoding or pilot symbols are the classic remedies.

Chapter 11

OFDM

Orthogonal frequency-division multiplexing turns a frequency-selective channel into many parallel flat channels, each equalized by a single complex division. This chapter derives the property, describes the subcarrier layout and the pilot-based receiver of `phy/waveform/ofdm.py`, and examines the two weaknesses of OFDM: synchronization and peak-to-average power ratio.

11.1 Cyclic prefix and circular convolution

An OFDM symbol carries N frequency-domain values X_k (the subcarrier symbols). The transmitter computes the unitary inverse DFT and prepends the last N_{cp} samples (the *cyclic prefix*):

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k \, e^{j2\pi kn/N}, \qquad n = -N_{\mathrm{cp}}, \dots, N-1. \tag{11.1}$$

Proposition 11.1. If the channel impulse response has length $L \le N_{\mathrm{cp}}+1$, then after removing the prefix and applying the unitary DFT,

$$Y_k = H_k X_k + W_k, \qquad H_k = \sum_{l=0}^{L-1} h_l \, e^{-j2\pi kl/N}, \qquad W_k \sim \mathcal{CN}(0, \sigma^2) \text{ i.i.d.} \tag{11.2}$$

Proof. Thanks to the prefix, the N retained received samples equal the *circular* convolution of x with h: $\boldsymbol{y} = \mathbf{C}\boldsymbol{x} + \boldsymbol{w}$, with $\mathbf{C}$ circulant. Every circulant matrix is diagonalized by the DFT, $\mathbf{C} = \mathbf{F}^{\mathsf{H}} \operatorname{diag}(H_0, \dots, H_{N-1}) \mathbf{F}$ with $\mathbf{F}$ the unitary DFT matrix. Hence $\mathbf{F}\boldsymbol{y} = \operatorname{diag}(H)\mathbf{F}\boldsymbol{x} + \mathbf{F}\boldsymbol{w}$, and $\mathbf{F}\boldsymbol{w}$ is white with the same variance because $\mathbf{F}$ is unitary. □

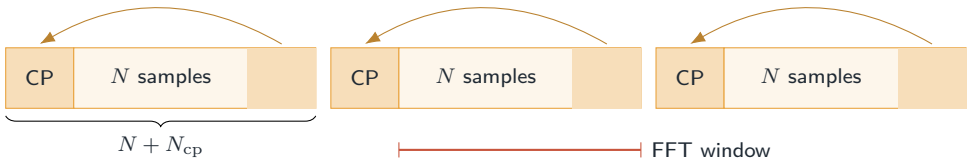


Figure 11.1: OFDM symbols with cyclic prefix. Any FFT window starting inside the prefix (after the channel transient) sees a cyclically shifted symbol, which only multiplies X_k by a linear phase.

> **In the code** `ofdm_modulator` builds an `OFDMLayout`, maps data and pilots onto the grid, applies `np.fft.ifft(..., norm="ortho")` and adds the prefix. The unitary transform keeps the noise variance per subcarrier equal to the noise variance per time sample.
> It sets `sps`$= N/N_{\mathrm{act}}$ and `power`$= (N_{\mathrm{data}}P_d + N_{\mathrm{pil}})/N$. By Equation (5.3) the AWGN block in E_s/N_0 mode then produces $\sigma^2 = 1/(E_s/N_0)$, the per-subcarrier SNR. Cyclic-prefix and pilot overheads are deliberately not counted in E_s.

Code 11.1: Resource mapping, unitary IFFT and cyclic prefix. `linkstudio/phy/waveform/ofdm.py`, lines 94–108

```
94 n_sym = -(-n // lay.n_data)
```

```
data = np.zeros((B, n_sym * lay.n_data), dtype=np.complex128)
data[:, :n] = x.data
grid = np.zeros((B, n_sym, lay.n_fft), dtype=np.complex128)
grid[..., lay.bins(lay.data_freqs)] = data.reshape(B, n_sym, lay.n_data)
grid[..., lay.bins(lay.pilot_freqs)] = lay.pilot_values
time = np.fft.ifft(grid, axis=-1, norm="ortho")
with_cp = np.concatenate([time[..., lay.n_fft - lay.cp_len:], time], axis=-1)
fs = self.params.subcarrier_spacing * lay.n_fft
data_power = (x.meta.get("power") or 1.0)
n_pilots = lay.pilot_freqs.size
meta = {**x.meta, "ofdm": lay, "n_data_symbols": n, "sps": lay.n_fft / lay.active_freqs.
    size,
        "delay": 0.0, "data_power": data_power,
        "power": (lay.n_data * data_power + n_pilots) / lay.n_fft}
frame = with_cp.reshape(B, -1)
```

11.2 Subcarrier layout

N_{act} subcarriers are used around DC. DC itself is left empty, as in Wi-Fi and LTE, and the remaining edge subcarriers act as guard bands. Every `pilot_spacing`-th active subcarrier carries a known pilot. The last active subcarrier is always a pilot, so that interpolation never extrapolates. Pilot values are a fixed pseudo-random QPSK sequence; identical pilots would add up coherently into a periodic time-domain peak.

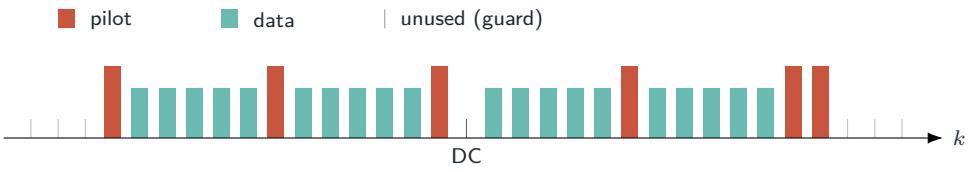


Figure 11.2: Layout with $N_{\text{act}} = 26$ active subcarriers and a pilot every 6 (the first and the last active subcarriers are pilots).

11.3 Channel estimation and equalization

11.3.1 Least squares on pilots

At pilot subcarriers Equation (11.2) gives the LS estimate $\hat{H}_p = Y_p / X_p$, with error variance σ^2 since pilots have unit modulus. The channel at data subcarriers is obtained by linear interpolation, implemented as a fixed matrix from pilot to active subcarriers. Optionally the estimates are averaged over the OFDM symbols of a frame, which lowers the estimation noise when the channel is static.

11.3.2 Removing the timing phase slope

A timing offset of δ samples inside the prefix multiplies H_k by $e^{-j2\pi k\delta/N}$. This fast phase rotation across subcarriers is poorly approximated by linear interpolation of the real and imaginary parts. The estimator therefore:

1. estimates the dominant slope from products of adjacent pilots, $\hat{s} = \arg \sum_p \hat{H}_{p+1} \hat{H}_p^* \,/\, \Delta_p$;
2. removes it;
3. interpolates the now slowly varying response;

4. restores the slope.

Code 11.2: LS estimation with phase-slope removal (`ofdm_channel_estimator`).
`linkstudio/phy/waveform/ofdm.py`, lines 235–251

```
hp = grid.data[..., lay.bins(lay.pilot_freqs)] / lay.pilot_values
if self.params.average_symbols:
    hp = np.broadcast_to(hp.mean(axis=1, keepdims=True), hp.shape)
# Remove the linear phase of the dominant delay (timing offset inside the CP)
# before interpolating, then restore it.
pf = lay.pilot_freqs
gaps = np.diff(pf)
if gaps.size:
    step = np.bincount(gaps).argmax()
    pairs = np.nonzero(gaps == step)[0]
    slope = np.angle(np.sum(hp[..., pairs + 1] * np.conj(hp[..., pairs]), axis=-1)) / step
else:
    slope = np.zeros(hp.shape[:-1])
hp = hp * np.exp(-1j * slope[..., None] * pf)
interp = self._interp_matrix(lay)
H = np.ones((B, n_sym, n_fft), dtype=np.complex128)
H[..., lay.bins(lay.active_freqs)] = (hp @ interp.T) * np.exp(1j * slope[..., None] * lay.
    active_freqs)
```

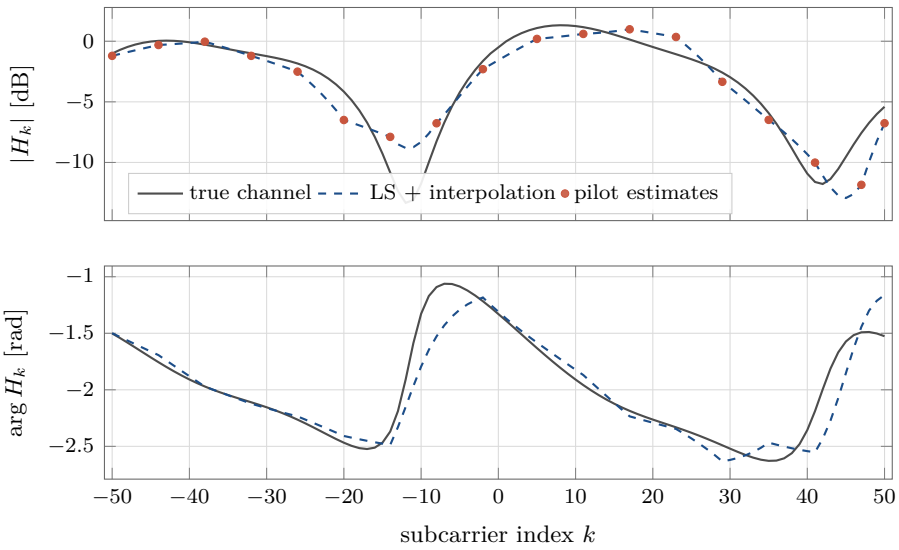


Figure 11.3: One OFDM symbol over the EVA channel ($N = 128$, 100 active subcarriers, pilot every 6, 15 kHz spacing, $E_s/N_0 = 15$ dB): true response from the channel's `csi` (`perfect` estimator) and LS estimate.

11.3.3 One-tap equalization

`ofdm_equalizer` extracts the data subcarriers and divides by the estimate, $\hat{X}_k = Y_k / \hat{H}_k$. It writes the per-subcarrier noise variance $\sigma^2/|\hat{H}_k|^2$ to `noise_var`. With coding and interleaving across subcarriers, the soft demapper then gives low weight to faded subcarriers. This is how OFDM obtains frequency diversity (see the Wi-Fi and LTE scenarios).

11.4 Blind timing and frequency synchronization

The cyclic prefix is a copy of the end of each symbol, so the received signal is correlated with itself N samples later over the prefix. Van de Beek *et al.* [10] use this redundancy. With a CFO

of ε subcarrier spacings, $r[k+N] \approx r[k]e^{j2\pi\varepsilon}$ inside the prefix, so the correlation

$$\gamma(\theta) = \sum_{m} \sum_{k=\theta+mL}^{\theta+mL+N_{\mathrm{cp}}-1} r[k]\, r^{*}[k+N], \qquad L = N + N_{\mathrm{cp}}, \tag{11.3}$$

peaks in magnitude at the symbol start, and its phase gives the fractional CFO:

$$\hat{\theta} = \arg\max_{\theta} |\gamma(\theta)|, \qquad \hat{\varepsilon} = -\frac{1}{2\pi} \arg \gamma(\hat{\theta}), \quad |\varepsilon| < \tfrac{1}{2}. \tag{11.4}$$

The block accumulates γ over all OFDM symbols of the frame with cumulative sums, which makes the search over θ linear in the frame length. It then starts the FFT window `backoff` samples early, inside the prefix, to stay clear of the channel's delay spread. The resulting linear phase is absorbed by the channel estimator. Compared with the full ML metric of van de Beek, the energy correction term is omitted; it matters only at low SNR.

Code 11.3: CP correlation, timing and fractional CFO (`ofdm_sync`).
linkstudio/phy/waveform/ofdm.py, lines 148–159

```
148 prod = r[:, :n - N] * np.conj(r[:, N:])  # r[k] r*[k+N]
149 csum = np.concatenate([np.zeros((B, 1)), np.cumsum(prod, axis=1)], axis=1)
150 n_sym = (n - N - cp - D) // L + 1
151 theta = np.arange(D + 1)
152 gamma = np.zeros((B, D + 1), dtype=np.complex128)
153 for m in range(max(n_sym, 1)):
154     start = theta + m * L
155     gamma += csum[:, start + cp] - csum[:, start]
156 d = np.argmax(np.abs(gamma), axis=1)
157 eps = -np.angle(gamma[np.arange(B), d]) / (2 * np.pi) if self.params.correct_cfo else np.
        zeros(B)
158 backoff = self.params.backoff if self.params.backoff is not None else cp // 4
159 d = np.maximum(d - backoff, 0)
```

Watch out An integer CFO (a multiple of the subcarrier spacing) shifts the whole grid and is invisible to the CP correlation. It must be resolved with a preamble or pilots, which this block does not do.

11.5 Peak-to-average power ratio

Each time sample is a sum of N_{act} independent terms. By the central limit theorem it is nearly $\mathcal{CN}(0, P)$, so $|x|^2/P$ is approximately exponential. Treating the N Nyquist-rate samples of a symbol as independent gives the classic approximation

$$P(\mathrm{PAPR} > \gamma) \approx 1 - \left(1 - e^{-\gamma}\right)^{N_{\mathrm{act}}}. \tag{11.5}$$

Remark. The modulator produces Nyquist-rate samples, which miss part of the analog peaks; with oversampling the PAPR is typically about 0.5 dB higher.

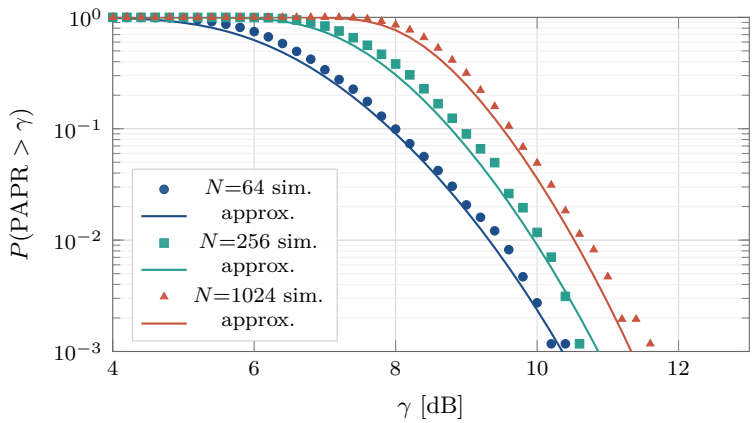


Figure 11.4: CCDF of the per-symbol PAPR of QPSK-OFDM at the output of `ofdm_modulator` (no oversampling, no prefix) and the approximation above. A PAPR of 10–12 dB forces the power-amplifier back-off discussed in Chapter 8.

Chapter 12

Multiple antennas

12.1 The flat-fading MIMO channel

With N_t transmit and N_r receive antennas and flat fading, one channel use is

$$\boldsymbol{y} = \mathbf{H}\boldsymbol{x} + \boldsymbol{w}, \qquad \mathbf{H} = \frac{1}{\sqrt{N_t}}\mathbf{G}, \quad G_{ij} \sim \mathcal{CN}(0,1), \quad \boldsymbol{w} \sim \mathcal{CN}(\mathbf{0}, \sigma^2\mathbf{I}). \tag{12.1}$$

The normalization by $\sqrt{N_t}$ keeps the total transmitted power independent of the number of antennas. Every receive antenna then sees unit average signal power, and the SNR set in the `awgn` block is the SNR *per receive antenna*. Antenna correlation follows the Kronecker model with exponential correlation matrices,

$$\mathbf{H} = \frac{1}{\sqrt{N_t}}\,\mathbf{R}_r^{1/2}\mathbf{G}\,\mathbf{R}_t^{1/2}, \qquad [\mathbf{R}]_{ij} = \rho^{|i-j|}. \tag{12.2}$$

In the code Multi-stream signals have shape (B, N_t, n).

- `stream_demux` distributes symbols round-robin over the streams and sets `spatial_rate`$= N_t$, so that E_b/N_0 accounts for the N_t symbols sent per channel use (Equation (5.4)).
- `mimo_channel` draws $\mathbf{H}$ per frame (`block`) or per symbol (`fast`) and publishes it on `csi` (`csi_type="mimo"`).
- `mimo_detector` and `alamouti_decoder` use it, and `stream_mux` restores a single stream.

12.2 Detection for spatial multiplexing

12.2.1 Zero forcing

$\mathbf{W}_{\mathrm{ZF}} = (\mathbf{H}^{\mathsf{H}}\mathbf{H})^{-1}\mathbf{H}^{\mathsf{H}}$ separates the streams exactly, $\hat{\boldsymbol{x}} = \boldsymbol{x} + \mathbf{W}\boldsymbol{w}$. Stream k then has noise variance $\sigma^2[(\mathbf{H}^{\mathsf{H}}\mathbf{H})^{-1}]_{kk}$, which becomes large when $\mathbf{H}$ is ill-conditioned. For $N_t = N_r$ each stream has only one degree of diversity.

12.2.2 MMSE and its bias

The linear MMSE estimator is $\mathbf{W} = (\mathbf{H}^{\mathsf{H}}\mathbf{H} + \sigma^2\mathbf{I})^{-1}\mathbf{H}^{\mathsf{H}}$ (unit-power symbols). Its output is biased: $\mathbb{E}[\hat{x}_k | x_k] = b_k x_k$ with $b_k = (\mathbf{WH})_{kk} < 1$.

Proposition 12.1. For the MMSE filter, the residual interference plus noise of stream k has variance $b_k(1-b_k)$. After unbiasing ($\hat{x}_k/b_k$), the noise variance is therefore $(1-b_k)/b_k$, and the post-detection SINR is $b_k/(1-b_k)$.

Proof. For the MMSE filter, the error covariance is $\mathbf{E} = \mathbb{E}[(\hat{\boldsymbol{x}} - \boldsymbol{x})(\hat{\boldsymbol{x}} - \boldsymbol{x})^{\mathsf{H}}] = \mathbf{I} - \mathbf{WH}$, so the MSE of stream k is $1 - b_k$. Write $\hat{x}_k = b_k x_k + \nu_k$ with ν_k uncorrelated with x_k (an orthogonality property of the MMSE estimator). Then $\mathrm{MSE}_k = (1-b_k)^2 + \operatorname{Var}\nu_k = 1 - b_k$, which gives $\operatorname{Var}\nu_k = b_k(1-b_k)$. □

The block outputs the unbiased symbols and writes $(1-b_k)/b_k$ to `noise_var`, so the soft demapper receives correctly scaled LLRs.

Code 12.1: ZF and unbiased MMSE detection with post-detection noise variances.
linkstudio/phy/mimo.py, lines 152–161

```
gram = Hh @ H
reg = n0 if self.params.method == "mmse" else 0.0
inv = np.linalg.inv(gram + reg * np.eye(nt))
s = (inv @ Hh @ yv)[..., 0]  # (B, n, Nt)
if self.params.method == "zf":
    nv = n0 * np.real(np.diagonal(inv, axis1=-2, axis2=-1))
else:
    bias = np.real(np.diagonal(inv @ gram, axis1=-2, axis2=-1))  # (W H)_kk
    s = s / bias
    nv = (1 - bias) / np.maximum(bias, 1e-12)
```

12.2.3 Maximum likelihood

$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathcal{S}^{N_t}} \|\boldsymbol{y}-\mathbf{H}\boldsymbol{x}\|^2$ is optimal and achieves diversity N_r per stream. The block searches all M^{N_t} candidates exhaustively (limited to 4096) and outputs hard decisions.

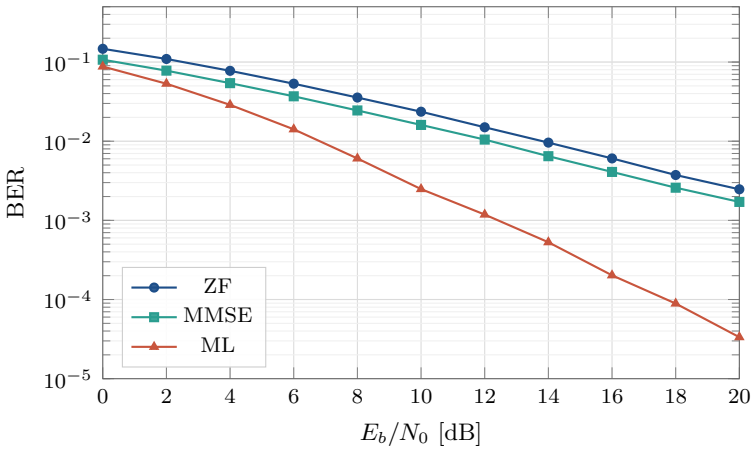


Figure 12.1: 2×2 spatial multiplexing of QPSK over fast Rayleigh fading (simulated). ZF and MMSE have diversity 1, the same slope as single-antenna Rayleigh fading, and MMSE gains about 2 dB. ML reaches diversity 2.

12.3 Diversity

12.3.1 Receive diversity: maximal-ratio combining

With one transmit antenna, ZF and MMSE reduce to maximal-ratio combining, $\hat{x} = \boldsymbol{h}^{\mathsf{H}}\boldsymbol{y}/\|\boldsymbol{h}\|^2$, whose SNR is the sum of the branch SNRs. For BPSK with L i.i.d. Rayleigh branches of mean SNR $\bar{\gamma}$ each [1],

$$\bar{P}_b = p^L \sum_{l=0}^{L-1} \binom{L-1+l}{l} (1-p)^l, \qquad p = \frac{1-\mu}{2}, \quad \mu = \sqrt{\frac{\bar{\gamma}}{1+\bar{\gamma}}}, \tag{12.3}$$

and asymptotically $\bar{P}_b \propto \bar{\gamma}^{-L}$: the slope of the curve is the diversity order.

12.3.2 Transmit diversity: the Alamouti code

Two symbols are sent over two antennas in two symbol periods [11]:

	antenna 1	antenna 2
t_1	s_1	s_2
t_2	$-s_2^*$	s_1^*

$$\Longrightarrow \qquad \begin{aligned} \tilde{s}_1 &= h_1^* r_1 + h_2 r_2^* = (|h_1|^2 + |h_2|^2)\, s_1 + \tilde{w}_1, \\ \tilde{s}_2 &= h_2^* r_1 - h_1 r_2^* = (|h_1|^2 + |h_2|^2)\, s_2 + \tilde{w}_2. \end{aligned} \tag{12.4}$$

The code is orthogonal, so the linear combiner decouples the symbols and gives full diversity 2 without channel knowledge at the transmitter. The decoder sums over any number of receive antennas. Because the transmit power is split between the antennas, the SNR is 3 dB below two-branch MRC: $\bar{P}_b^{\text{Alamouti}}(\bar{\gamma}) = \bar{P}_b^{\text{MRC},L=2}(\bar{\gamma}/2)$.

Code 12.2: Alamouti combining over N_r receive antennas. `linkstudio/phy/mimo.py`, lines 196–200

```
196 r1, r2 = y[:, :, 0::2], y[:, :, 1::2]  # (B, Nr, n/2)
197 h1, h2 = H[:, :, 0:1], H[:, :, 1:2]  # (B, Nr, 1)
198 norm = np.sum(np.abs(h1) ** 2 + np.abs(h2) ** 2, axis=1)  # (B, 1)
199 s1 = np.sum(np.conj(h1) * r1 + h2 * np.conj(r2), axis=1) / norm
200 s2 = np.sum(np.conj(h2) * r1 - h1 * np.conj(r2), axis=1) / norm
```

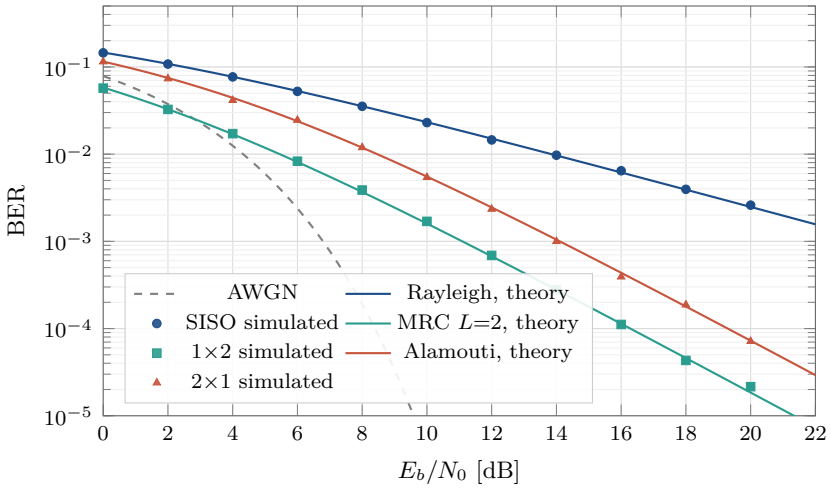


Figure 12.2: BPSK over Rayleigh fading: single antenna (Equation (9.5)), 1×2 MRC and 2×1 Alamouti (Equation (12.3)). The E_b/N_0 axis is per receive antenna. Diversity 2 doubles the slope; Alamouti runs parallel to MRC, 3 dB to the right.

Chapter 13

Channel coding

Channel coding adds structured redundancy so that the receiver can detect and correct errors. Figure 13.1 shows the coding blocks of `phy/coding/`. They can be combined freely. Every encoder multiplies the `code_rate` metadata by its own rate, including tail bits and CRC overhead, so the `awgn` block always refers E_b/N_0 to the *information* bits (Equation (5.4)).

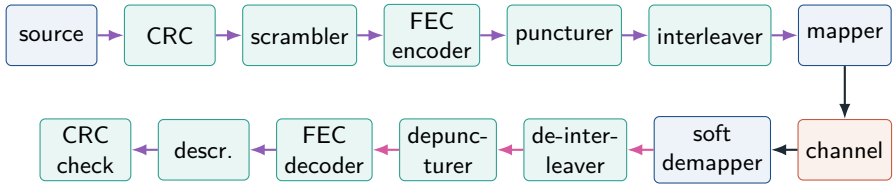


Figure 13.1: A complete coded chain. Purple edges carry bits, pink edges LLRs. The scrambler and the interleaver are their own inverses (`inverse=true` for the interleaver).

13.1 Cyclic redundancy check

A CRC with generator polynomial $g(x)$ of degree r appends the remainder $p(x) = x^r u(x) \bmod g(x)$ to the message $u(x)$; the receiver recomputes it and compares. Because polynomial division over GF(2) is linear, the parity is a linear function of the message, $\boldsymbol{p} = \boldsymbol{u}\mathbf{P} \bmod 2$. Row i of $\mathbf{P}$ is the remainder of the monomial $x^{k-1-i+r}$. Computing all of them once turns CRC encoding and checking of a whole batch into a single integer matrix product.

Code 13.1: The CRC parity matrix, built once per frame length and cached.
`linkstudio/phy/coding/crc.py`, lines 30–39

```
30 r = poly.bit_length() - 1
31 rems = np.empty(k, dtype=np.int64)
32 rem = poly ^ (1 << r)  # x^r mod g
33 for j in range(k):
34     rems[k - 1 - j] = rem
35     rem <<= 1
36     if rem >> r & 1:
37         rem ^= poly
38 shifts = np.arange(r - 1, -1, -1)
39 return ((rems[:, None] >> shifts) & 1).astype(np.int32)
```

A CRC of degree r detects all error bursts of length $\leq r$. It misses a random error pattern with probability $\approx 2^{-r}$. `crc_decoder` reports `bler`, the block error rate detected by the CRC, and outputs a validity flag per frame. CRC-8, CRC-16-CCITT, LTE CRC-24A and IEEE 802.3 CRC-32 are available.

13.2 LFSR sequences and scrambling

A linear feedback shift register produces $s[k] = \bigoplus_{t \in \mathcal{T}} s[k-t]$. When the feedback polynomial $1 + \sum_{t \in \mathcal{T}} x^t$ is primitive of degree m, the sequence is an m-sequence of period $2^m - 1$ with nearly ideal autocorrelation. The `prbs` source (PRBS-7 to PRBS-31, ITU-T O.150) and the additive `scrambler` (default $x^7 + x^4 + 1$, IEEE 802.11) are built on it. Since every output bit

depends only on bits at least $\min \mathcal{T}$ positions back, chunks of that many bits can be computed with vector operations:

Code 13.2: Vectorized LFSR generation (`dsp/lfsr.py`). linkstudio/phy/dsp/lfsr.py, lines 27–40

```
deg, step = max(taps), min(taps)
s = np.empty(deg + n, dtype=np.uint8)
s[:deg] = 1 if state is None else state
pos = deg
# Every output depends on bits at least `step` positions back, so chunks of
# `step` bits can be produced with vector operations.
while pos < deg + n:
    m = min(step, deg + n - pos)
    acc = np.zeros(m, dtype=np.uint8)
    for t in taps:
        acc ^= s[pos - t:pos - t + m]
    s[pos:pos + m] = acc
    pos += m
return s[deg:].copy(), s[n:].copy()
```

Scrambling is a XOR with the sequence and is its own inverse. On LLRs, descrambling flips the sign of the affected values. The self-synchronizing $x^{58} + x^{39} + 1$ scrambler of 64B/66B is described in Section 14.1.

13.3 Interleaving

Fading and decoders both produce errors in bursts, while most codes are designed for independent errors. An interleaver permutes the coded bits, and the de-interleaver applies the inverse permutation, so a burst on the channel is spread over many codewords. The `block` interleaver writes row by row into an $R \times (n/R)$ matrix and reads column by column. The `random` interleaver applies a seeded pseudo-random permutation.

13.4 Block codes: repetition and Hamming

The repetition code sends every bit n times. Its soft decoder simply adds the n LLRs. The Hamming code $(2^m - 1, 2^m - 1 - m)$ has as parity-check matrix $\mathbf{H}$ all non-zero m-bit columns, arranged so that the code is systematic. A single error in position j produces the syndrome $\boldsymbol{s} = \mathbf{H}\boldsymbol{e}^{\mathsf{T}} = \boldsymbol{h}_j$, which identifies the position; a lookup table maps each syndrome to the bit to flip. Decoding errors occur with ≥ 2 channel errors; the approximation

$$P_b \approx \frac{1}{n} \sum_{j=2}^{n} (j+1) \binom{n}{j} p^j (1-p)^{n-j}, \qquad p = \mathrm{Q}\left(\sqrt{2R\,E_b/N_0}\right), \tag{13.1}$$

counts the j channel errors plus at most one wrong correction (`ber_hamming_hard`).

13.5 Convolutional codes and the Viterbi algorithm

13.5.1 Encoder and trellis

A rate-$1/n$ feed-forward convolutional encoder with constraint length K computes n output bits per input bit,

$$c_j[t] = \bigoplus_{i=0}^{K-1} g_{j,i}\, u[t-i], \qquad j = 1, \ldots, n, \tag{13.2}$$

with generators given in octal: $(171, 133)_8$ is the industry-standard $K = 7$ code of IEEE 802.11, DVB and many others, with free distance $d_{\text{free}} = 10$. The encoder state is the last $K - 1$ input bits, giving 2^{K-1} states. Appending $K - 1$ zeros (`terminate`) returns the trellis to state 0.

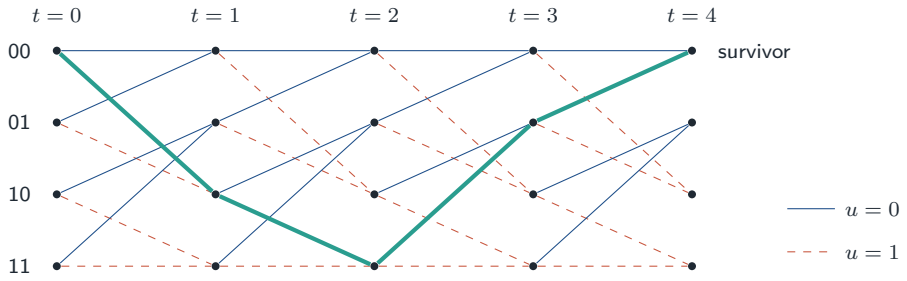


Figure 13.2: Trellis of the $K = 3$, $(7, 5)_8$ code (4 states, MSB = most recent input). The Viterbi algorithm keeps one survivor path per state and time; the green path is the decoded sequence `1 1 0 0`.

13.5.2 Maximum-likelihood sequence decoding

Given channel LLRs $L_{t,j}$ for the coded bits, the log-likelihood of a candidate codeword is, up to a constant,

$$\ln p(\boldsymbol{y} \mid \boldsymbol{c}) = \sum_{t,j} \frac{L_{t,j}}{2}(1 - 2c_{t,j}) + \text{const}, \tag{13.3}$$

a *correlation* metric. The Viterbi algorithm maximizes it over all paths of the trellis by dynamic programming. At each time, for every state it keeps the better of the two incoming paths (add–compare–select) and stores the decision. A traceback from state 0 (terminated trellis) recovers the input bits. Path metrics are renormalized every step to avoid overflow.

Code 13.3: Viterbi decoding vectorized over states and frames (`Trellis.decode`).
linkstudio/phy/coding/convolutional.py, lines 67–86

```
S = self.n_states
pm = np.full((B, S), -np.inf)
pm[:, 0] = 0.0
decisions = np.empty((T, B, S), dtype=bool)
for t in range(T):
    bm0 = 0.5 * llr[:, t, :] @ self.sign0.T  # (B, S) correlation metrics
    bm1 = 0.5 * llr[:, t, :] @ self.sign1.T
    c0 = pm[:, self.prev0] + bm0
    c1 = pm[:, self.prev1] + bm1
    d = c1 > c0
    decisions[t] = d
    pm = np.where(d, c1, c0)
    pm -= pm.max(axis=1, keepdims=True)
state = np.zeros(B, dtype=np.int64) if terminated else np.argmax(pm, axis=1)
bits = np.empty((B, T), dtype=np.uint8)
rows = np.arange(B)
for t in range(T - 1, -1, -1):
    bits[:, t] = self.input_bit[state]
    state = np.where(decisions[t, rows, state], self.prev1[state], self.prev0[state])
return bits[:, :T - (self.K - 1)] if terminated else bits
```

With hard decisions (bit inputs) the metric degenerates to the Hamming distance, losing about 2 dB (`tests/test_physics.py::test_conv_coded_soft_vs_hard`). The asymptotic coding gain of soft decoding is $10 \log_{10}(R\, d_{\text{free}}) = 7$ dB for the $K = 7$ code.

13.5.3 Puncturing

Deleting coded bits periodically raises the rate without a new decoder. The default pattern `[1,1,1,0,0,1]` turns the rate-1/2 code into rate 3/4, as in IEEE 802.11. The depuncturer reinserts the deleted positions as erasures ($L = 0$, no information) before decoding. When the original length is not available in the metadata, it is inferred as the shortest mother length whose pattern keeps exactly the received number of bits.

Code 13.4: Erasure insertion (`depuncturer`). linkstudio/phy/coding/puncture.py, lines 55–64

```
m = llr.shape[-1]
n = x.meta.get("punctured_length")
if n is None:  # shortest mother length whose pattern keeps exactly m positions
    period = len(self.params.pattern)
    big = _keep_mask(self.params.pattern, (m // sum(self.params.pattern) + 1) * period)
    n = int(np.nonzero(np.cumsum(big) == m)[0][0]) + 1
keep = _keep_mask(self.params.pattern, n)
out = np.zeros((*llr.shape[:-1], n))
out[..., keep] = llr
return {"out": Signal(out, Kind.LLR, meta=x.meta)}
```

13.6 LDPC codes

13.6.1 Structure

A low-density parity-check code [13] is the null space of a sparse matrix $\mathbf{H} \in \{0,1\}^{m \times n}$: $\mathbf{H}\boldsymbol{c}^{\mathsf{T}} = \mathbf{0}$. Its Tanner graph has a variable node per code bit and a check node per row (Figure 13.3). LinkStudio builds quasi-cyclic codes in the style of IEEE 802.11n and 5G NR: $\mathbf{H}$ is an array of $z \times z$ circulant permutation matrices (or zero blocks), described by a base matrix of shifts (Figure 13.4).

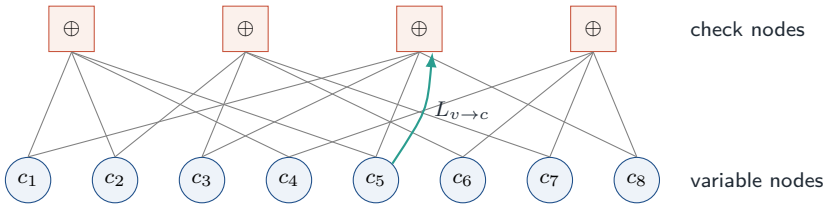


Figure 13.3: A Tanner graph. Belief propagation exchanges messages along the edges: variables tell checks what they believe, checks reply with what the other bits imply.

13.6.2 Encoding by accumulation

Split $\mathbf{H} = [\mathbf{H}_i \,|\, \mathbf{H}_p]$ and the codeword $\boldsymbol{c} = [\boldsymbol{u} \,|\, \boldsymbol{p}]$. The parity part is block dual-diagonal: identities on the diagonal and on the sub-diagonal. The parity equations $\mathbf{H}_i\boldsymbol{u}^{\mathsf{T}} \oplus \mathbf{H}_p\boldsymbol{p}^{\mathsf{T}} = \mathbf{0}$ then read, block by block,

$$\boldsymbol{p}_0 = \boldsymbol{s}_0, \qquad \boldsymbol{p}_j = \boldsymbol{p}_{j-1} \oplus \boldsymbol{s}_j, \qquad \boldsymbol{s} = \mathbf{H}_i\boldsymbol{u}^{\mathsf{T}}, \tag{13.4}$$

so the parity is a cumulative XOR of the syndrome blocks. Encoding costs one sparse product and needs no Gaussian elimination. The dual-diagonal structure also guarantees that $\mathbf{H}$ has full rank.

Code 13.5: Systematic QC-LDPC encoding. linkstudio/phy/coding/ldpc.py, lines 71–74

```
s = (self.H_info @ u.T.astype(np.int32)) % 2  # (m, B)
```

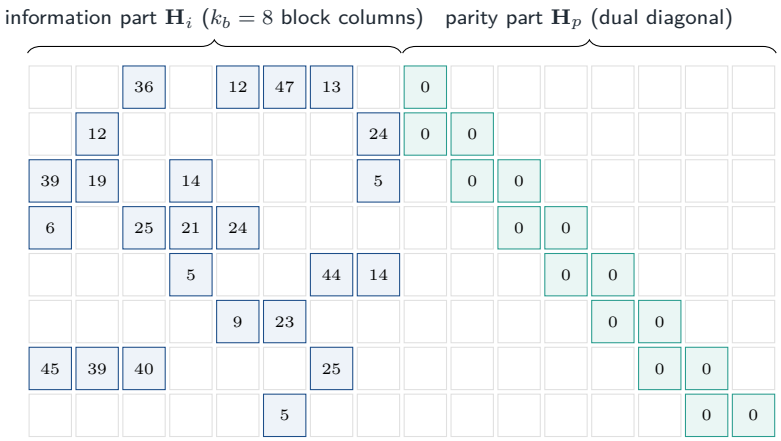


Figure 13.4: Base matrix of the default code ($k_b = m_b = 8$, $z = 48$, rate $1/2$, $n = 768$), read back from `QCLDPCCode.H`. Each number is the cyclic shift of a 48×48 identity; empty cells are zero blocks. Every information column has weight 3.

```
blocks = s.reshape(self.m // self.z, self.z, -1).astype(np.uint8)
parity = np.bitwise_xor.accumulate(blocks, axis=0).reshape(self.m, -1)
return np.concatenate([u, parity.T.astype(np.uint8)], axis=1)
```

13.6.3 Belief-propagation decoding

With channel LLRs L_v, the flooding schedule alternates two updates.

$$\text{variable to check:}\quad L_{v\to c} = L_v + \sum_{c' \in \mathcal{N}(v)\setminus c} L_{c'\to v}, \tag{13.5}$$

$$\text{check to variable:}\quad L_{c\to v} = 2\tanh^{-1}\prod_{v' \in \mathcal{N}(c)\setminus v} \tanh\frac{L_{v'\to c}}{2} \quad \text{(sum-product)}, \tag{13.6}$$

$$L_{c\to v} \approx \alpha \prod_{v'} \operatorname{sgn}(L_{v'\to c}) \cdot \min_{v' \in \mathcal{N}(c)\setminus v} |L_{v'\to c}| \quad \text{(normalized min-sum)}. \tag{13.7}$$

The min-sum rule only needs, for each check, the two smallest input magnitudes: the output to the edge holding the minimum is the second minimum, and to all others the minimum. The factor $\alpha \approx 0.8$ compensates its overestimation of reliabilities. After every iteration the hard decisions are tested against the syndrome. Frames that satisfy it stop early, and the average number of iterations is reported as a metric.

Code 13.6: The normalized min-sum check-node update, vectorized over all checks of all frames.
linkstudio/phy/coding/ldpc.py, lines 92–103

```
if method == "min_sum":
    sign = np.where(V < 0, -1.0, 1.0)
    sign[:, ~valid] = 1.0
    mag = np.abs(V)
    mag[:, ~valid] = np.inf
    i1 = np.argmin(mag, axis=-1)[..., None]
    min1 = np.take_along_axis(mag, i1, -1)
    np.put_along_axis(mag, i1, np.inf, -1)
    min2 = mag.min(axis=-1, keepdims=True)
    pos = np.arange(V.shape[-1])
    out_mag = np.where(pos == i1, min2, min1) * alpha
    out = np.prod(sign, axis=-1, keepdims=True) * sign * out_mag
```

The message passing is vectorized with a padded table of the edges of every check node (irregular row weights are padded with neutral entries) and sparse incidence matrices for the variable-node sums.

13.7 Polar codes

13.7.1 Channel polarization

Arıkan's transform [12] $\boldsymbol{x} = \boldsymbol{u}\mathbf{G}_N$, with $\mathbf{G}_N = \left[\begin{smallmatrix}1 & 0\\ 1 & 1\end{smallmatrix}\right]^{\otimes n}$, $N = 2^n$, combines N uses of a channel W into N synthetic channels $W_N^{(i)}$ seen by the bits u_i under successive decoding. As N grows they polarize: a fraction tends to noiseless channels, and the rest to useless ones. One step splits a channel with Bhattacharyya parameter Z into a worse and a better one,

$$Z(W^-) \le 2Z - Z^2, \qquad Z(W^+) = Z^2. \tag{13.8}$$

Starting from $Z = e^{-E_s/N_0}$ (BPSK over AWGN at the `design_snr_db`) and applying the recursion n times ranks the synthetic channels. The K best carry information and the others are *frozen* to 0.

Code 13.7: Reliability ranking by the Bhattacharyya recursion. `linkstudio/phy/coding/polar.py`, lines 35–39

```
def rec(z: float, size: int) -> list[float]:
    if size == 1:
        return [z]
    return rec(2 * z - z * z, size // 2) + rec(z * z, size // 2)
return np.array(rec(float(np.exp(-10 ** (design_snr_db / 10))), 1 << n_log2))
```

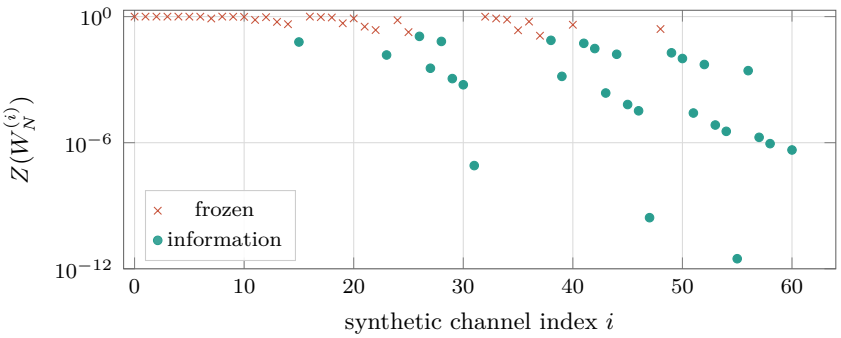


Figure 13.5: Polarization for $N = 64$, $K = 32$ at a design SNR of 0 dB (`reliability`, `frozen_mask`). Reliable channels concentrate at high indices, the frozen set at low indices.

13.7.2 Encoding and successive-cancellation decoding

Without bit reversal, $\boldsymbol{u} = [\boldsymbol{u}_a, \boldsymbol{u}_b]$ encodes recursively as $\boldsymbol{x} = [\mathrm{enc}(\boldsymbol{u}_a) \oplus \mathrm{enc}(\boldsymbol{u}_b),\ \mathrm{enc}(\boldsymbol{u}_b)]$, computed in place with $\log_2 N$ butterfly stages:

Code 13.8: The polar transform in $\log_2 N$ vectorized stages. `linkstudio/phy/coding/polar.py`, lines 44–52

```
x = u.copy()
n = x.shape[-1]
half = 1
while half < n:
    x = x.reshape(*x.shape[:-1], -1, 2, half)
```

```
    x[..., 0, :] ^= x[..., 1, :]
    x = x.reshape(*x.shape[:-3], n)
    half *= 2
return x
```

The SC decoder follows the same recursion on LLRs. For the halves $\boldsymbol{L}_1, \boldsymbol{L}_2$,

$$f(a,b) = \operatorname{sgn} a \operatorname{sgn} b \min(|a|,|b|) \ \text{(left child)}, \qquad g(a,b,\hat{x}) = b + (1-2\hat{x})\, a \ \text{(right child)}. \tag{13.9}$$

The left half is decoded first; its re-encoded decisions $\hat{\boldsymbol{x}}_a$ enter the right half. Subtrees made only of frozen bits return zeros immediately, and subtrees made only of information bits (rate-1 nodes) are decided by a direct hard decision. Both shortcuts are exact.

Code 13.9: Recursive SC decoding with rate-0 and rate-1 node shortcuts.
linkstudio/phy/coding/polar.py, lines 59–83

```
def sc_decode(llr: np.ndarray, frozen: np.ndarray) -> np.ndarray:
    """Successive-cancellation decoding. Returns the estimated u ``(B, N)``."""
    u_hat = np.zeros(llr.shape, dtype=np.uint8)

    def rec(L: np.ndarray, lo: int) -> np.ndarray:
        n = L.shape[-1]
        mask = frozen[lo:lo + n]
        if mask.all():
            return np.zeros(L.shape, dtype=np.uint8)
        if not mask.any():  # rate-1 node: direct hard decision
            x = (L < 0).astype(np.uint8)
            u_hat[:, lo:lo + n] = polar_encode(x)
            return x
        if n == 1:
            x = (L < 0).astype(np.uint8)
            u_hat[:, lo:lo + 1] = x
            return x
        h = n // 2
        L1, L2 = L[:, :h], L[:, h:]
        xa = rec(_f(L1, L2), lo)
        xb = rec(L2 + (1 - 2.0 * xa) * L1, lo + h)
        return np.concatenate([xa ^ xb, xb], axis=1)

    rec(llr, 0)
    return u_hat
```

13.8 Comparison

Key idea Soft decisions are worth about 2 dB, and iterative decoding of long sparse codes approaches capacity. The rate-1/2 limit for binary-input AWGN is 0.19 dB. The 768-bit LDPC code is about 3 dB away at 10^{-5}; longer codes get closer.

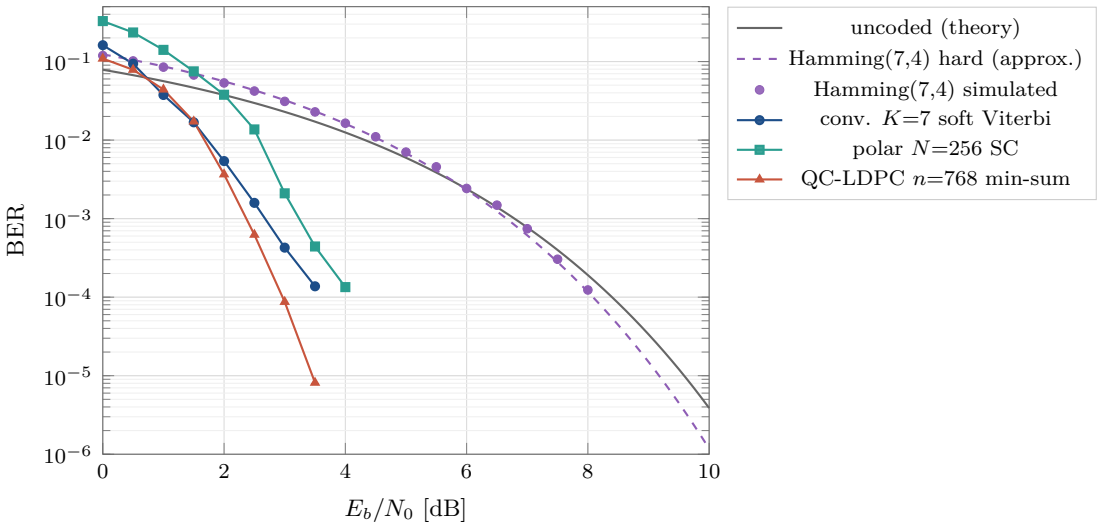


Figure 13.6: QPSK over AWGN with the codes of this chapter (rate $\approx 1/2$ except Hamming 4/7; max-log LLRs; simulated with the real blocks, up to 200 errors or 20 batches per point). At 10^{-4} the Hamming code gains well under 1 dB with hard decisions, while the convolutional and LDPC codes gain about 5 and 5.5 dB.

Chapter 14

Wireline communications

Copper links (Ethernet over twisted pair, SerDes over backplanes, DSL over telephone loops) are real-valued baseband systems. Their channel is a lossy, dispersive transmission line, and their noise is dominated by crosstalk, echo and jitter rather than thermal noise. `phy/wireline.py` models these effects; all its blocks keep real signals real.

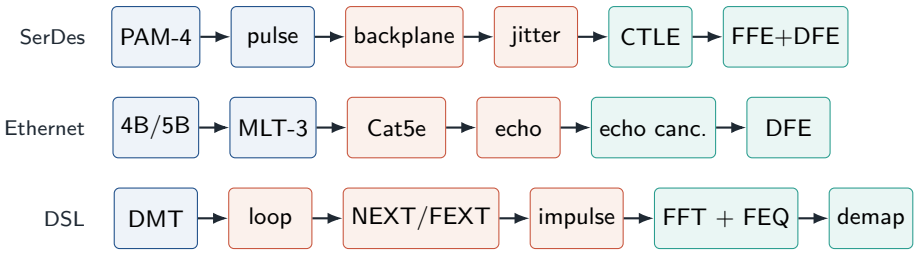


Figure 14.1: Simplified chains of the wireline scenarios (10GBASE-KR / PAM-4 SerDes, 100BASE-TX, ADSL/VDSL).

14.1 Line codes

A line code shapes the spectrum of the transmitted symbols: it removes DC (transformer coupling), guarantees transitions (clock recovery) or concentrates energy at low frequency (cable loss, emissions). `line_encoder` implements four codes:

code	rule	levels	PSD with rectangular pulses (T = bit time)
NRZ	$1 \to +1$, $0 \to -1$	2	$T\,\mathrm{sinc}^2(fT)$
Manchester	$0 \to (+1,-1)$, $1 \to (-1,+1)$	2	$T\,\mathrm{sinc}^2(fT/2)\sin^2(\pi fT/2)$
AMI	$0 \to 0$, $1 \to \pm 1$ alternately	3	$T\,\mathrm{sinc}^2(fT)\sin^2(\pi fT)$
MLT-3	each 1 steps through $0,+1,0,-1$	3	concentrated below $1/(4T)$

Manchester and AMI have a spectral null at DC. MLT-3, used by 100BASE-TX, completes a full cycle every four ones, so its fundamental is at most a quarter of the bit rate.

Block line codes. `line_block_encoder` implements two block codes.

- **4B/5B** maps each nibble to a 5-bit word with at most one leading and two trailing zeros. Long zero runs are thus excluded, at rate 0.8.
- **64B/66B** prepends the sync header `01` to each 64-bit block and scrambles the payload with the self-synchronizing polynomial $x^{58} + x^{39} + 1$, $s[n] = d[n] \oplus s[n-39] \oplus s[n-58]$. The descrambler needs no synchronization, but each channel error produces three errors after descrambling (error multiplication, tested in `tests/test_wireline_optical.py`).

14.2 The cable as a transmission line

A uniform line of length ℓ, terminated in its characteristic impedance, has transfer function $H(f) = e^{-\gamma(f)\ell}$ with propagation constant $\gamma = \sqrt{(R + j\omega L)(G + j\omega C)} = \alpha + j\beta$. Two loss mechanisms dominate:

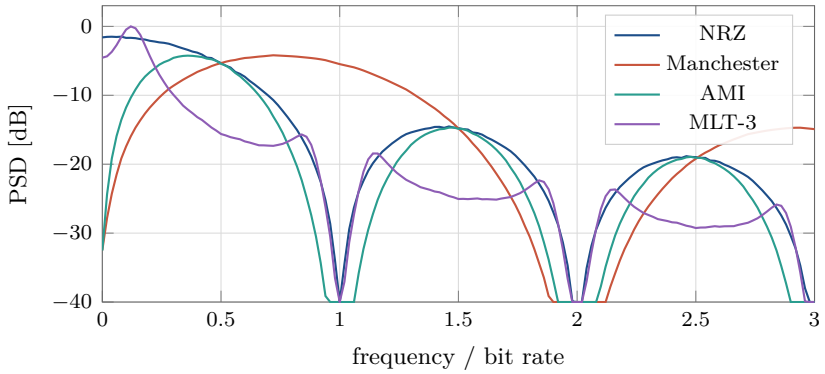


Figure 14.2: Welch spectra of random data through `line_encoder` and a rectangular `pulse_shaper`, normalized to the NRZ peak and clipped at −40 dB. Note the DC nulls of Manchester and AMI, and Manchester's doubled bandwidth.

- the *skin effect*: current crowds near the conductor surface, so $R \propto \sqrt{f}$ and the attenuation $\alpha_s \propto \sqrt{f}$;
- *dielectric loss*: $G \propto f$ gives $\alpha_d \propto f$.

In dB per km,

$$\alpha_{\mathrm{dB}}(f) = a\sqrt{f_{\mathrm{MHz}}} + b\, f_{\mathrm{MHz}}. \tag{14.1}$$

Causality fixes the phase A causal, minimum-phase channel cannot have an arbitrary attenuation: the log-magnitude and the phase form a Hilbert-transform pair. For the skin effect the pair is exact and elementary. The function $e^{-k\sqrt{j\omega}}$ is analytic in the right half s-plane (it is the Laplace-domain response of a diffusion line), and

$$e^{-k\sqrt{j\omega}} = e^{-k\sqrt{\omega}\,(1+j)/\sqrt{2}}\,:$$

the $\sqrt{f}$ loss comes with an *equal* $\sqrt{f}$ phase lag. LinkStudio therefore uses

$$H(f) = \exp\Big(-\alpha_s(f)\,\ell\,(1 + j\ \mathrm{sgn}\, f) - \alpha_d(f)\,\ell\Big), \tag{14.2}$$

with α in nepers. Dropping the phase would produce a symmetric, non-causal pulse; keeping it produces the long causal tail characteristic of copper (Figure 14.3, right), which is what makes DSL and Ethernet equalizers work hard.

Code 14.1: Cable response with skin-effect phase (`wireline.py`). `linkstudio/phy/wireline.py`, lines 206–214

```
206 pa, pb = CABLE_PRESETS[preset]
207 a = pa if a is None else a
208 b = pb if b is None else b
209 f_mhz = np.abs(f) / 1e6
210 km = length_m / 1000
211 neper = np.log(10) / 20
212 skin = a * np.sqrt(f_mhz) * km * neper
213 dielectric = b * f_mhz * km * neper
214 return np.exp(-skin * (1 + 1j * np.sign(f)) - dielectric)
```

In the code `copper_line` applies $H(f)$ in the frequency domain, with zero padding to avoid circular wrap-around. It publishes the sampled impulse response, truncated at

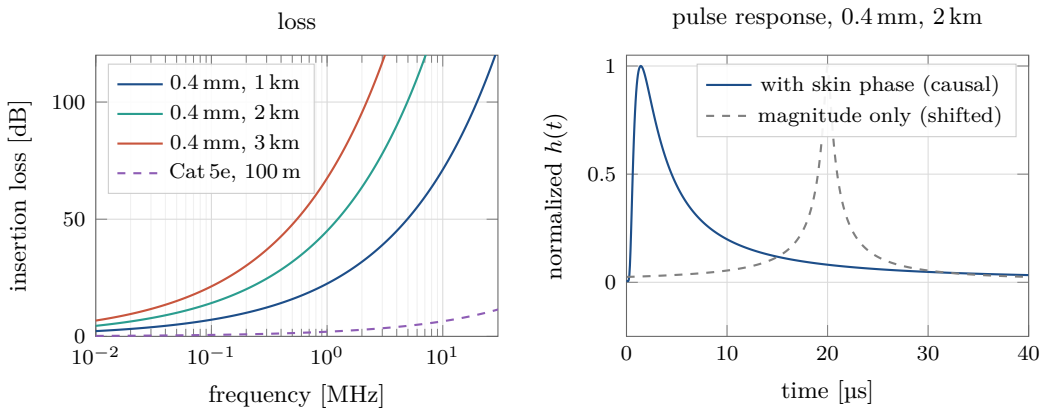


Figure 14.3: Left: insertion loss of the presets (`cable_response`). Right: impulse response at 17.664 MSa/s with the minimum-phase term (used by `copper_line`) and, for comparison, the zero-phase response with the same magnitude, shifted by 20 µs.

99.95 % of its energy, on `csi` (`csi_type="cir"`), for `linear_equalizer` or the DMT FEQ. Presets: Cat 5e, Cat 6A, 0.4 and 0.5 mm telephone pairs, RG-6 coax and an FR-4 backplane trace.

14.3 Crosstalk

Signals on neighbouring pairs of a binder couple electromagnetically. The ANSI T1.417 [21] 99 % worst-case models for N disturbers give the coupling power transfer functions

$$|H_{\mathrm{NEXT}}(f)|^2 = 8.536 \times 10^{-15}\, N^{0.6} f^{1.5}, \tag{14.3}$$

$$|H_{\mathrm{FEXT}}(f)|^2 = 7.744 \times 10^{-21}\, N^{0.6}\, \ell_{\mathrm{ft}}\, f^2\, |H_{\mathrm{line}}(f)|^2. \tag{14.4}$$

NEXT (near end) does not depend on the length and grows with frequency. FEXT (far end) travels the whole line and is attenuated by it. `crosstalk` builds the disturbance from the spectrum of the neighbour's transmit signal (input `tx`, or the signal itself) with random phases, shapes it with the model and adds it. The crosstalk-to-signal ratio is reported.

Code 14.2: ANSI crosstalk models. `linkstudio/phy/wireline.py`, lines 261–266

```
261 fa = np.abs(f)
262 if model == "next":
263     return 8.536e-15 * n ** 0.6 * fa ** 1.5
264 length_ft = length_m * 3.281
265 line_gain = np.abs(line) ** 2 if line is not None else 1.0
266 return 7.744e-21 * n ** 0.6 * length_ft * fa ** 2 * line_gain
```

14.4 Full duplex: hybrid echo and echo cancellation

Full-duplex transceivers (1000BASE-T, DSL) send and receive on the same pair. The hybrid leaks part of the local transmit signal into the receiver as *echo*, often much stronger than the far-end signal. `hybrid_echo` adds the transmit signal filtered by a short echo path; `echo_canceller` identifies that path with an NLMS filter driven by the known local transmit signal and subtracts its estimate.

NLMS converges at a speed proportional to μ, but leaves an excess mean-square error, the misadjustment $\mathcal{M} \approx \mu/(2-\mu)$ for NLMS. That excess error is noise added to the far-end signal.

The block therefore uses a *gear shift*: a large step for the first `train_samples` samples (fast acquisition, as at link start-up), then a small one (low residual). The power reduction after convergence is reported as a metric.

Code 14.3: NLMS echo cancellation with gear shifting. `linkstudio/phy/wireline.py`, lines 383–388

```
for n in range(N):
    u = ref[:, n:n + T][:, ::-1]
    e = y[:, n] - np.sum(w * u, axis=1)
    out[:, n] = e
    mu = self.params.train_step_size if n < self.params.train_samples else self.params.
    step_size
    w += mu * e[:, None] * np.conj(u) / (1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=
    True))
```

14.5 SerDes equalization

14.5.1 Continuous-time linear equalizer

A CTLE is an analog high-pass with one zero and two poles,

$$H_{\mathrm{CTLE}}(s) = A\,\frac{1 + s/\omega_z}{(1 + s/\omega_{p1})(1 + s/\omega_{p2})}, \tag{14.5}$$

providing a peaking of about $20\log_{10}(\omega_{p1}/\omega_z)$ dB that partially inverts the channel loss (Figure 14.4). It amplifies high-frequency noise and crosstalk as well, so it is followed by a decision-feedback equalizer.

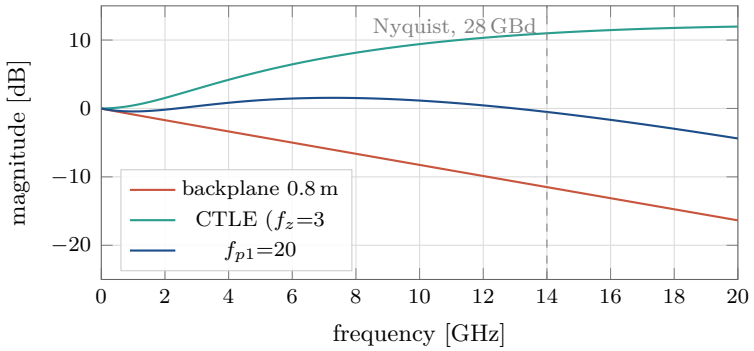


Figure 14.4: The CTLE keeps the combined response within about 2 dB up to the Nyquist frequency of a 28 GBd link (`cable_response` and `CTLE.response`). Beyond it the second pole and the growing loss take over.

14.5.2 Feed-forward plus decision-feedback equalizer

The DFE output is

$$z[n] = \sum_{i=0}^{F-1} w_i\, y[n + D - i] - \sum_{j=1}^{K} b_j\, \hat{a}[n - j], \tag{14.6}$$

where the feedback taps cancel the post-cursor ISI of past decisions $\hat{a}$. Because past decisions are noise-free when correct, the DFE removes ISI *without* noise enhancement, unlike a linear equalizer. The price is error propagation: a wrong decision feeds wrong ISI back. Both filters adapt jointly with NLMS on the stacked regressor $[\boldsymbol{u}; \hat{\boldsymbol{a}}]$, trained on `n_train` known symbols and then decision-directed.

Code 14.4: Joint NLMS adaptation of the FFE and the DFE (`dfe_equalizer`).
linkstudio/phy/wireline.py, lines 466–479

```
466 for n in range(N):
467     u = pad[:, n:n + F][:, ::-1]
468     z = np.sum(w * u, axis=1) - np.sum(bfb * past, axis=1)
469     out[:, n] = z
470     d = ref.data[:, n] if n < p.n_train else const.points[const.nearest(z)]
471     e = d - z
472     if n >= p.n_train:
473         err += float(np.sum(np.abs(e) ** 2))
474         count += B
475     norm = 1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=True) + np.sum(np.abs(past) ** 2,
        axis=1, keepdims=True)
476     w += p.step_size * e[:, None] * np.conj(u) / norm
477     if K:
478         bfb -= p.step_size * e[:, None] * np.conj(past) / norm
479         past = np.concatenate([d[:, None], past[:, :-1]], axis=1)
```

14.5.3 Jitter

Sampling-clock jitter perturbs the sampling instants, $t_n = nT_s + \Delta(n)$, with a random (Gaussian, RJ) and a sinusoidal (SJ) component expressed in unit intervals. `jitter` resamples the waveform at the perturbed instants with the cubic interpolator; RJ is drawn once per symbol and interpolated between symbols. To first order, the error is $x'(t)\,\Delta$, which is why steep edges, that is high bandwidth, make jitter costly.

Code 14.5: Perturbed sampling instants. linkstudio/phy/wireline.py, lines 503–507

```
503 n_sym = N // sps + 2
504 rj = self.rng.standard_normal((B, n_sym)) * self.params.rj_rms_ui
505 rj_t = np.stack([np.interp(n / sps, np.arange(n_sym), r) for r in rj])
506 sj = self.params.sj_amplitude_ui * np.sin(2 * np.pi * self.params.sj_frequency_hz * n / fs)
507 t = n[None, :] + sps * (rj_t + sj[None, :])
```

14.6 Discrete multitone (DSL)

14.6.1 Real OFDM

DMT is OFDM for real baseband channels. Imposing Hermitian symmetry, $X_{N-k} = X_k^*$, makes the IFFT output real. Tones $k = 1, \ldots, N/2 - 1$ carry independent QAM symbols; DC and Nyquist are unused. The cyclic prefix turns the loop into $N/2 - 1$ parallel flat channels, as in Chapter 11. The demodulator equalizes each tone with one division (FEQ) using the channel's impulse response.

Code 14.6: Per-tone mapping and Hermitian symmetry (`dmt_modulator`).
linkstudio/phy/wireline.py, lines 587–593

```
587 X = np.zeros((B, n_sym, p.n_fft), dtype=np.complex128)
588 for b, idx in self.groups.items():
589     sel = self.offsets[idx][:, None] + np.arange(b)  # (n_tones_b, b)
590     X[..., self.tones[idx]] = self.consts[b].map(bits[..., sel].reshape(B, n_sym, -1))
591 X[..., p.n_fft - self.tones] = np.conj(X[..., self.tones])
592 t = np.fft.ifft(X, axis=-1, norm="ortho").real
593 out = np.concatenate([t[..., p.n_fft - p.cp_len:], t], axis=-1).reshape(B, -1)
```

14.6.2 Bit loading and the SNR gap

For uncoded QAM at a target error rate, the achievable bits per tone are well approximated by the *gap* formula

$$b_k = \log_2\!\left(1 + \frac{\mathrm{SNR}_k}{\Gamma\,\gamma_m}\right), \qquad \Gamma \approx 9.8\,\mathrm{dB}\ (P_s = 10^{-7}), \tag{14.7}$$

where γ_m is a noise margin [14]. The block computes the expected SNR_k from the cable model and the AWGN level. With the time-domain power $P = 2N_{\mathrm{tones}}/N$ and $\sigma^2 = P/\mathrm{SNR}$, $\mathrm{SNR}_k = |H_k|^2\,\mathrm{SNR}\,N/(2N_{\mathrm{tones}})$. It then rounds b_k down to the sizes the mapper supports: 0, 1 (BPSK) or an even number (square QAM).

Code 14.7: Gap-approximation bit loading. `linkstudio/phy/wireline.py`, lines 517–520

```
gamma = 10 ** ((gap_db + margin_db) / 10)
b = np.floor(np.log2(1 + snr / gamma)).astype(int)
b = np.clip(b, 0, max_bits)
return np.where(b >= 2, b - (b % 2), b)
```

Bit loading versus water-filling The transmit PSD is flat; only the bits adapt to the channel. True water-filling would also redistribute power, pouring more where the SNR is high. With a flat PSD the loss is small at high SNR, but it is a modelling choice to keep in mind.

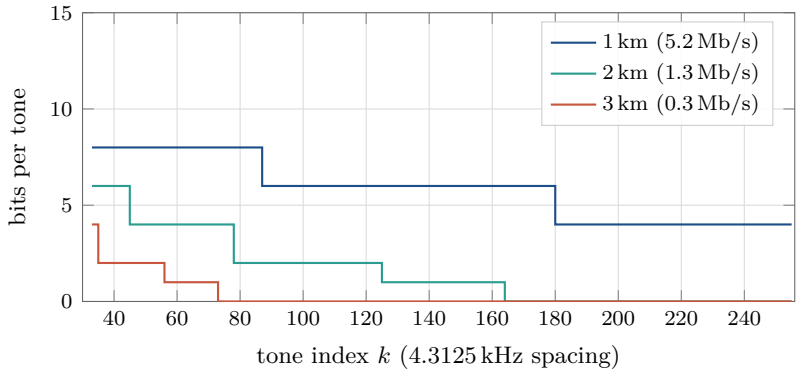


Figure 14.5: Bit loading of `dmt_modulator` on a 0.4 mm loop (ADSL downstream tones 33–255, 50 dB SNR, 9.8 dB gap, 3 dB margin). Data rates are those reported by the block.

Chapter 15

Optical communications

`phy/optical.py` models fiber-optic and free-space optical links, from the laser to the coherent receiver's digital signal processing. Figure 15.1 shows the two families of systems it supports.

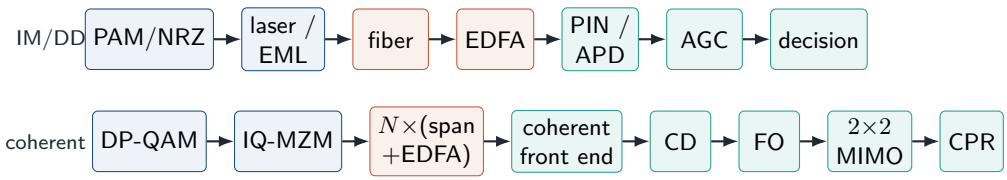


Figure 15.1: Intensity modulation with direct detection, and dual-polarization coherent transmission.

In the code Optical signals are complex field envelopes around the optical carrier, scaled so that $|E|^2$ is the instantaneous optical power in watts. Dual polarization uses the shape $(B, 2, n)$. Optical blocks maintain the following metadata:

- `wavelength_nm`;
- `power`, the average launch or received power in W;
- `cd_ps_nm`, the accumulated chromatic dispersion $D\ell$;
- `ase_psd`, the accumulated ASE density per polarization in W/Hz.

The OSNR is computed from the last two at any point of the link.

15.1 Transmitters

15.1.1 Directly modulated lasers

An intensity modulator maps the electrical drive $x \in [-1, 1]$ to the optical power

$$P(t) = P_{\text{avg}}\big(1 + m\,x(t)\big), \qquad m = \frac{\text{ER} - 1}{\text{ER} + 1}, \tag{15.1}$$

since the extinction ratio is $\text{ER} = P_1/P_0 = (1+m)/(1-m)$. Three non-idealities are modelled:

- **Relative intensity noise**: multiplicative Gaussian noise of variance $\text{RIN} \cdot f_s/2$ on P.
- **Phase noise**: a Wiener process with the laser linewidth (Equation (8.2)).
- **Transient chirp** of a directly modulated laser. The instantaneous frequency follows the power changes, $\Delta\nu(t) = \frac{\alpha}{4\pi}\frac{d}{dt}\ln P(t)$, with α the Henry linewidth-enhancement factor. Integrating gives the phase

$$\phi_{\text{chirp}}(t) = \frac{\alpha}{2}\ln\frac{P(t)}{P_{\text{avg}}}. \tag{15.2}$$

The field is $E = \sqrt{P}\,e^{j\phi}$.

Code 15.1: Laser intensity modulation with RIN, phase noise and chirp (`intensity_modulator`).
`linkstudio/phy/optical.py`, lines 90–102

```
90  drive = np.real(x.data)
```

```
peak = np.max(np.abs(drive)) or 1.0
er = 10 ** (p.extinction_ratio_db / 10)
m = (er - 1) / (er + 1)
p_avg = dbm_to_w(p.power_dbm)
power = p_avg * (1 + m * np.clip(drive / peak, -1, 1))
if p.rin_db_hz is not None:
    power = power * (1 + np.sqrt(10 ** (p.rin_db_hz / 10) * fs / 2) * self.rng.
     standard_normal(power.shape))
power = np.maximum(power, 0.0)
phase = _wiener_phase(self.rng, *power.shape, p.linewidth_hz, fs)
if p.chirp_alpha:
    phase = phase + p.chirp_alpha / 2 * np.log(np.maximum(power, 1e-12 * p_avg) / p_avg)
field = np.sqrt(power) * np.exp(1j * phase)
```

Not a rate-equation model The laser is modelled behaviourally (power, chirp, linewidth). Relaxation oscillations, the adiabatic chirp term and thermal effects, which require the laser rate equations, are not included.

15.1.2 Mach–Zehnder and IQ modulators

A Mach–Zehnder modulator in push–pull configuration, biased at null, has the field transfer function $E_{\text{out}}/E_{\text{in}} = \sin\left(\frac{\pi}{2} V/V_\pi\right)$. An IQ modulator nests two of them, with a 90° phase shift between the arms:

$$\frac{E_{\text{out}}}{E_{\text{in}}} \propto \sin\left(\frac{\pi}{2}\frac{V_I}{V_\pi}\right) + j \sin\left(\frac{\pi}{2}\frac{V_Q}{V_\pi}\right). \tag{15.3}$$

With the peak drive set to `drive_swing`$\times V_\pi$, small swings are nearly linear, while large swings compress the outer constellation points (Figure 15.2). The `linear` model maps the drive directly onto the field. The laser power is split equally between the polarizations.

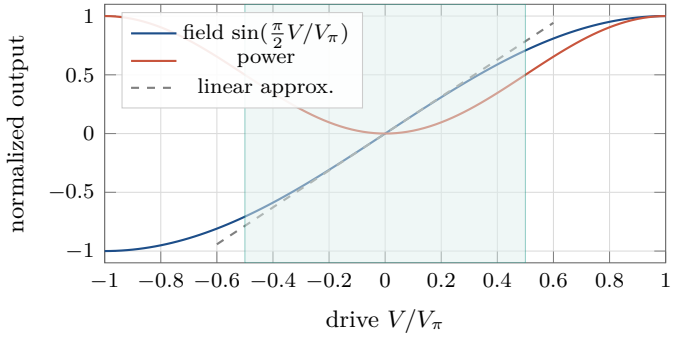


Figure 15.2: MZM transfer function at null bias. The shaded region is the default drive swing ($0.5\,V_\pi$).

Code 15.2: Sinusoidal MZM transfer and power normalization (`iq_modulator`).
`linkstudio/phy/optical.py`, lines 133–139

```
if p.model == "mzm":
    peak = max(np.max(np.abs(d.real)), np.max(np.abs(d.imag))) or 1.0
    arg = np.pi / 2 * p.drive_swing
    d = (np.sin(arg * d.real / peak) + 1j * np.sin(arg * d.imag / peak))
rms = np.sqrt(np.mean(np.abs(d) ** 2, axis=_pol_axes(d), keepdims=True))
p_tot = dbm_to_w(p.power_dbm)
field = d / np.maximum(rms, 1e-30) * np.sqrt(p_tot / npol)
```

15.2 Propagation in single-mode fiber

15.2.1 The nonlinear Schrödinger equation

The slowly varying envelope $A(z,t)$ of a single-mode fiber, in a frame moving with the group velocity, obeys

$$\frac{\partial A}{\partial z} = \underbrace{-\frac{\alpha}{2}A - j\frac{\beta_2}{2}\frac{\partial^2 A}{\partial t^2}}_{\hat{D}A} + \underbrace{j\gamma|A|^2 A}_{\hat{N}A}, \tag{15.4}$$

with three ingredients [2]:

- the power attenuation α, given in dB/km as `attenuation_db_km`;
- the group-velocity dispersion β_2, related to the dispersion parameter D (ps/nm/km) by

$$\beta_2 = -\frac{D\lambda^2}{2\pi c} \qquad (\beta_2 \approx -21.7\,\mathrm{ps}^2/\mathrm{km} \text{ for } D = 17,\ \lambda = 1550\,\mathrm{nm}); \tag{15.5}$$

- the Kerr coefficient γ, given in 1/W/km as `gamma_w_km`.

With the Fourier convention of Equation (1), $\partial_t \to j\omega$, so the linear part has the exact solution

$$\tilde{A}(z,\omega) = \tilde{A}(0,\omega)\exp\Big(j\frac{\beta_2}{2}\omega^2 z - \frac{\alpha}{2}z\Big), \tag{15.6}$$

an all-pass filter with quadratic phase. The dispersion therefore depends only on the accumulated $D\ell$ (ps/nm), the quantity tracked in `cd_ps_nm`.

Code 15.3: From D to β_2. `linkstudio/phy/optical.py`, lines 148–152

```
148 def beta2(dispersion_ps_nm_km: float, wavelength_nm: float) -> float:
149     """Group-velocity dispersion beta2 in s^2/m."""
150     D = dispersion_ps_nm_km * 1e-6  # s/m^2
151     lam = wavelength_nm * 1e-9
152     return -D * lam ** 2 / (2 * np.pi * C_LIGHT)
```

Code 15.4: The dispersion operator of Equation (15.6). `linkstudio/phy/optical.py`, lines 155–159

```
155 def dispersion_operator(n: int, fs: float, cd_ps_nm: float, wavelength_nm: float) -> np.
        ndarray:
156     """exp(j beta2 L / 2 w^2) for an accumulated dispersion D*L in ps/nm."""
157     w = 2 * np.pi * np.fft.fftfreq(n, 1 / fs)
158     b2l = beta2(cd_ps_nm, wavelength_nm) * 1e3  # D*L in ps/nm == (ps/nm/km) * km
159     return np.exp(1j * b2l / 2 * w ** 2)
```

Dispersion alone. An unchirped Gaussian pulse $A(0,t) = e^{-t^2/(2T_0^2)}$ stays Gaussian and broadens as

$$\frac{T_1(z)}{T_0} = \sqrt{1 + \Big(\frac{z}{L_D}\Big)^2}, \qquad L_D = \frac{T_0^2}{|\beta_2|}. \tag{15.7}$$

For $T_0 = 5\,\mathrm{ps}$ and $|\beta_2| = 21.68\,\mathrm{ps}^2/\mathrm{km}$, $L_D = 1.153\,\mathrm{km}$. Figure 15.3 compares the `propagate` solver with this result. The simulated RMS widths agree with the formula to all printed digits, as expected from an exact spectral method.

Kerr nonlinearity alone. Without dispersion Equation (15.4) integrates exactly. The power profile is unchanged and the phase grows in proportion to it, $A(L,t) = A(0,t)\,e^{-\alpha L/2}e^{j\gamma|A(0,t)|^2 L_{\mathrm{eff}}}$ with $L_{\mathrm{eff}} = (1 - e^{-\alpha L})/\alpha$. This self-phase modulation (SPM) chirps the pulse and broadens its spectrum, with a number of spectral peaks that grows with the peak phase $\phi_{\max} = \gamma P_0 L_{\mathrm{eff}}$ (Figure 15.4).

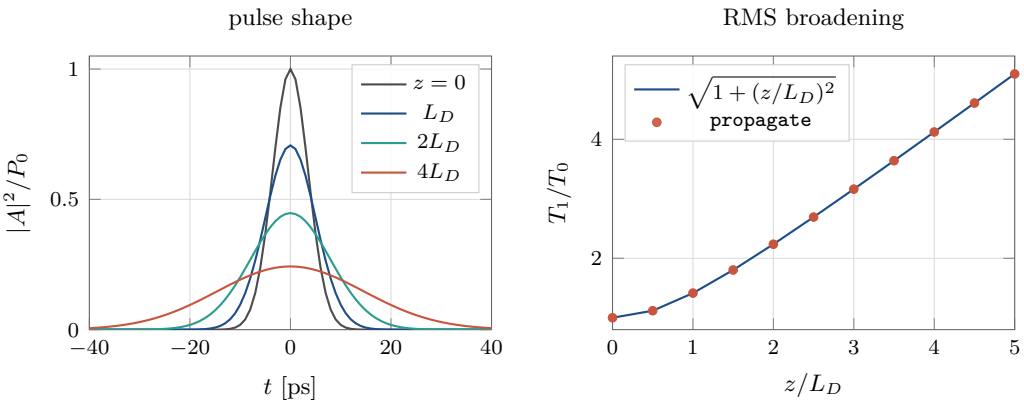


Figure 15.3: Chromatic dispersion of a 5-ps Gaussian pulse in standard single-mode fiber ($D = 17\,\mathrm{ps/nm/km}$, no loss).

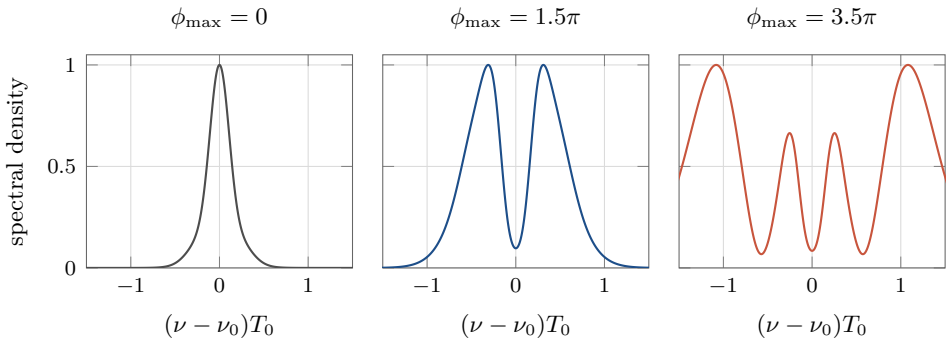


Figure 15.4: SPM-broadened spectra of a Gaussian pulse computed by `propagate` with $D = 0$, the classic figure of Agrawal [2]: $\phi_{\max} = (M - \frac{1}{2})\pi$ produces M peaks.

15.2.2 The split-step Fourier method

With both terms present, Equation (15.4) has no general closed form. Formally $A(z+h) = e^{h(\hat{D}+\hat{N})}A(z)$. The two operators do not commute, but each is trivial to apply on its own: $\hat{D}$ is a multiplication in the frequency domain, and $\hat{N}$ a multiplication in the time domain. The symmetric (Strang) splitting

$$A(z+h) = e^{\frac{h}{2}\hat{D}}\, e^{h\hat{N}}\, e^{\frac{h}{2}\hat{D}} A(z) + \mathcal{O}(h^3) \tag{15.8}$$

is exact to second order, by the Baker–Campbell–Hausdorff formula: the first commutator $[\hat{D}, \hat{N}]$ cancels by symmetry. Over a fixed length the L/h local errors add up to a global error $\mathcal{O}(h^2)$.

Attenuation is included in the half steps. The nonlinear phase is applied at mid-step to the field whose power has been reduced by the first half step. The effective length that makes this exact for a constant envelope is

$$L_{\text{eff}}(h) = \int_{-h/2}^{h/2} e^{-\alpha z'}\, dz' = \frac{2\sinh(\alpha h/2)}{\alpha}. \tag{15.9}$$

For dual polarization the Manakov equation replaces $|A|^2$ by $|A_x|^2 + |A_y|^2$ and γ by $\frac{8}{9}\gamma$, the average over fast random polarization evolution.

Code 15.5: The split-step Fourier solver of the NLSE / Manakov equation (`propagate`).
linkstudio/phy/optical.py, lines 168–187

```python
def propagate(e: np.ndarray, fs: float, length_km: float, att_db_km: float, d_ps_nm_km: float,
              wavelength_nm: float, nonlinear: bool, gamma: float, steps_per_km: float) -> np.ndarray:
    """Split-step Fourier solution of the NLSE (Manakov equation for dual polarization)."""
    n = e.shape[-1]
    alpha = att_db_km / (10 * np.log10(np.e)) / 1e3  # 1/m
    if not nonlinear or gamma == 0:
        lin = dispersion_operator(n, fs, d_ps_nm_km * length_km, wavelength_nm) * np.exp(-alpha * length_km * 1e3 / 2)
        return np.fft.ifft(np.fft.fft(e, axis=-1) * lin, axis=-1)
    steps = max(1, int(np.ceil(length_km * steps_per_km)))
    dz = length_km * 1e3 / steps
    half = dispersion_operator(n, fs, d_ps_nm_km * length_km / steps / 2, wavelength_nm) * np.exp(-alpha * dz / 4)
    # Effective length around the step midpoint (where the nonlinear phase is applied).
    leff = 2 * np.sinh(alpha * dz / 2) / alpha if alpha > 0 else dz
    g = gamma * 1e-3 * (8 / 9 if e.ndim == 3 else 1.0)  # 1/(W km) -> 1/(W m)
    for _ in range(steps):
        e = np.fft.ifft(np.fft.fft(e, axis=-1) * half, axis=-1)
        p = np.sum(np.abs(e) ** 2, axis=1, keepdims=True) if e.ndim == 3 else np.abs(e) ** 2
        e = e * np.exp(1j * g * p * leff)
        e = np.fft.ifft(np.fft.fft(e, axis=-1) * half, axis=-1)
    return e
```

Choosing the step The default `steps_per_km=0.5` (2-km steps) is adequate for typical launch powers around 0 dBm per channel, where the nonlinear length $1/(\gamma P)$ is hundreds of km. The soliton of Figure 15.5 carries 28 dBm, a nonlinear length of about 1 km. There the default step leaves a 12 % error, and 16 steps per km are needed for 10^{-3}. For high powers, dense WDM or solitons, reduce the step until the result stops changing. Without `nonlinear` the solver applies one exact FFT for the whole span.

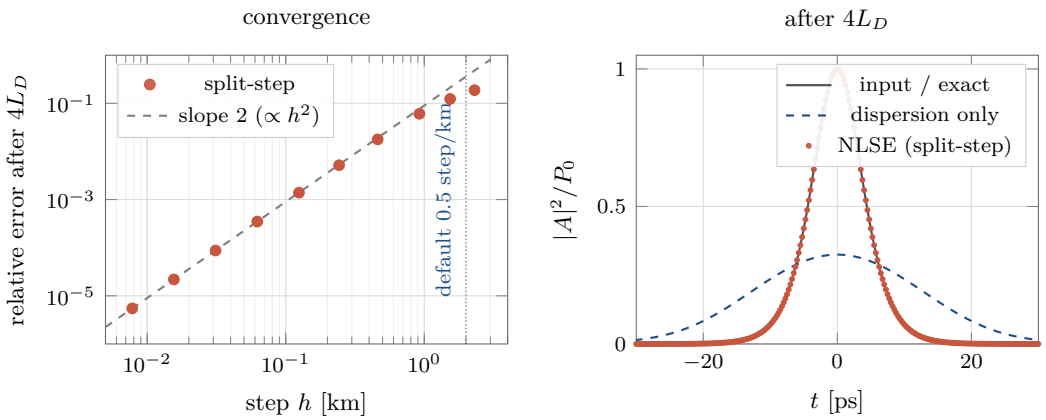


Figure 15.5: Validation of the solver on the fundamental soliton, an exact solution of Equation (15.4) with $\alpha = 0$. With $T_0 = 5\,\mathrm{ps}$, $P_0 = |\beta_2|/(\gamma T_0^2) = 0.667\,\mathrm{W}$ and $L = 4.61\,\mathrm{km} = 4L_D$, the pulse must return unchanged. Left: the relative L^2 error falls as h^2, confirming the order of Equation (15.8). Right: dispersion alone would broaden the pulse, while the balance with SPM preserves it.

15.2.3 Polarization-mode dispersion

Random birefringence rotates the state of polarization and delays the two principal states by the differential group delay (DGD). The first-order model is

$$\tilde{\boldsymbol{A}}_{\mathrm{out}}(\omega) = \mathbf{U}_2 \begin{pmatrix} e^{j\omega\tau/2} & 0 \\ 0 & e^{-j\omega\tau/2} \end{pmatrix} \mathbf{U}_1 \tilde{\boldsymbol{A}}_{\mathrm{in}}(\omega), \qquad \tau = D_{\mathrm{PMD}}\sqrt{\ell}, \tag{15.10}$$

with $\mathbf{U}_{1,2}$ Haar-random unitary matrices. These are generated by a QR decomposition of a complex Gaussian matrix with the phases of the diagonal of $\mathbf{R}$ removed.

Code 15.6: Random polarization rotation and first-order PMD. `linkstudio/phy/optical.py`, lines 190–201

```
190 def apply_pmd(rng: np.random.Generator, e: np.ndarray, fs: float, dgd_s: float) -> np.
        ndarray:
191     """Random polarization rotation plus first-order PMD (differential group delay)."""
192     B, _, n = e.shape
193     u1, u2 = _random_unitary(rng, B), _random_unitary(rng, B)
194     e = np.einsum("bij,bjn->bin", u1, e)
195     if dgd_s > 0:
196         w = 2 * np.pi * np.fft.fftfreq(n, 1 / fs)
197         E = np.fft.fft(e, axis=-1)
198         E[:, 0] *= np.exp(1j * w * dgd_s / 2)
199         E[:, 1] *= np.exp(-1j * w * dgd_s / 2)
200         e = np.fft.ifft(E, axis=-1)
201     return np.einsum("bij,bjn->bin", u2, e)
```

Here τ is set to its mean value; a full statistical model would draw it from a Maxwellian distribution.

15.3 Optical amplification and OSNR

15.3.1 ASE noise

An erbium-doped fiber amplifier of gain G adds amplified spontaneous emission with power spectral density, per polarization,

$$S_{\mathrm{ASE}} = n_{\mathrm{sp}}\, h\nu\,(G-1), \qquad \mathrm{NF} = \frac{2n_{\mathrm{sp}}(G-1)}{G} \;\Rightarrow\; n_{\mathrm{sp}} = \frac{\mathrm{NF}\cdot G}{2(G-1)}, \tag{15.11}$$

where the noise figure NF is at least 2 (3 dB) for $n_{\mathrm{sp}} \geq 1$. In the simulation band f_s, the ASE is circular Gaussian noise of variance $S_{\mathrm{ASE}} f_s$ per polarization.

Code 15.7: ASE density from gain and noise figure. linkstudio/phy/optical.py, lines 239–245

```
def ase_psd(gain_lin: float, nf_db: float, wavelength_nm: float) -> float:
    """ASE power spectral density per polarization (W/Hz) of an amplifier."""
    if gain_lin <= 1:
        return 0.0
    nu = C_LIGHT / (wavelength_nm * 1e-9)
    nsp = max(10 ** (nf_db / 10) * gain_lin / (2 * (gain_lin - 1)), 0.5)
    return nsp * H_PLANCK * nu * (gain_lin - 1)
```

Code 15.8: ASE noise in the simulated bandwidth. linkstudio/phy/optical.py, lines 248–250

```
def _add_ase(rng: np.random.Generator, e: np.ndarray, psd: float, fs: float) -> np.ndarray:
    sigma = np.sqrt(psd * fs / 2)
    return e + sigma * (rng.standard_normal(e.shape) + 1j * rng.standard_normal(e.shape))
```

15.3.2 Optical signal-to-noise ratio

The OSNR is measured in a reference bandwidth $B_{\mathrm{ref}} = 12.5\,\mathrm{GHz}$ (0.1 nm at 1550 nm), counting the noise in both polarizations:

$$\mathrm{OSNR} = \frac{P}{2S_{\mathrm{ASE}}B_{\mathrm{ref}}}. \tag{15.12}$$

In a chain of N identical spans, each followed by an amplifier compensating its loss ($G = e^{\alpha L_{\mathrm{span}}}$), the ASE adds up while the signal power is restored. With the NF definition above, $2S_{\mathrm{ASE}} \approx \mathrm{NF}\, G\, h\nu$, so

$$\mathrm{OSNR}_{\mathrm{dB}} \approx \underbrace{-10\log_{10}\frac{h\nu B_{\mathrm{ref}}}{1\,\mathrm{mW}}}_{58.0\ \mathrm{dB}} + P_{\mathrm{dBm}} - G_{\mathrm{dB}} - \mathrm{NF}_{\mathrm{dB}} - 10\log_{10} N, \tag{15.13}$$

the "58 dB rule" of link design.

Code 15.9: The multi-span link: propagate, amplify, accumulate the ASE density. linkstudio/phy/optical.py, lines 319–327

```
g = 10 ** (p.attenuation_db_km * p.span_km / 10)
s = ase_psd(g, p.noise_figure_db, lam)
for _ in range(p.n_spans):
    e = propagate(e, fs, p.span_km, p.attenuation_db_km, p.dispersion_ps_nm_km, lam,
                  p.nonlinear, p.gamma_w_km, p.steps_per_km)
    if e.ndim == 3 and (p.polarization_rotation or p.pmd_ps_sqrt_km):
        e = apply_pmd(self.rng, e, fs, p.pmd_ps_sqrt_km * 1e-12 * np.sqrt(p.span_km))
    e = _add_ase(self.rng, e * np.sqrt(g), s, fs)
    ase = ase + s  # the span loss and the amplifier gain cancel
```

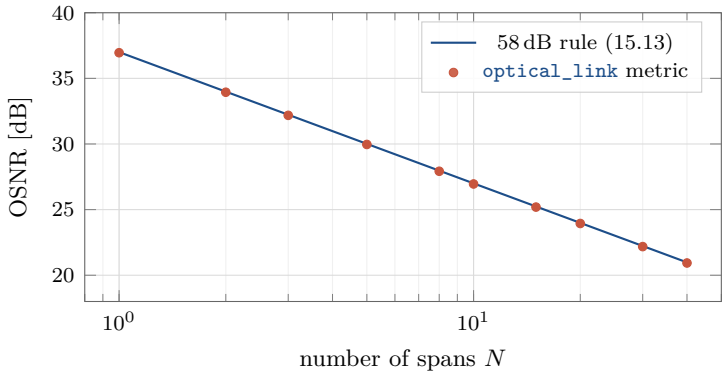


Figure 15.6: OSNR after $N \times 80\,\mathrm{km}$ (0.2 dB/km, NF = 5 dB, 0 dBm per channel). The block's metric and the rule differ by less than 0.05 dB, the effect of the $G - 1 \approx G$ approximation and of the exact photon energy.

15.3.3 From OSNR to E_s/N_0

For a coherent receiver whose noise is the ASE, the noise density per polarization is $N_0 = S_{\mathrm{ASE}}$ and the signal power per polarization is P/p for p polarizations. Hence

$$\frac{E_s}{N_0} = \frac{P/p}{S_{\mathrm{ASE}} R_s} = \mathrm{OSNR} \cdot \frac{2B_{\mathrm{ref}}}{p\,R_s}, \tag{15.14}$$

implemented as `osnr_to_esn0_db`. The test-suite uses it to check that the full coherent DP-QPSK link (Section 15.7.2) performs within about 0.3 dB of theory.

15.4 Wavelength-division multiplexing

`wdm_multiplexer` surrounds the channel under test with $N_{\mathrm{ch}} - 1$ neighbours at multiples of the channel spacing. The neighbours carry independent symbols with the same constellation, pulse shape and power. Propagating the composite field through a nonlinear fiber produces cross-phase modulation (the power of the neighbours modulates the phase of the channel under test) and four-wave mixing. `wdm_demultiplexer` selects a channel with a second-order super-Gaussian optical filter, $|H(f)| = \exp(-\frac{\ln 2}{2}(2f/B)^4)$, which is 3 dB down at $\pm B/2$. The sample rate must cover all channels. The *WDM with fiber nonlinearity* scenario shows the MER of the centre channel dropping by several dB when the neighbours are switched on.

Code 15.10: Channel selection by frequency shift and optical filtering. `linkstudio/phy/optical.py`, lines 410–416

```
410 shift = np.exp(-2j * np.pi * self.params.channel * spacing * np.arange(n) / fs)
411 f = np.fft.fftfreq(n, 1 / fs)
412 if self.params.shape == "rect":
413     H = (np.abs(f) <= bw / 2).astype(float)
414 else:
415     H = np.exp(-np.log(2) / 2 * (2 * f / bw) ** 4)  # -3 dB at +-bw/2
416 y = np.fft.ifft(np.fft.fft(x.data * shift, axis=-1) * H, axis=-1)
```

15.5 Free-space optics

Atmospheric turbulence makes the received intensity fluctuate (scintillation). `fso_turbulence` multiplies the field by $\sqrt{h}$, where the normalized irradiance h ($\mathbb{E}h = 1$) is constant over a frame and follows one of two models.

- **Log-normal** (weak turbulence): $\ln h \sim \mathcal{N}(-\sigma^2/2, \sigma^2)$.
- **Gamma-gamma** (moderate to strong turbulence): h is the product of two independent unit-mean gamma variables with parameters α (large eddies) and β (small eddies). Its density is

$$f(h) = \frac{2(\alpha\beta)^{\frac{\alpha+\beta}{2}}}{\Gamma(\alpha)\Gamma(\beta)}\, h^{\frac{\alpha+\beta}{2}-1} K_{\alpha-\beta}\!\left(2\sqrt{\alpha\beta h}\right). \tag{15.15}$$

Code 15.11: Scintillation models. linkstudio/phy/optical.py, lines 441–447

```
if p.model == "lognormal":
    s2 = p.log_intensity_variance
    h = np.exp(self.rng.normal(-s2 / 2, np.sqrt(s2), B))
else:
    h = self.rng.gamma(p.alpha, 1 / p.alpha, B) * self.rng.gamma(p.beta, 1 / p.beta, B)
shape = (B,) + (1,) * (x.data.ndim - 1)
gain = np.sqrt(h).reshape(shape)
```

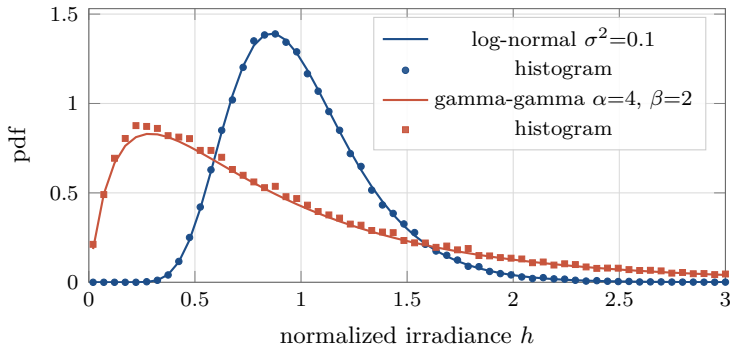


Figure 15.7: Histograms of h from `fso_turbulence` (60 000 frames) against the closed-form densities.

15.6 Direct detection

15.6.1 PIN and APD receivers

A photodiode converts power into current, $I = \mathcal{R}P$, with responsivity $\mathcal{R}$ (A/W). Over the noise bandwidth $B = f_s/2$ the current carries three contributions:

- **shot noise**, $\sigma^2_{\text{sh}} = 2qM^2F\,(\mathcal{R}P + I_d)B$;
- **thermal noise**, $\sigma^2_{\text{th}} = i_n^2 B$, with i_n the input-referred current density (`thermal_noise_pa_rthz`);
- **avalanche gain** (APD): the multiplication M comes with McIntyre's excess noise factor $F = kM + (1-k)(2 - 1/M)$, with k the ionization ratio.

The APD gain raises the signal above thermal noise but amplifies shot noise by M^2F, so there is an optimum M.

Code 15.12: Square-law detection with shot, thermal and APD excess noise. linkstudio/phy/optical.py, lines 480–486

```
M = p.apd_gain
F = p.apd_k * M + (1 - p.apd_k) * (2 - 1 / M)
i_primary = p.responsivity * power + p.dark_current_na * 1e-9
var = (p.thermal_noise_pa_rthz * 1e-12) ** 2 * bw * np.ones_like(power)
if p.shot_noise:
```

```
485      var = var + 2 * Q_E * M ** 2 * F * i_primary * bw
486  current = M * i_primary + np.sqrt(var) * self.rng.standard_normal(power.shape)
```

15.6.2 OOK sensitivity and the Q factor

For on–off keying with levels $I_{1,0} = \mathcal{R}P_{1,0}$ and Gaussian noise of standard deviations $\sigma_{1,0}$, the optimum threshold gives

$$\mathrm{BER} = \frac{1}{2}\operatorname{erfc}\left(\frac{Q}{\sqrt{2}}\right), \qquad Q = \frac{I_1 - I_0}{\sigma_1 + \sigma_0}, \qquad P_{1,0} = \frac{2P_{\mathrm{avg}}}{1 + \mathrm{ER}^{\mp 1}}. \tag{15.16}$$

Thermal noise is signal independent, so $\sigma_1 = \sigma_0$ and $Q \propto P_{\mathrm{avg}}$: the BER falls by many decades within a few dB (Figure 15.8). The receiver sensitivity is conventionally quoted at 10^{-9} ($Q = 6$) or at a FEC threshold.

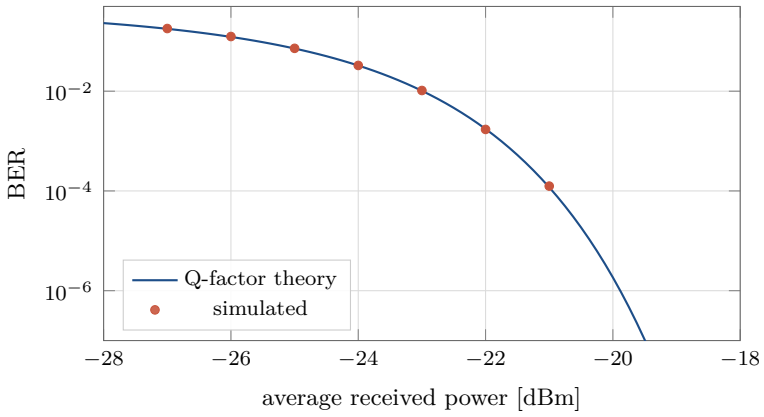


Figure 15.8: 10 Gb/s NRZ-OOK into a thermal-noise-limited PIN receiver ($\mathcal{R} = 0.8$ A/W, 20 pA/$\sqrt{\mathrm{Hz}}$, ER = 10 dB): chain `intensity_modulator`→`photodiode`→`agc`→`line_decoder` against the Q-factor formula.

> **In the code** `agc` removes the mean (the DC photocurrent) and scales each frame to the nominal symbol power of the upstream line code or constellation. It subtracts the known noise variance when estimating the signal power, so the decision thresholds stay correct at low SNR.

15.7 Coherent detection

15.7.1 The coherent front end

A coherent receiver mixes the signal with a local oscillator (LO) in a 90° hybrid and detects the four outputs with two balanced photodiode pairs per polarization (Figure 15.9). Balanced detection cancels the direct-detection terms $|E|^2$ and $|E_{\mathrm{LO}}|^2$ and leaves the beat term:

$$i_I + j\, i_Q = \mathcal{R}\sqrt{P_{\mathrm{LO}}}\; E(t)\, e^{-j(2\pi \Delta f\, t + \phi_{\mathrm{LO}}(t))} + n(t). \tag{15.17}$$

The field is thus translated linearly to the electrical domain, including its phase and polarization. With a strong LO, the LO shot noise $2q\mathcal{R}P_{\mathrm{LO}}B$ dominates the thermal noise.

> **In the code** The simulator processes each frame with circular FFTs (fiber, dispersion compensation). An LO phase that does not return to its initial value at the end of the

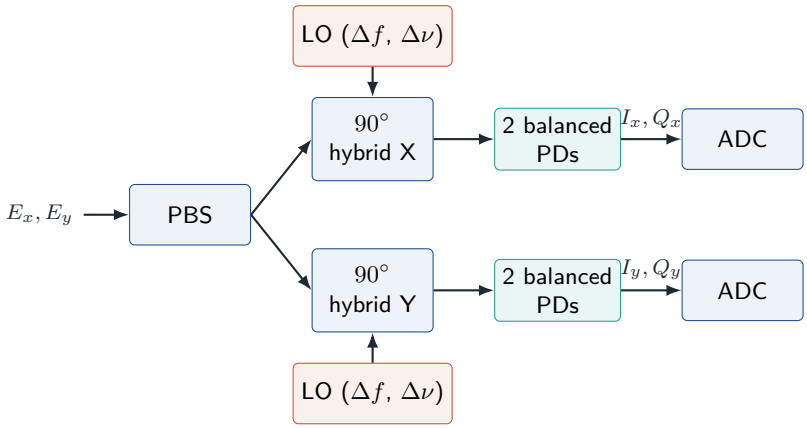


Figure 15.9: Polarization-diverse coherent front end, modelled by `coherent_receiver`. One LO laser is split between the two hybrids.

frame would create a discontinuity at the frame boundary, spread over the frame by the dispersion compensator. The block therefore makes the LO phase periodic:

- the frequency offset is rounded to the frame's frequency grid f_s/n;
- the Wiener phase is turned into a Brownian bridge, $\phi_k - \frac{k}{n}\phi_n$, which has the same local increments but returns to zero.

Code 15.13: Coherent detection with a periodic LO phase (`coherent_receiver`). `linkstudio/phy/optical.py`, lines 518–530

```
518 walk = _wiener_phase(self.rng, B, n + 1, p.lo_linewidth_hz, fs)
519 bridge = walk[:, :n] - np.arange(n) / n * walk[:, n:n + 1]
520 fo = np.round(p.frequency_offset_hz * n / fs) * fs / n
521 phase = bridge + 2 * np.pi * fo * np.arange(n) / fs
522 lo = np.exp(-1j * (phase[:, None, :] if e.ndim == 3 else phase))
523 p_lo = dbm_to_w(p.lo_power_dbm)
524 scale = p.responsivity * np.sqrt(p_lo)
525 bw = fs / 2
526 var = 2 * (p.thermal_noise_pa_rthz * 1e-12) ** 2 * bw
527 if p.shot_noise:
528     var += 2 * Q_E * p.responsivity * p_lo * bw
529 noise = np.sqrt(var / 2) * (self.rng.standard_normal(e.shape) + 1j * self.rng.
        standard_normal(e.shape))
530 out = scale * e * lo + noise
```

15.7.2 Coherent digital signal processing

The chain follows the architecture described by Savory [17]. Figure 15.10 follows a 128 Gb/s DP-QPSK signal through the DSP chain after 20×80 km, with random polarization, a 1 GHz LO offset and 100 kHz lasers.

Chromatic dispersion compensation. The accumulated dispersion is known from the metadata (or given), and its all-pass response Equation (15.6) is inverted exactly by multiplying with its conjugate in the frequency domain.

Code 15.14: Frequency-domain CD compensation. `linkstudio/phy/optical.py`, lines 594–595

```
594 H = np.conj(dispersion_operator(x.length, fs, cd, lam))
595 y = np.fft.ifft(np.fft.fft(x.data, axis=-1) * H, axis=-1)
```

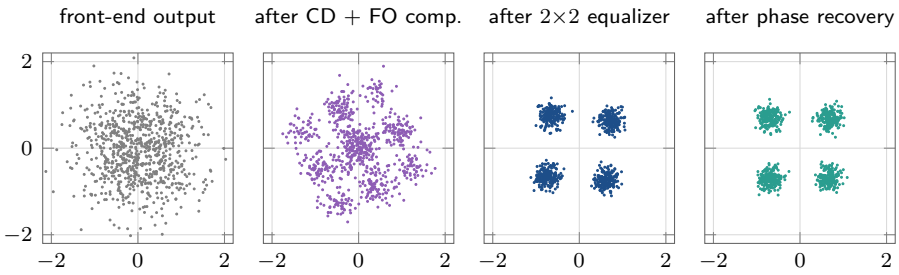


Figure 15.10: Polarization X of the *Coherent DP-QPSK long haul* scenario at four points of the DSP chain (normalized to unit power).

Frequency-offset estimation. Raising QPSK to the fourth power removes the modulation: $(s\,e^{j2\pi\Delta f t})^4 = s^4 e^{j2\pi 4\Delta f t}$ with $s^4 = -1$ for every QPSK symbol, and approximately constant on average for square QAM. The spectrum of r^4 therefore has a line at $4\Delta f$. The estimate is the position of its peak divided by 4, refined by zero-padding and computed on a low-pass filtered copy to reject noise. In dual polarization a rotated mixture $as_1 + bs_2$ has a fourth-power line proportional to $a^4 + b^4$, which can vanish. The estimator therefore also uses the combinations $(x_0 + x_1)/\sqrt{2}$ and $(x_0 + jx_1)/\sqrt{2}$.

Compensating the offset *after* the dispersion compensator leaves a residual: the CD filter was applied to a spectrum shifted by $\Delta\omega$, so the error is $\exp(j\frac{\beta_2 L}{2}(2\omega\Delta\omega + \Delta\omega^2))$, a bulk delay $\beta_2 L\Delta\omega$ plus a constant phase. The block removes it.

Code 15.15: Fourth-power FO estimation over several polarization combinations.
linkstudio/phy/optical.py, lines 633–640

```
633 nfft = int(2 ** np.ceil(np.log2(4 * n)))
634 spec = np.abs(np.fft.fft(lp ** m, n=nfft, axis=-1)) ** 2
635 if spec.ndim == 3:
636     spec = spec.sum(axis=1)
637 f = np.fft.fftfreq(nfft, 1 / fs)
638 df = f[np.argmax(spec, axis=-1)] / m  # (B,)
639 self._estimates.extend(df.tolist())
640 rot = np.exp(-2j * np.pi * df[:, None] * np.arange(n) / fs)
```

Adaptive 2×2 equalization. Polarization rotation, PMD and residual dispersion form a 2×2 MIMO channel, inverted by a fractionally spaced butterfly of four FIR filters (Figure 15.11):

$$y_p[k] = \sum_{q\in\{x,y\}} \boldsymbol{w}_{pq}^{\mathsf{T}}\,\boldsymbol{u}_q[k], \qquad \boldsymbol{w}_{pq} \leftarrow \boldsymbol{w}_{pq} + \mu\, e_p[k]\,\boldsymbol{u}_q^{*}[k]. \tag{15.18}$$

The error e_p is either:

- the constant-modulus error $y_p(R - |y_p|^2)$ with $R = \mathbb{E}|s|^4/\mathbb{E}|s|^2$ (Godard's CMA [15]), which is blind and insensitive to the carrier phase; or
- the LMS error $d_p - y_p$, trained on known symbols and then decision-directed, with a larger step during training.

Code 15.16: Butterfly update with CMA or trained/decision-directed LMS.
linkstudio/phy/optical.py, lines 704–718

```
704 for k in range(n_sym):
705     c = start + k * sps + D  # centre index in the padded signal
706     u = pad[:, :, c - D:c + D + 1][:, :, ::-1]  # (B, P, T)
707     y = np.einsum("bpqt,bqt->bp", W, u)
708     out[:, :, k] = y
```

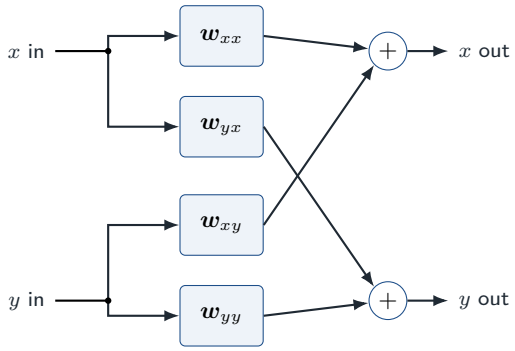


Figure 15.11: The butterfly equalizer (2 samples per symbol, `n_taps` taps per filter).

```
    if not use_lms:  # blind: phase-insensitive, leaves the carrier phase to phase_recovery
        e = y * (radius - np.abs(y) ** 2)
    elif k < p.n_train:
        e = refd[:, :, k] - y
    else:
        e = const.points[const.nearest(y)] - y if const is not None else refd[:, :, k] - y
        err += float(np.sum(np.abs(e) ** 2))
        count += e.size
    mu = p.train_step_size if use_lms and k < p.n_train else p.step_size
    W += mu * e[:, :, None, None] * np.conj(u)[:, None, :, :]
```

Carrier phase recovery. Laser phase noise of $2 \times 100\,\mathrm{kHz}$ at $32\,\mathrm{GBd}$ is slow, but large enough to need feed-forward tracking. Two estimators are available.

- The Viterbi–Viterbi estimator removes the QPSK modulation with the fourth power: $\hat{\phi} = \frac{1}{4}\big(\arg \sum_{|i|\le W} y_{k+i}^4 - \pi\big)$.
- Blind phase search (BPS) [16] works for any square QAM. It rotates the symbols by B test phases in $[-\pi/4, \pi/4)$, sums over a window the squared distances to the nearest constellation points, and keeps the phase with the smallest sum.

Both estimates are defined modulo $\pi/2$; they are unwrapped (on $4\hat{\phi}$) to follow the phase continuously. The remaining quadrant ambiguity is resolved once per frame with a short block of known symbols, taken after the equalizer has converged.

Code 15.17: Viterbi–Viterbi and blind phase search. `linkstudio/phy/optical.py`, lines 758–771

```
if p.method == "viterbi_viterbi":
    s = smooth(y ** 4)
    phi = (np.angle(s) - np.pi) / 4
else:
    if const is None:
        raise BlockError("BPS needs the constellation (mapper upstream).")
    phases = np.arange(p.n_test_phases) / p.n_test_phases * np.pi / 2 - np.pi / 4
    dist = np.empty((*y.shape, phases.size))
    for i, ph in enumerate(phases):
        r = y * np.exp(-1j * ph)
        dist[..., i] = np.abs(r - const.points[const.nearest(r)]) ** 2
    dist = np.moveaxis(smooth(np.moveaxis(dist, -1, 0)), 0, -1)
    phi = phases[np.argmin(dist, axis=-1)]
phi = np.unwrap(4 * phi, axis=-1) / 4
```

Chapter 16

Validation

A simulator is only useful if its numbers can be trusted. LinkStudio is validated at three levels:

- unit tests of the engine and the DSP helpers;
- *physics tests*, which compare complete chains with closed-form results;
- a test that loads and runs every shipped scenario.

The demo scripts repeat the physics checks in readable form, and this manual's figures are a third, visual check of the same models.

16.1 Running the checks

```
cd backend
python -m pytest                 # all tests (about 170, a few minutes)
python -m pytest tests/test_physics.py -q
python scripts/main.py           # demos: simulated vs theory, PASS/FAIL per check
python scripts/main.py --quick   # fewer errors per point, faster
```

Physics tests stop after 300–500 errors and accept a relative deviation of 20–25 %. By Section 5.3 this is several standard deviations, so false alarms are negligible, while a systematic error of 1 dB (a factor of 2–3 in BER on steep curves) is caught.

16.2 Physics checks

Model	Reference	Checked in
AWGN, PSK/QAM/PAM	exact Gray BER (Section 6.3)	`test_physics.py`, Figure 6.2
Soft demapping	max-log decisions = hard decisions	`test_physics.py`
Pulse shaping	RRC link BER = symbol-rate theory; noise metadata after MF	`test_physics.py`
OFDM over AWGN	per-subcarrier E_s/N_0 theory	`test_physics.py`
OFDM multipath	LS and perfect CSI equalization	`test_physics.py`, Figure 11.3
Rayleigh + ZF	Equation (9.5)	`test_physics.py`, Figure 12.2
ISI equalizers	ZF/MMSE/LMS remove ISI	`test_physics.py`
Soft vs hard Viterbi	soft better by ≈ 2 dB	`test_physics.py`
Synchronization	receiver close to theory with delay and CFO; fails without sync	`test_advanced.py`
Carrier recovery	tracks phase noise	`test_advanced.py`
Timing recovery	estimates the fractional delay	`test_advanced.py`
LDPC	codewords satisfy $\mathbf{H}\boldsymbol{c}^{\mathsf{T}} = 0$; corrects noise; coding gain	`test_advanced.py`
Polar	noiseless decoding; frozen set; coding gain	`test_advanced.py`
Puncturing	rate 3/4; decodable	`test_advanced.py`
SIMO MRC	Equation (12.3), $L = 2$	`test_advanced.py`, Figure 12.2

Model	Reference	Checked in
Spatial multiplexing	ZF 2×2 = SISO Rayleigh; ML < MMSE < ZF	`test_advanced.py`, Figure 12.1
Alamouti	MRC $L = 2$ shifted by 3 dB	`test_advanced.py`, Figure 12.2
Time-varying multipath	OFDM with Jakes taps	`test_advanced.py`
Line codes	round trips; 4B/5B and 64B/66B rates; error multiplication	`test_wireline_optical.py`
Real NRZ over AWGN	BPSK theory	`test_wireline_optical.py`
Cable	insertion loss equals the preset formula	`test_wireline_optical.py`, Figure 14.3
SerDes	CTLE peaking; CTLE + DFE open the eye	`test_wireline_optical.py`, Figure 14.4
DMT	water-filling bit loading	`test_wireline_optical.py`, Figure 14.5
Crosstalk, echo	coupling level; canceller reduces echo	`test_wireline_optical.py`
Jitter	zero jitter is identity; RJ degrades	`test_wireline_optical.py`
Fiber loss, metadata	$P\,e^{-\alpha L}$, $D\ell$	`test_wireline_optical.py`
Dispersion	Gaussian broadening; exact compensation	`test_wireline_optical.py`, Figure 15.3
SPM	CW phase $\gamma P L_{\text{eff}}$	`test_wireline_optical.py`, Figure 15.4
Split-step solver	second-order convergence on the soliton	Figure 15.5
EDFA	OSNR formula	`test_wireline_optical.py`, Figure 15.6
IM/DD OOK	Q-factor BER	`test_wireline_optical.py`, Figure 15.8
IM/DD PAM-4	dispersion-induced power fading	`test_wireline_optical.py`
Coherent DP-QPSK	BER vs OSNR theory within $\approx$ 0.3 dB; FO estimate	`test_wireline_optical.py`
FSO	mean and variance of the scintillation	`test_wireline_optical.py`, Figure 15.7
WDM	XPM lowers the MER of the centre channel	`test_wireline_optical.py`
CP-based OFDM sync	recovers delay and CFO	`test_wireline_optical.py`
Confidence interval	Wilson bounds contain the BER	`test_wireline_optical.py`
All scenarios	load, validate, run; BER below a threshold	`test_scenarios.py`

16.3 Known limitations

Every model in this manual states its assumptions; the most important simplifications are collected here.

- Lasers are behavioural (no rate equations); PMD is first order with the mean DGD.
- WDM simulates the full multi-channel field but demodulates only one channel.

- The OFDM CP synchronizer estimates the fractional CFO only.
- DMT adapts bits to a flat transmit PSD (no power water-filling).
- MIMO channels are flat; frequency-selective MIMO-OFDM is not yet a block.
- Subsystems are edited in their YAML files, not in the GUI.
- Error counts assume independent errors; for coded and block-fading links, also judge the accuracy by the number of frame errors.

Part III

Reference

Chapter A

Block reference

This appendix lists the 86 registered blocks, grouped by category. It is generated by `manual/tools/generate.py` from `block_catalogue()`, the same source as the GUI palette and `python -m linkstudio describe`. Each entry gives:

- the first paragraph of the block documentation;
- the input and output ports with their accepted kinds;
- the parameters with their defaults. *auto* means the value is taken from upstream metadata or measured; *required* means the parameter has no default.

A.1 SOURCE

prbs Pseudo-random binary sequence (PRBS-7/9/11/15/20/23/31), continuous across frames.

in: - out: out:BITS

order=9; n_bits=1024

random_bits Independent Bernoulli bits, a new random frame for every batch row.

in: - out: out:BITS

n_bits=1024; p_one=0.5

A.2 MODULATION

demapper Hard or soft (LLR) demapping.

in: in:SYMBOLS out: out:BITS|LLR

method=maxlog; scheme=auto; order=4; phase_offset_deg=auto; points=auto; noise_var=auto

mapper Map groups of bits onto Gray-labelled constellation points (unit average energy).

in: in:BITS out: out:SYMBOLS

scheme=qam; order=4; phase_offset_deg=auto; points=auto; symbol_rate=1e+06

A.3 WAVEFORM

downsampler Keep one sample out of `factor` starting at `offset`.

in: in:SYMBOLS|SAMPLES out: out:SYMBOLS|SAMPLES

factor=2; offset=0; output_kind=samples

fir_filter Generic FIR filter with user-supplied taps (causal, output length preserved).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

taps=[1.0]

matched_filter Receive filter + symbol-rate sampling.

in: in:SAMPLES out: out:SYMBOLS|SAMPLES

shape=matched; sps=auto; span=10; rolloff=0.25; bt=0.3; normalize=True; decimate=True

ofdm_demodulator Remove the cyclic prefix and FFT every OFDM symbol into a resource grid.

in: in:SAMPLES out: out:GRID

n_fft=auto; cp_len=16; n_active=52; pilot_spacing=6; fft_offset=0

ofdm_modulator Map data symbols and pilots on subcarriers, IFFT and add the cyclic prefix.

in: in:SYMBOLS out: out:SAMPLES

n_fft=64; cp_len=16; n_active=52; pilot_spacing=6; subcarrier_spacing=15000; tail_zeros=0

pulse_shaper Upsample symbols by sps and filter them with a unit-energy pulse.

in: in:SYMBOLS out: out:SAMPLES

shape=rrc; sps=8; span=10; rolloff=0.25; bt=0.3

upsampler Insert factor - 1 zeros between samples.

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

factor=2

A.4 CHANNEL

awgn Add circularly-symmetric complex Gaussian noise.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

snr_db=10; mode=ebn0; power_reference=nominal; signal_power=auto; bits_per_symbol=auto; code_rate=auto

carrier_offset Carrier frequency offset and phase rotation: y = x * exp(j(2 pi cfo t + phi)).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

cfo_hz=0; phase_deg=0; random_phase=False

flat_fading Frequency-flat Rayleigh or Rician fading, E|h|^2 = 1.

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

model=rayleigh; k_factor_db=0; variation=block; doppler_hz=100; n_sinusoids=16

gain Scale the signal by gain_db (e.g. path loss). Noise metadata is scaled too.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

gain_db=0

iq_imbalance Receiver IQ imbalance and DC offset: y = K1 x + K2 conj(x) + dc.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

amplitude_db=0; phase_deg=0; dc_i=0; dc_q=0

mimo_channel Flat Rayleigh MIMO channel y = H x, H of size Nr x Nt, normalized by 1/sqrt(Nt).

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

n_rx=2; variation=block; tx_correlation=0; rx_correlation=0

multipath Tapped-delay-line channel (frequency selective).

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

preset=auto; delays=[0.0, 1.0, 2.0]; powers_db=[0.0, -3.0, -6.0]; delay_unit=samples; fading=True; variation=block; doppler_hz=50; normalize=True

phase_noise Oscillator phase noise modelled as a Wiener process with 3-dB linewidth_hz.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

linewidth_hz=100

power_amplifier Memoryless PA nonlinearity (Rapp, Saleh or ideal clipper).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

model=rapp; input_backoff_db=6; smoothness=2; saleh_beta_a=1.1517; saleh_alpha_p=4.0033; saleh_beta_p=9.104; normalize_output=True

quantizer Uniform mid-rise ADC on I and Q with clipping.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

bits=8; full_scale=auto; loading_factor=4

timing_offset Delay the signal by a (fractional) number of samples, keeping its length.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

delay_samples=0

A.5 SYNC

carrier_recovery Decision-directed second-order PLL tracking residual phase and frequency offsets.

in: in:SYMBOLS out: out:SYMBOLS

loop_bandwidth=0.005; damping=0.707

fo_compensation Blind frequency-offset estimation from the spectral peak of the M-th power (M = 4 for QPSK / square QAM), joint over polarizations, and correction.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

power=4; estimation_bandwidth=0.75

frame_sync Preamble-based frame synchronization.

in: in:SYMBOLS out: out:SYMBOLS

search=32; correct_cfo=True

ofdm_sync Blind OFDM timing and fractional CFO synchronization from the cyclic prefix (van de Beek).

in: in:SAMPLES out: out:SAMPLES

max_delay=auto; backoff=auto; correct_cfo=True

phase_recovery Feed-forward carrier phase recovery for large laser linewidths.

in: in:SYMBOLS, ref:SYMBOLS out: out:SYMBOLS

method=bps; n_test_phases=32; half_window=16; n_ref=64; ref_start=600

preamble Prepend a known preamble and append guard symbols to every frame.

in: in:SYMBOLS out: out:SYMBOLS

length=64; guard=16; seed=2024

timing_recovery Symbol timing recovery: Gardner detector, cubic interpolator, second-order loop.

in: in:SAMPLES out: out:SYMBOLS

loop_bandwidth=0.01; damping=0.707; detector_gain=1; initial_estimate=True; estimate_symbols=128

A.6 RECEIVER

adaptive_mimo_equalizer Fractionally spaced 2x2 (NxN) butterfly FIR equalizer for coherent receivers.

in: in:SAMPLES, ref:SYMBOLS out: out:SYMBOLS

n_taps=15; step_size=0.001; train_step_size=0.01; algorithm=lms; n_train=512; sps=2

agc Automatic gain control: remove the mean (optional) and scale every frame and stream to the nominal symbol power (`power` metadata of the upstream mapper / line code).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

remove_dc=True; target_power=auto

linear_equalizer FIR equalizer designed from the known channel impulse response (`csi` input).

in: in:SYMBOLS, csi:CSI out: out:SYMBOLS

n_taps=15; method=mmse; delay=auto

lms_equalizer Adaptive (N)LMS FIR equalizer: trained on known symbols, then decision directed.

in: in:SYMBOLS, ref:SYMBOLS out: out:SYMBOLS

n_taps=11; step_size=0.1; normalized=True; n_train=200; decision_directed=True

mimo_detector Linear (ZF, MMSE) or maximum-likelihood MIMO detection with known channel.
in: `in:SYMBOLS, csi:CSI` out: `out:SYMBOLS`
`method`=mmse

ofdm_channel_estimator Estimate the channel frequency response on every subcarrier.
in: `in:GRID, csi:CSI` out: `out:CSI`
`method`=ls; `average_symbols`=False

ofdm_equalizer Extract the data subcarriers and equalize them (one-tap ZF per subcarrier).
in: `in:GRID, csi:CSI` out: `out:SYMBOLS`

one_tap_equalizer Coherent equalization of flat fading with known gains (`csi` input).
in: `in:SYMBOLS, csi:CSI` out: `out:SYMBOLS`
`method`=zf

A.7 MIMO

alamouti_decoder Linear Alamouti combining with known (block-fading) channel and any number of RX antennas.
in: `in:SYMBOLS, csi:CSI` out: `out:SYMBOLS`

alamouti_encoder Alamouti space-time block code: 2 transmit antennas, rate 1, full diversity.
in: `in:SYMBOLS` out: `out:SYMBOLS`

stream_demux Spatial multiplexing: distribute symbols round-robin over `n_streams` antennas.
in: `in:SYMBOLS` out: `out:SYMBOLS`
`n_streams`=2

stream_mux Inverse of `stream_demux`: interleave the streams back into one sequence.
in: `in:SYMBOLS` out: `out:SYMBOLS`

A.8 CODING

conv_encoder Feed-forward convolutional encoder of rate 1/n.
in: `in:BITS` out: `out:BITS`
`generators`=[171, 133]; `constraint_length`=7; `terminate`=True

crc_decoder Check and strip the CRC; reports the frame error rate detected by the CRC.
in: `in:BITS|LLR` out: `out:BITS, valid:FLAGS`
`polynomial`=crc16

crc_encoder Append CRC parity bits to every frame.
in: `in:BITS` out: `out:BITS`
`polynomial`=crc16

depuncturer Re-insert punctured positions as erasures (LLR = 0) before decoding.
in: `in:BITS|LLR` out: `out:LLR`
`pattern`=[1, 1, 1, 0, 0, 1]

hamming_decoder Syndrome decoding (hard decision; LLR inputs are sliced first).
in: `in:BITS|LLR` out: `out:BITS`
`m`=3

hamming_encoder Systematic Hamming encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

m=3

interleaver Permute the bits (or LLRs) of every frame.

in: in:BITS|LLR|SYMBOLS out: out:same as 'in'

kind=random; rows=16; seed=1234; inverse=False

ldpc_decoder Belief-propagation LDPC decoder (normalized min-sum or sum-product) with early stopping.

in: in:BITS|LLR out: out:BITS

k_blocks=8; m_blocks=8; z=48; col_weight=3; seed=1; max_iter=30; method=min_sum; alpha=0.8

ldpc_encoder Systematic quasi-cyclic LDPC encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

k_blocks=8; m_blocks=8; z=48; col_weight=3; seed=1

polar_decoder Successive-cancellation polar decoder (soft LLR or hard input).

in: in:BITS|LLR out: out:BITS

block_length=256; k=128; design_snr_db=0

polar_encoder Polar encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

block_length=256; k=128; design_snr_db=0

puncturer Remove coded bits according to a periodic pattern (raises the code rate).

in: in:BITS out: out:BITS

pattern=[1, 1, 1, 0, 0, 1]

repetition_decoder Soft (LLR sum) or hard (majority vote) repetition decoding.

in: in:BITS|LLR out: out:BITS

n=3

repetition_encoder Repeat every bit `n` times (rate 1/n).

in: in:BITS out: out:BITS

n=3

scrambler XOR every frame with the same LFSR sequence.

in: in:BITS|LLR out: out:same as 'in'

taps=[7, 4]; seed=93

viterbi_decoder Maximum-likelihood sequence decoder; soft decisions with LLR input, hard with bits.

in: in:BITS|LLR out: out:BITS

generators=[171, 133]; constraint_length=7; terminate=True

A.9 WIRELINE

copper_line Cable channel with frequency-dependent loss (twisted pair, coax, PCB trace).

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

preset=cat5e; length_m=100; a=auto; b=auto; max_taps=256

crosstalk Near-end (NEXT) or far-end (FEXT) crosstalk from `n_disturbers` similar pairs.

in: in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES out: out:same as 'in'

model=next; n_disturbers=10; preset=dsl_0.4mm; length_m=1000

ctle Continuous-time linear equalizer: H(s) = A (1 + s/wz) / ((1 + s/wp1)(1 + s/wp2)).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

zero_hz=1e+09; pole1_hz=6e+09; pole2_hz=2.5e+10; dc_gain_db=0

dfe_equalizer Adaptive feed-forward + decision-feedback equalizer (NLMS), symbol spaced.

in: `in:SYMBOLS, ref:SYMBOLS` out: `out:SYMBOLS`

`ffe_taps`=7; `dfe_taps`=4; `step_size`=0.05; `n_train`=300

dmt_demodulator Remove the cyclic prefix, FFT, one-tap frequency-domain equalization and per-tone demapping.

in: `in:SAMPLES, csi:CSI` out: `out:BITS`

dmt_modulator Discrete multitone (ADSL/VDSL): real IFFT with Hermitian symmetry, per-tone bit loading.

in: `in:BITS` out: `out:SAMPLES`

`n_fft`=512; `cp_len`=40; `first_tone`=33; `last_tone`=255; `sample_rate`=2.208e+06; `loading`=waterfill; `uniform_bits`=4; `max_bits`=14; `gap_db`=9.8; `margin_db`=3; `snr_db`=60; `line_preset`=dsl_0.4mm; `line_length_m`=2000

echo_canceller NLMS adaptive filter estimating the echo from the local `tx` signal and subtracting it.

in: `in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`n_taps`=16; `train_step_size`=0.05; `step_size`=0.002; `train_samples`=1000

hybrid_echo Leakage of the local transmitter into the receiver through the hybrid (full duplex).

in: `in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`echo_db`=-10; `taps`=[1.0, 0.6, 0.3, 0.1]

impulse_noise Bursts of strong Gaussian noise (electrical switching, appliances).

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`burst_probability`=0.001; `burst_samples`=32; `power_db`=10

jitter Sampling-clock jitter: random (RJ, per symbol) and sinusoidal (SJ), in unit intervals.

in: `in:SAMPLES` out: `out:same as 'in'`

`rj_rms_ui`=0.02; `sj_amplitude_ui`=0; `sj_frequency_hz`=1e+06

line_block_decoder Inverse of `line_block_encoder`; counts invalid code words / sync headers.

in: `in:BITS|LLR` out: `out:BITS`

`code`=64b66b

line_block_encoder 4B/5B (FDDI, 100BASE-TX) or 64B/66B (10G/100G Ethernet) block line code.

in: `in:BITS` out: `out:BITS`

`code`=64b66b

line_decoder Hard decisions for NRZ / Manchester / AMI / MLT-3 symbols (the real part is used).

in: `in:SYMBOLS|SAMPLES` out: `out:BITS`

`code`=nrz; `threshold`=0.5

line_encoder Baseband line code: bits -> real symbols.

in: `in:BITS` out: `out:SYMBOLS`

`code`=nrz; `symbol_rate`=1e+09

A.10 OPTICAL

cd_compensation Frequency-domain chromatic dispersion compensation (inverse all-pass filter).

in: `in:SAMPLES` out: `out:SAMPLES`

`dispersion_ps_nm`=auto; `wavelength_nm`=auto

coherent_receiver Coherent front-end: local oscillator, 90-degree hybrid and balanced photodiodes.

in: in:SAMPLES out: out:SAMPLES

responsivity=0.8; thermal_noise_pa_rthz=20; shot_noise=True; lo_power_dbm=13; lo_linewidth_hz=100000; frequency_offset_hz=0

fiber Single-mode fiber span: attenuation, chromatic dispersion, optional Kerr nonlinearity and PMD.

in: in:SAMPLES out: out:SAMPLES

attenuation_db_km=0.2; dispersion_ps_nm_km=17; nonlinear=False; gamma_w_km=1.3; steps_per_km=0.5; pmd_ps_sqrt_km=0; polarization_rotation=False; length_km=80

fso_turbulence Free-space optical scintillation: log-normal (weak) or gamma-gamma (moderate/strong).

in: in:SAMPLES out: out:SAMPLES, csi:CSI

model=lognormal; log_intensity_variance=0.1; alpha=4; beta=2

intensity_modulator Laser with intensity modulation (directly modulated laser / EML) for IM/DD links.

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

power_dbm=0; extinction_ratio_db=10; rin_db_hz=auto; chirp_alpha=0; linewidth_hz=0; wavelength_nm=1550

iq_modulator Laser + IQ Mach-Zehnder modulator for coherent transmission (single or dual polarization).

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

power_dbm=0; model=linear; drive_swing=0.5; linewidth_hz=100000; wavelength_nm=1550

optical_amplifier Erbium-doped fiber amplifier with amplified spontaneous emission (ASE) noise.

in: in:SAMPLES out: out:SAMPLES

mode=power; gain_db=20; target_power_dbm=0; noise_figure_db=5

optical_link Multi-span amplified link: `n_spans` x (fiber span + EDFA compensating the span loss).

in: in:SAMPLES out: out:SAMPLES

attenuation_db_km=0.2; dispersion_ps_nm_km=17; nonlinear=False; gamma_w_km=1.3; steps_per_km=0.5; pmd_ps_sqrt_km=0; polarization_rotation=False; n_spans=10; span_km=80; noise_figure_db=5; launch_power_dbm=auto

photodiode Direct detection (PIN or APD): square-law current with shot and thermal noise.

in: in:SAMPLES out: out:SAMPLES

responsivity=0.8; thermal_noise_pa_rthz=20; shot_noise=True; dark_current_na=5; apd_gain=1; apd_k=0.2

wdm_demultiplexer Select one WDM channel with an optical band-pass filter (rectangular or 2nd-order super-Gaussian).

in: in:SAMPLES out: out:SAMPLES

channel=0; bandwidth_ghz=auto; shape=supergaussian

wdm_multiplexer Wavelength-division multiplexing: add neighbouring channels around the channel under test.

in: in:SAMPLES out: out:SAMPLES

n_channels=5; spacing_ghz=50; interferer_power_db=0

A.11 METRICS

ber_meter Count bit, symbol and frame errors between transmitted and received bits.

in: `tx:BITS, rx:BITS|LLR` out: `-`

`bits_per_symbol=auto; skip_bits=0`

evm_meter Error vector magnitude and modulation error ratio between reference and received symbols.

in: `ref:SYMBOLS, in:SYMBOLS` out: `-`

`skip_symbols=0`

optical_power_meter Average optical power (dBm) and OSNR (0.1 nm) of an optical signal.

in: `in:SAMPLES` out: `-`

power_meter Average power and peak-to-average power ratio of a waveform.

in: `in:SYMBOLS|SAMPLES` out: `-`

Chapter B

Metadata keys

Metadata travels with every signal (Section 2.8). The table lists the keys used by the built-in library, the blocks that set them and the blocks that read them.

Key	Meaning	Set by / read by
`bits_per_symbol`	bits per constellation symbol k	mapper, line encoder / awgn (E_b/N_0), BER meter
`constellation`	the `Constellation` object	mapper / demapper, equalizers (decisions), PLL, BPS, AGC, WDM
`code_rate`	cumulative code rate R (with tails, CRC, puncturing)	encoders, CRC, puncturer, block line codes / awgn
`power`	nominal mean power per sample	mapper, pulse shaper, OFDM/DMT, optical blocks / awgn (SNR reference), AGC, OSNR
`sps`	samples per symbol N_{sps}	pulse shaper, rate changers, OFDM (N/N_{act}) / awgn, matched filter, timing recovery, equalizers
`delay`	known processing delay in samples	pulse shaper, matched filter, FIR / matched filter, timing recovery, MIMO equalizer
`pulse_taps`	transmit pulse	pulse shaper / matched filter, WDM multiplexer
`n_symbols`	symbols per frame	mapper, pulse shaper, preamble, MIMO / matched filter, equalizers
`noise_var`	noise variance per complex sample or symbol (scalar or array)	awgn, equalizers, detectors, photodiode, coherent receiver / soft demapper, AGC, MIMO detector
`spatial_rate`	streams per channel use N_s	stream demux / awgn
`csi_type`	form of CSI: `gain`, `cir`, `cir_tv`, `freq`, `mimo`	channels, estimators / equalizers, detectors
`ofdm`	`OFDMLayout`	OFDM modulator / demodulator, estimator, equalizer, sync
`n_data_symbols`, `data_power`	payload size and data-subcarrier power	OFDM modulator / OFDM equalizer
`preamble`, `n_payload_symbols`	preamble sequence and payload size	preamble / frame sync
`punctured_length`	mother-code length before puncturing	puncturer / depuncturer
`line_code`	line code name	line encoder / AGC (target power)
`dmt`, `n_bits`	DMT modulator and payload size	DMT modulator / DMT demodulator
`optical`	the signal is an optical field	optical transmitters / detectors
`wavelength_nm`	carrier wavelength	optical transmitters / fiber, EDFA, CD and FO compensation
`cd_ps_nm`	accumulated dispersion $D\ell$	fiber, link / CD compensation

Key	Meaning	Set by / read by
`cd_compensated_ps_nm`	dispersion removed so far	CD compensation / FO compensation (bulk-delay fix)
`ase_psd`	accumulated ASE density per polarization	EDFA, link, fiber / OSNR metrics
`wdm`	channel count and spacing	WDM multiplexer / WDM demultiplexer

Chapter C

Scenario library

The 36 scenario files shipped in `backend/scenarios/` (subsystem files excluded) are listed below. The table is generated from the files themselves. Open them from the GUI scenario browser (Ctrl+O) or run them with `python -m linkstudio run <file>`. Each scenario's `description` field explains what to observe and which parameters to sweep.

Category	File	Name	Blocks	Mode
Tutorials	`tutorials/01_bpsk_awgn.yaml`	1 - BPSK over AWGN	5	simulate
Tutorials	`tutorials/02_qam_soft_demapping.yaml`	2 - 16-QAM and soft demapping	6	simulate
Tutorials	`tutorials/03_pulse_shaping.yaml`	3 - Pulse shaping and the eye diagram	8	simulate
Tutorials	`tutorials/04_rayleigh_fading.yaml`	4 - Rayleigh fading	7	simulate
Tutorials	`tutorials/05_coding_gain.yaml`	5 - Coding gain (two branches)	11	simulate
Tutorials	`tutorials/06_isi_equalization.yaml`	6 - ISI and adaptive equalization	7	simulate
Wireless	`wireless/alamouti_2x1.yaml`	Alamouti 2x1 transmit diversity	8	simulate
Wireless	`wireless/lte_like_ofdm.yaml`	LTE-like OFDM, mobile channel	14	simulate
Wireless	`wireless/mimo_2x2_smux.yaml`	2x2 MIMO spatial multiplexing	9	simulate
Wireless	`wireless/ofdm_eva_ls.yaml`	OFDM over EVA multipath	11	simulate
Wireless	`wireless/ofdm_synchronization.yaml`	OFDM timing and CFO synchronization	12	simulate
Wireless	`wireless/satellite_8psk.yaml`	Satellite link, 8-PSK with TWTA	12	simulate
Wireless	`wireless/single_carrier_16qam.yaml`	RRC 16-QAM link	11	simulate
Wireless	`wireless/synchronized_receiver.yaml`	Synchronized QPSK receiver	14	simulate
Wireless	`wireless/wifi_like_ofdm.yaml`	Wi-Fi-like OFDM (802.11a/g)	17	simulate
Coding	`coding/crc_conv_fading.yaml`	CRC + convolutional code over fading	13	simulate
Coding	`coding/hamming_block_code.yaml`	Hamming code, hard decisions	7	simulate
Coding	`coding/ldpc_qpsk.yaml`	LDPC-coded QPSK	7	simulate
Coding	`coding/polar_sc.yaml`	Polar code with SC decoding	7	simulate
Coding	`coding/punctured_conv.yaml`	Punctured convolutional code (rate 3/4)	9	simulate
Wireline	`wireline/10gbase_kr_backplane.yaml`	10GBASE-KR backplane	13	simulate
Wireline	`wireline/adsl_dmt.yaml`	ADSL-like DMT over a telephone pair	7	simulate
Wireline	`wireline/ethernet_100base_tx.yaml`	Fast Ethernet 100BASE-TX	14	simulate
Wireline	`wireline/line_code_spectra.yaml`	Line code spectra	9	simulate
Wireline	`wireline/serdes_pam4_backplane.yaml`	SerDes 25 GBd PAM-4 backplane	12	simulate

Category	File	Name	Blocks	Mode
Wireline	`wireline/vdsl_fext.yaml`	VDSL-like DMT with far-end crosstalk	7	simulate
Optical	`optical/apd_access_link.yaml`	O-band access link with APD	9	simulate
Optical	`optical/coherent_dp_16qam.yaml`	Coherent DP-16QAM metro link	17	simulate
Optical	`optical/coherent_dp_qpsk.yaml`	Coherent DP-QPSK long haul	17	simulate
Optical	`optical/fso_turbulence.yaml`	Free-space optical link with turbulence	8	simulate
Optical	`optical/imdd_pam4.yaml`	56 Gb/s PAM-4 IM/DD optical link	11	simulate
Optical	`optical/ook_receiver_sensitivity.yaml`	10G OOK receiver sensitivity	7	simulate
Optical	`optical/wdm_nonlinear.yaml`	WDM with fiber nonlinearity	19	simulate
Examples (CLI)	`coded_link_subsystems.yaml`	coded-16qam-subsystems	4	simulate
Examples (CLI)	`qpsk_awgn_sweep.yaml`	qpsk-awgn-sweep	5	sweep
Examples (CLI)	`rrc_16qam_impairments.yaml`	rrc-16qam-impairments	12	simulate

Two reusable subsystems, `subsystems/coded_qam_tx.yaml` and `subsystems/coded_qam_rx.yaml`, are used by `coded_link_subsystems.yaml`.

Bibliography

[1] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill, 2008.

[2] G. P. Agrawal, *Nonlinear Fiber Optics*, 6th ed. Academic Press, 2019.

[3] U. Mengali and A. N. D'Andrea, *Synchronization Techniques for Digital Receivers*. Plenum Press, 1997.

[4] K. Cho and D. Yoon, "On the general BER expression of one- and two-dimensional amplitude modulations," *IEEE Trans. Commun.*, vol. 50, no. 7, pp. 1074–1080, 2002.

[5] J. W. Craig, "A new, simple and exact result for calculating the probability of error for two-dimensional signal constellations," in *Proc. IEEE MILCOM*, 1991.

[6] A. A. M. Saleh, "Frequency-independent and frequency-dependent nonlinear models of TWT amplifiers," *IEEE Trans. Commun.*, vol. 29, no. 11, pp. 1715–1720, 1981.

[7] F. M. Gardner, "A BPSK/QPSK timing-error detector for sampled receivers," *IEEE Trans. Commun.*, vol. 34, no. 5, pp. 423–429, 1986.

[8] M. Oerder and H. Meyr, "Digital filter and square timing recovery," *IEEE Trans. Commun.*, vol. 36, no. 5, pp. 605–612, 1988.

[9] T. M. Schmidl and D. C. Cox, "Robust frequency and timing synchronization for OFDM," *IEEE Trans. Commun.*, vol. 45, no. 12, pp. 1613–1621, 1997.

[10] J.-J. van de Beek, M. Sandell, and P. O. Börjesson, "ML estimation of time and frequency offset in OFDM systems," *IEEE Trans. Signal Process.*, vol. 45, no. 7, pp. 1800–1805, 1997.

[11] S. M. Alamouti, "A simple transmit diversity technique for wireless communications," *IEEE J. Sel. Areas Commun.*, vol. 16, no. 8, pp. 1451–1458, 1998.

[12] E. Arıkan, "Channel polarization: a method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, 2009.

[13] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.

[14] T. Starr, J. M. Cioffi, and P. J. Silverman, *Understanding Digital Subscriber Line Technology*. Prentice Hall, 1999.

[15] D. N. Godard, "Self-recovering equalization and carrier tracking in two-dimensional data communication systems," *IEEE Trans. Commun.*, vol. 28, no. 11, pp. 1867–1875, 1980.

[16] T. Pfau, S. Hoffmann, and R. Noé, "Hardware-efficient coherent digital receiver concept with feedforward carrier recovery for M-QAM constellations," *J. Lightw. Technol.*, vol. 27, no. 8, pp. 989–999, 2009.

[17] S. J. Savory, "Digital coherent optical receivers: algorithms and subsystems," *IEEE J. Sel. Topics Quantum Electron.*, vol. 16, no. 5, pp. 1164–1179, 2010.

[18] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[19] R. H. Clarke, "A statistical theory of mobile-radio reception," *Bell Syst. Tech. J.*, vol. 47, no. 6, pp. 957–1000, 1968.

[20] 3GPP TS 36.104, *Evolved Universal Terrestrial Radio Access (E-UTRA); Base Station (BS) radio transmission and reception*, Annex B.

[21] ANSI T1.417-2001, *Spectrum Management for Loop Transmission Systems.*